

JAVA SDE-2 INTERVIEW PREPARATION SERIES

JAVA ENGINEERING - BOOK 01 OF 09 - STUDY STEP JAVA-01

Java Foundations for Problem Solving

Java Fundamentals for DSA and SDE-2 Interviews

BY

Vinay Reddy Kalluri

Restart from the first Java program, master language and collection behavior, then prove interview readiness through executable examples, traps, debugging, and focused practice.

JAVA BASICS | TYPES | OBJECTS | COLLECTIONS | TRAPS | INTERVIEWS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to **Book 01 in the Java Engineering segment**. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume is the canonical Java language foundation and the recommended first book in the series. It starts with source-to-execution and program structure, builds through types, control flow, methods, arrays, strings, objects, core libraries, exceptions, generics, and usage-first collections, then converts that knowledge into interview-quality implementation and debugging skill.

Readiness map

Decision	Evidence
START HERE	A correct algorithm is failing because of Java semantics; Primitive, reference, equality, or pass-by-value behavior is unclear; Core List, Set, Map, Queue, Deque, or PriorityQueue usage is not automatic; The interviewer asks for cleaner, safer, or more testable Java.
PREPARE FIRST	No prior Java is required; A Java 21 JDK is useful for running the examples; Curiosity and willingness to dry-run small programs.
MOVE ON WHEN	Explain Java program execution and write compiling Java from the first program onward; Use types, control flow, methods, arrays, strings, objects, exceptions, generics, and core collections precisely; Diagnose common Java interview traps and write safe comparators and numeric expressions; Continue to Time and Space Complexity with enough Java fluency to analyze real code.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Java Segment Position

START OF SEGMENT	YOU ARE HERE JAVA 01 - Java Problem-Solving Foundations	JAVA 02 - Git and GitHub
------------------	---	--------------------------

A note from Vinay

When Java feels rusty, do not rush toward clever syntax. Rebuild one behavior at a time, run it, and explain why the output is correct before you move on.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	5
Chapter 1 - Java and Program Foundations	5
Chapter 2 - Values, Variables, Types, and Literals	10
Chapter 3 - Operators, Conversions, and Expressions	15
Chapter 4 - Conditions, Loops, and Index Boundaries	20
Chapter 5 - Methods and Java's Pass-by-Value Rule	26
Chapter 6 - Arrays from First Principles	33
Chapter 7 - Strings, Characters, and StringBuilder	39
Chapter 8 - Practice Lab A - Language and Execution	45
Chapter 9 - Classes, Objects, and Constructors	51
Chapter 10 - Object-Oriented Foundations in Interview-Sized Steps	58
Chapter 11 - Practice Lab B - Arrays, Strings, and Objects	65
Chapter 12 - Wrapper Classes, Basic Generics, and Enums	71
Chapter 13 - Java Collections Basics for SDE-2 Interviews	77
Chapter 14 - Exceptions, Input, Output, and Everyday Utilities	87
Chapter 15 - Practice Lab C - Wrappers, Generics, Collections, Exceptions, I/O	95
Chapter 16 - Writing Interview-Quality Java	101

CONTENTS NAVIGATION

Chapter 17 - Forty Java Interview Traps	109
Chapter 18 - Practice Lab D - Traps, Refactoring, and Readiness	121
Chapter 19 - Solution Studio A - Language and Execution	127
Chapter 20 - Solution Studio B - Arrays, Strings, and Objects	133
Chapter 21 - Solution Studio C - Wrappers, Generics, Collections, Exceptions, I/O	138
Chapter 22 - Solution Studio D - Traps and Readiness	143
Chapter 23 - Java Fundamentals Wave 1: Complete Solution Studio	148
Chapter 24 - Java Syntax and Language Quick Reference	153
Chapter 25 - Seventy Compiling Java Fundamentals Examples	164

Part II - Practice and Handoff**185**

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Java and Program Foundations

Learning objectives

By the end of this chapter, you can explain why Java is useful in coding interviews, trace a source file through compilation and execution, read the smallest complete program, and distinguish compile-time, runtime, and logical failures.

Why Java works well for DSA interviews

Java makes many contracts visible. A variable has a declared type, array indexes are checked, and the standard library provides lists, sets, maps, queues, deques, heaps, sorting, parsing, and numeric utilities. Automatic memory management removes manual deallocation from ordinary interview code. These properties let a candidate concentrate on the invariant and the algorithm while still writing code that is explicit enough to review.

The trade-off is ceremony. Java solutions are usually longer than equivalent Python solutions, primitive and reference behavior must be understood, and a careless choice such as `int` instead of `long` can silently overflow. Good interview Java is intentionally plain: familiar APIs, descriptive names, small helpers, explicit boundary handling, and no abstraction that does not clarify the solution.

Java is not "always faster," and platform independence is not absolute. `javac` normally produces platform-neutral class files, and a compatible JVM translates their operations for a host. Native libraries, file systems, default charsets, environment variables, timing, and resource limits can still make a program environment-dependent.

Source, bytecode, and execution

TEXT

```
Hello.java source
      |
      | javac Hello.java
      v
Hello.class bytecode
      |
      | java Hello
      v
JVM execution on the host machine
```

- The **JDK** contains development tools such as `javac`, the `java` launcher, and a runtime.
- The **JVM** is the execution engine for class files.

- **JRE** remains useful as the conceptual name for a JVM plus the libraries and support files needed to run Java. Modern deployments often ship a JDK or a custom runtime image rather than a separately branded JRE.

Keep this mental model high level here. Class loading, JIT compilation, garbage collectors, and stack-frame internals belong in the JVM volume.

The first complete program

File: `HelloInterview.java`

JAVA

```
public class HelloInterview {  
    public static void main(String[] args) {  
        System.out.println("Ready for Java");  
    }  
}
```

Expected output:

TEXT

Ready for Java

Compile and run:

BASH

```
javac HelloInterview.java  
java HelloInterview
```

Read it from the outside inward:

- 1 `public class HelloInterview` declares a class named `HelloInterview`. A public top-level class normally lives in a file with the same name.
- 2 `{` and `}` delimit the class body and method body.
- 3 `main` is the entry method used by the launcher.
- 4 `public` lets the launcher access it.
- 5 `static` means the launcher does not need to construct `HelloInterview` first.
- 6 `void` means the method returns no value.
- 7 `String[] args` receives command-line arguments as an array of strings.
- 8 `System.out.println(...)` calls a method that writes a line.
- 9 The semicolon terminates the statement.

Do not memorize the signature as unexplained punctuation. Read it as: "an accessible class method named main, returning nothing, that accepts an array of text arguments."

Statements, blocks, and comments

A **statement** performs an action. A **block** groups statements and creates a scope. Java ignores most whitespace, but consistent indentation exposes structure.

JAVA

```
int score = 72;           // end-of-line comment
if (score >= 70) {        // block begins
    System.out.println("pass");
}

/* A block comment can span lines. */
```

Use comments to explain a decision or invariant, not to translate obvious syntax. In interviews, `sum += number; // add number to sum` adds noise; `// sum equals numbers[0..index)` records useful reasoning.

Three failure stages

Failure	When found	Example	Repair direction
Compile-time error	before a class file is produced	missing semicolon, incompatible type, uninitialized local	repair Java syntax or type rules
Runtime error	while the program executes	null dereference, invalid index, division by zero	validate state or correct the execution path
Logical error	program runs but result is wrong	loop skips the last element	repair the invariant, condition, or formula

Compile-time-error demonstration:

JAVA

```
int answer;
System.out.println(answer); // local variable answer might not have been initialized
```

Corrected version:

JAVA

```
int answer = 0;
System.out.println(answer);
```

Fields receive specified default values when an object or class is initialized. Local variables do not; a local must be definitely assigned before it is read.

TRACE IT | A beginner execution dry run

JAVA

```
public class SumArguments {  
    public static void main(String[] args) {  
        int sum = 0;  
        for (String argument : args) {  
            sum += Integer.parseInt(argument);  
        }  
        System.out.println(sum);  
    }  
}
```

For `java SumArguments 4 7 9`:

- 1 `args` refers to `{"4", "7", "9"}`.
- 2 `sum` starts at 0.
- 3 Each string is parsed as an `int`; `sum` becomes 4, then 11, then 20.
- 4 The loop ends and the program prints `20`.

`Integer.parseInt` can throw `NumberFormatException`; later chapters define parsing and exception contracts. The point here is the execution sequence: initialize, repeat, update state, produce output.

WATCH OUT | Interview angle and common mistakes

- Saying "write once, run anywhere" without the compatible-JVM and environment qualification.
- Calling bytecode machine code.
- Saying the JDK, JRE, and JVM are interchangeable names.
- Omitting the filename/class-name relationship for a public top-level class.
- Claiming local variables automatically start at zero.
- Copying a long `main` signature from memory without being able to explain each token.
- Adding console-input code when the interview platform already provides a method signature.

An interviewer may ask where a failure occurs. Answer with evidence: the compiler rejects an incompatible assignment; the JVM throws an exception on a reached bad operation; a wrong answer with no error is a logical defect.

Quick check

- 1 What artifact does `javac` normally produce?
- 2 Why is `main` static?
- 3 Does platform independence guarantee identical operating-system behavior?
- 4 Why can a field be read at its default value while an unassigned local cannot?
- 5 Which failure category describes an off-by-one result with no exception?

Small practice

Foundation: Write `Greeting.java` so `java Greeting Vinay` prints `Hello, Vinay`. Handle an empty `args` array.

Interview Core: Predict which of four lines in a supplied program fail at compilation, runtime, or logically, and justify the stage.

SDE-2 Follow-up: Explain what a build using JDK 21 and `javac --release 17` does and does not guarantee.

Cross-book boundary

Use this chapter's execution model for the rest of [JAVA-01](#). Continue to [JAVA-06](#) for class loading, JIT, garbage collection, stack frames, and memory diagnostics. Continue to [JAVA-04](#) for modules, packaging, and language evolution in depth.

RECAP | Chapter summary

Java interview code is valuable because types and library contracts are explicit. Source is compiled to class files and executed by a compatible JVM; that portability has environmental limits. A complete program is a class containing an accessible static `main`, and failures must be located at compile time, runtime, or in logic before they can be repaired.

SDE-2 CORE CHAPTER

Chapter 2 - Values, Variables, Types, and Literals

This is where Java starts to feel predictable. A variable has a type, the type limits which values it can hold, and the spelling of a literal determines how Java begins evaluating an expression.

A note from Vinay: When a candidate says, "I knew the algorithm, but Java gave the wrong number," the cause is often in this chapter: an uninitialized local, the wrong primitive, or arithmetic that overflowed before it reached a `long`.

Start with one value

JAVA

```
int solvedProblems = 12;
```

Read this from right to left:

- `12` is an integer literal.
- `int` is the declared type.
- `solvedProblems` is the variable name.
- `=` initializes the variable with the value.

Later assignment changes the value stored in that variable:

JAVA

```
solvedProblems = 13;
```

Use a name that carries meaning. `solvedProblems` is easier to review than `x` unless the variable is a conventional short-lived index.

The eight primitive types

Type	Width	Value model	Common interview use
<code>byte</code>	8 bits	signed integer	encoded data; rarely arithmetic
<code>short</code>	16 bits	signed integer	uncommon in DSA code
<code>int</code>	32 bits	signed integer	indexes, counts, ordinary values
<code>long</code>	64 bits	signed integer	sums, products, timestamps, wide ranges
<code>float</code>	32 bits	binary floating point	uncommon in interview algorithms
<code>double</code>	64 bits	binary floating point	averages and approximate measurements
<code>char</code>	16 bits	UTF-16 code unit	basic character processing
<code>boolean</code>	JVM-defined storage, language values <code>true/false</code>	logical state	predicates and flags

Important ranges:

TEXT
<pre>byte : -128 to 127 short : -32,768 to 32,767 int : -2^31 to 2^31 - 1 long : -2^63 to 2^63 - 1 char : 0 to 65,535</pre>

Java specifies these value ranges. It does not promise that every local variable occupies exactly its primitive width in a physical stack frame.

Primitive values and references are different

TEXT
<pre>int score = 90; score stores the value 90 String name = new String("Ada"); name stores a reference value</pre>

A reference can designate an object or be `null`. It does not contain the entire object. Copying a primitive copies its value. Copying a reference copies the reference value, so both variables can designate the same object.

Declaration, initialization, assignment, and scope

JAVA

```
public class ScopeExample {  
    private int completed;           // field: default value 0  
  
    int next(boolean includeBonus) {  
        int bonus;                  // local: no automatic usable default  
        if (includeBonus) {  
            bonus = 1;  
        } else {  
            bonus = 0;  
        }  
        int result = completed + bonus;  
        return result;  
    }  
}
```

Fields and array elements receive default values. Local variables must be definitely assigned on every path before Java allows a read.

Scope is the region where a name is available. Prefer the smallest useful scope; a loop counter normally belongs inside the loop declaration.

Literals you must recognize

JAVA

```
int decimal = 42;  
long distance = 3_000_000_000L;  
double ratio = 0.75;  
float sample = 0.75F;  
char grade = 'A';  
String topic = "arrays";  
boolean ready = true;  
int binary = 0b101010;  
int hexadecimal = 0x2A;
```

Uppercase **L** and **F** make the literal type visible. Underscores improve readability but do not change the value.

Why overflow can happen before assignment

JAVA

```
long incorrect = 100_000 * 100_000;  
long correct = 100_000L * 100_000;
```

In the first line both operands are `int`, so Java performs 32-bit multiplication first. The wrapped `int` result is widened only afterward. In the second line one operand is `long`; numeric promotion makes the multiplication itself 64-bit.

Complete example

File: ValuesAndTypesExample.java

JAVA

```
public final class ValuesAndTypesExample {
    public static void main(String[] args) {
        int solvedProblems = 12;
        long safeProduct = 100_000L * 100_000;
        char digitCharacter = '7';
        boolean ready = solvedProblems >= 10;

        System.out.println(solvedProblems);
        System.out.println(safeProduct);
        System.out.println(digitCharacter);
        System.out.println(ready);
    }
}
```

Expected output:

TEXT

```
12
100000000000
7
true
```

The matching executable companion also asserts minimum and maximum primitive values and the difference between a field default and local definite assignment.

Edge-case matrix

Case	What Java does	Interview response
unassigned local is read	compile-time error	initialize every reachable path
field is not explicitly assigned	receives a specified default	do not confuse this with local variables
integer result exceeds range	wraps without an exception	widen before arithmetic or validate
<code>final</code> reference points to mutable object	reference cannot change; object might	<code>final</code> is not deep immutability
reference is <code>null</code>	no object is designated	validate before dereference
<code>char</code> holds an emoji	many emoji need two code units	do not promise one visible character per <code>char</code>

Interview room

Interviewer: Why does assigning an expression to `long` not guarantee safe arithmetic?

Model answer: Assignment conversion occurs after the expression is evaluated. If every operand is `int`, operations such as multiplication happen as `int` and may overflow first. I make an operand `long`, for example `left * (long) right`, when the operation needs a 64-bit range.

Follow-up: Do local variables receive zero by default?

Model answer: No. Fields and array elements are default-initialized. A local variable must be definitely assigned before it is read, or compilation fails.

Practice

- 1 **Foundation:** Declare one variable of every primitive type and print it.
- 2 **Foundation:** Predict the value of `long result = Integer.MAX_VALUE + 1;`.
- 3 **Interview Core:** Repair an `int` overflow in the area of a large rectangle.
- 4 **Debugging:** Explain why `int value; System.out.println(value);` does not compile inside a method.
- 5 **SDE-2 Follow-up:** Define the input range that makes `int` safe for a count and explain when you would still choose `long`.

Chapter takeaway

Choose a type from the value domain, not habit. Remember that expression types determine arithmetic before assignment, locals need definite assignment, and references designate objects rather than containing them.

SDE-2 CORE CHAPTER

Chapter 3 - Operators, Conversions, and Expressions

An expression is a small program: Java evaluates operands, applies conversions, performs the operation, and produces a value. Interview output questions become straightforward when you trace those four steps.

Arithmetic without surprises

JAVA

```
int quotient = 7 / 2;      // 3
double precise = 7 / 2.0;  // 3.5
int remainder = -7 % 3;    // -1
```

Integer division truncates toward zero. Integral division by zero throws `ArithmeticException`; floating-point division by zero can produce infinity or NaN.

`+`, `-`, `*`, `/`, and `%` do not automatically check integral overflow.

Numeric promotion

Arithmetic on `byte`, `short`, and `char` normally promotes operands to `int`:

JAVA

```
byte left = 10;
byte right = 20;
int sum = left + right;
// byte invalid = left + right; // compile-time error: the expression is int
```

Binary numeric promotion chooses `double`, then `float`, then `long`, otherwise `int`, based on the operands.

Widening and narrowing

JAVA

```
int count = 10;
long total = count;           // widening

double measurement = 10.8;
int truncated = (int) measurement; // narrowing: 10
```

Widening usually needs no cast, but widening to floating point can lose precision. Narrowing needs a cast and may discard bits, truncate a fraction, or change the mathematical value.

The order of a cast matters:

JAVA

```
long correct = (long) Integer.MAX_VALUE + 1;
long incorrect = (long) (Integer.MAX_VALUE + 1);
```

The inner addition in the second line already overflowed.

Comparison and equality

<, <=, >, and >= compare numeric values. == and != compare primitive values or reference identity.

JAVA

```
String first = new String("java");
String second = new String("java");

System.out.println(first == second);      // false: distinct objects
System.out.println(first.equals(second)); // true: same content
```

Use `equals` or `Objects.equals` for object value comparison when the type defines that meaning.

Logical operators and short-circuiting

JAVA

```
if (text != null && !text.isEmpty()) {
    System.out.println(text);
}
```

If `text` is null, `&&` skips the right operand, preventing a dereference. `&` on booleans evaluates both operands. Similarly, `||` can skip its right operand while `|` does not.

Prefix and postfix

JAVA

```
int value = 5;
int before = value++; // before=5, value=6
int after = ++value;  // value=7, after=7
```

Both mutate the variable. Postfix produces the old value; prefix produces the updated value. Keep increments out of larger expressions when clarity matters.

Compound assignment hides a cast

JAVA

```
short value = 10;
value += 5;
```

This behaves roughly like:

JAVA

```
value = (short) (value + 5);
```

The left side is evaluated only once, but the implicit narrowing can hide overflow.

Ternary expressions

JAVA

```
int absolute = number >= 0 ? number : -number;
```

Use a ternary when it clearly selects one of two values. Nested ternaries usually make interview code harder to discuss. Remember that negating `Integer.MIN_VALUE` still overflows.

Precedence worth remembering

Higher to lower	Operators
unary	!, unary +, unary -, ++, --
multiplicative	*, /, %
additive	+, -
comparison	<, <=, >, >=
equality	==, !=
conditional AND/OR	&&, ^
ternary	?:
assignment	=, +=, -=, and others

Use parentheses when the grouping is not obvious. Interviewers value readable intent more than memorization of the complete precedence table.

Complete example

File: OperatorsAndConversionsExample.java

JAVA

```
public final class OperatorsAndConversionsExample {  
    public static void main(String[] args) {  
        int maximum = Integer.MAX_VALUE;  
        long correct = (long) maximum + 1;  
        double average = (8 + 9) / 2.0;  
        String text = null;  
        boolean safe = text != null && !text.isEmpty();  
  
        System.out.println(correct);  
        System.out.println(average);  
        System.out.println(safe);  
    }  
}
```

Expected output:

TEXT

```
2147483648  
8.5  
false
```

Edge-case matrix

Case	Failure	Correction
5 / 2 expected as 2.5	both operands are integral	make one operand floating point
cast after multiplication	overflow already happened	cast an operand before arithmetic
value != null & value.isEmpty()	right side still executes	use && for a null guard
compare object contents with ==	tests identity	use the type's value equality
comparator returns left - right	subtraction can overflow	use Integer.compare(left, right)
Math.abs(Integer.MIN_VALUE)	result remains negative	define a wider or exceptional contract

Interview room

Interviewer: Why does `short value = 10; value += 5;` compile while `value = value + 5;` does not?

Model answer: `value + 5` is promoted to `int`, so assigning it directly to `short` needs an explicit cast. Compound assignment includes an implicit conversion back to the left-side type and evaluates the left side once. That convenience can also hide narrowing.

Follow-up: What is the difference between `&&` and `&` for booleans?

Model answer: Both combine boolean values, but `&&` conditionally skips the right operand when the left is false. `&` always evaluates both operands. I use `&&` for guard conditions unless both evaluations are deliberately required.

Practice

- 1 **Foundation:** Predict `-7 / 3` and `-7 % 3`.
- 2 **Foundation:** Convert `double 19.99` to `int` and explain the lost information.
- 3 **Interview Core:** Repair overflow in `long area = width * height` when both inputs are `int`.
- 4 **Predict the output:** Trace `int x = 3; int y = x++ + ++x;`, then rewrite it clearly.
- 5 **SDE-2 Follow-up:** Define how an API should react when an exact arithmetic result does not fit its return type.

Chapter takeaway

Trace operand types before the operation, then the result, then assignment. That single habit explains promotion, integer division, overflow, casting, short-circuiting, and most output questions in this area.

SDE-2 CORE CHAPTER

Chapter 4 - Conditions, Loops, and Index Boundaries

Control flow decides which statements run and how often. The syntax is small; the real skill is expressing a boundary that is easy to verify.

Conditions start with a boolean question

JAVA

```
if (temperature < 0) {
    System.out.println("freezing");
} else if (temperature < 20) {
    System.out.println("cool");
} else {
    System.out.println("warm");
}
```

Java does not treat arbitrary integers or references as truthy. The condition must be **boolean**, or a **Boolean** that can be unboxed safely.

Use a guard clause when it makes the normal path flatter:

JAVA

```
static int firstValue(int[] values) {
    if (values == null || values.length == 0) {
        throw new IllegalArgumentException("values must not be empty");
    }
    return values[0];
}
```

Traditional switch and fall-through

JAVA

```
static String dayType(int day) {
    String result;
    switch (day) {
        case 6:
        case 7:
            result = "weekend";
            break;
        default:
            result = "weekday";
    }
    return result;
}
```


Colon-style cases can fall through. Sometimes that combines labels intentionally; missing `break` accidentally is a common interview trap.

Java 14 made switch expressions permanent:

JAVA

```
static String dayTypeModern(int day) {  
    return switch (day) {  
        case 6, 7 -> "weekend";  
        default -> "weekday";  
    };  
}
```

Label version-specific syntax when the interview target is older than Java 14.

Four loop forms

Use a `for` loop when initialization, continuation, and update naturally belong together:

JAVA

```
for (int index = 0; index < values.length; index++) {  
    System.out.println(values[index]);  
}
```

Use `while` when continuation is the central idea:

JAVA

```
while (node != null) {  
    node = node.next;  
}
```

Use `do-while` only when the body must run at least once. Use enhanced `for` when each value is needed and no index or structural mutation is required.

Index ranges should be spoken aloud

For an array of length `n`, valid indexes are `[0, n)`. That half-open interval gives two useful facts:

- zero is the first valid index;
- `n` is the number of elements and the first invalid index.

Forward traversal:

JAVA

```
for (int index = 0; index < values.length; index++) {  
    consume(values[index]);  
}
```

Reverse traversal:

JAVA

```
for (int index = values.length - 1; index >= 0; index--) {
    consume(values[index]);
}
```

For an empty array, reverse traversal starts at `-1` and performs no iteration.

WHY IT WORKS | Loop invariant and progress

An invariant is a statement that remains true before and after every completed iteration. Progress explains why the loop ends.

For a running sum:

JAVA

```
int sum = 0;
for (int index = 0; index < values.length; index++) {
    sum += values[index];
}
```

Invariant: before each iteration, `sum` equals the total of `values[0..index)`.

Progress: `index` increases by one and is bounded by `values.length`.

break, continue, and return

- `break` exits the nearest loop or switch statement.
- `continue` skips to the next loop iteration.
- `return` exits the method.

JAVA

```
static int firstNegative(int[] values) {
    for (int index = 0; index < values.length; index++) {
        if (values[index] < 0) {
            return index;
        }
    }
    return -1;
}
```

Labeled break and continue exist, but extracting a named helper is often clearer than deeply nested labels.

Two-dimensional traversal

Do not assume every row has the same length:

JAVA

```
for (int row = 0; row < matrix.length; row++) {  
    if (matrix[row] == null) {  
        continue;  
    }  
    for (int column = 0; column < matrix[row].length; column++) {  
        System.out.println(matrix[row][column]);  
    }  
}
```

This works for rectangular and jagged arrays and deliberately skips null rows.

Complete example

File: ControlFlowExample.java

JAVA

```
public final class ControlFlowExample {  
    static int firstAtLeast(int[] values, int threshold) {  
        for (int index = 0; index < values.length; index++) {  
            if (values[index] >= threshold) {  
                return index;  
            }  
        }  
        return -1;  
    }  
  
    public static void main(String[] args) {  
        int[] values = {3, 5, 8, 13};  
        System.out.println(firstAtLeast(values, 8));  
        System.out.println(firstAtLeast(values, 20));  
    }  
}
```

Expected output:

TEXT

```
2  
-1
```

Edge-case matrix

Case	Typical bug	Reliable check
empty input	assumes index zero exists	loop should perform zero iterations
last element	uses <code>index < length - 1</code>	use the stated half-open range
reverse loop	uses <code>index > 0</code>	include index zero with <code>>= 0</code>
<code>continue</code>	skips a required update	keep progress in the loop update or prove every path
nested rows	uses <code>matrix[0].length</code> for every row	inspect the current row
colon switch	missing <code>break</code>	label intentional fall-through
search miss	returns a valid-looking index	choose and document a sentinel or richer result

Interview room

Interviewer: How do you prove an index loop is correct?

Model answer: I state the valid range, an invariant describing processed and unprocessed elements, and a progress measure. For forward array traversal, indexes in `[0, index)` are processed, `index` increases by one, and the loop stops when `index == length`, so every valid index is visited once.

Follow-up: When would you use an enhanced `for` loop?

Model answer: When I need each element and do not need its index or structural mutation. For two-pointer movement, index replacement, or position reporting, I use an indexed loop.

Practice

- Foundation:** Print an array forward and backward.
- Foundation:** Sum only positive values using `continue`.
- Interview Core:** Traverse a jagged matrix that can contain null rows.
- Debugging:** Repair `for (int i = 0; i <= values.length; i++)`.
- SDE-2 Follow-up:** Write the invariant and termination proof for a loop that consumes paginated results.

Chapter takeaway

Treat every loop as a range plus an invariant plus progress. This is more dependable than remembering isolated patterns, and it prepares you for complexity analysis in the next book.

SDE-2 CORE CHAPTER

Chapter 5 - Methods and Java's Pass-by-Value Rule

A method gives a useful operation a name. In interview code, a good helper makes an invariant visible and lets the main solution read like a sequence of decisions.

The smallest useful method

JAVA

```
static int larger(int left, int right) {  
    if (left >= right) {  
        return left;  
    }  
    return right;  
}
```

- `static` lets the method be called without creating an object; instance methods come later.
- `int` before the name is the return type.
- `left` and `right` are parameters.
- `return` supplies the result and exits the method.

At a call site, `larger(4, 9)`, the expressions `4` and `9` are arguments.

Parameters, arguments, and local scope

Parameters are local variables initialized for one invocation. Variables declared inside the method are also local to that invocation.

JAVA

```
static int calculateSum(int[] numbers) {  
    int sum = 0;  
    for (int number : numbers) {  
        sum += number;  
    }  
    return sum;  
}
```

The method's contract should answer:

- Are null or empty inputs allowed?
- Can the method mutate the input?
- What does it return?

- Which failures can occur?
- What is its time and space cost?

void methods

A **void** method produces no return value:

JAVA

```
static void printSeparator() {  
    System.out.println("----");  
}
```

It can still use **return;** to exit early. Prefer returning computed information rather than printing from every helper; returned values are easier to test and reuse.

Java always passes argument values by value

This wording matters. Java has no pass-by-reference parameter mode.

Primitive argument

JAVA

```
static void increment(int number) {  
    number++;  
}  
  
int callerNumber = 7;  
increment(callerNumber);  
System.out.println(callerNumber); // 7
```

The parameter receives a copy of the primitive value. Updating that local copy cannot change the caller's variable.

Reference argument and shared mutation

JAVA

```
static final class Counter {  
    int value;  
}  
  
static void increment(Counter counter) {  
    counter.value++;  
}
```

The parameter receives a copy of the reference value. Caller and callee can use their reference copies to reach the same object, so mutation of that object is visible.

TEXT

```
caller variable ----+
                    +----> Counter{value=1}
parameter copy ----+
```

Reassigning the parameter

JAVA

```
static void replace(Counter counter) {
    counter = new Counter();
    counter.value = 99;
}
```

Reassignment changes only the callee's local parameter. It does not point the caller's variable at the new object.

Arrays follow the same rule

An array variable holds a reference. Passing it copies that reference value:

JAVA

```
static void markFirst(int[] values) {
    values[0] = -1; // mutates the shared array
    values = new int[0]; // reassigns only the parameter
}
```

Document mutation. A name such as `sortInPlace` is more honest than `sort` when the input changes.

Method overloading

Overloading uses the same method name with a different parameter list:

JAVA

```
static int clamp(int value, int minimum, int maximum) { ... }
static long clamp(long value, long minimum, long maximum) { ... }
```

Return type alone cannot distinguish overloads. Overload selection occurs at compile time. At the fundamentals level, prefer overloads whose intent remains obvious; complex boxing/varargs resolution belongs in the Advanced Java Language book.

Varargs as a convenient array

JAVA

```
static int sum(int... values) {  
    int total = 0;  
    for (int value : values) {  
        total += value;  
    }  
    return total;  
}
```

`int...` is represented as `int[]`. Only the final parameter may be variable-arity. The caller can supply separate arguments or an existing array. Treat the received array according to an explicit ownership contract.

Recursion preview

JAVA

```
static long factorial(int number) {  
    if (number < 0) {  
        throw new IllegalArgumentException("number must be nonnegative");  
    }  
    if (number <= 1) {           // base case  
        return 1;  
    }  
    return number * factorial(number - 1); // progress toward base case  
}
```

Every recursive method needs a base case and progress. Each unfinished call consumes invocation state; excessive depth can throw `StackOverflowError`. Full recursive problem solving belongs in the Recursion book.

Complete example

File: `MethodsAndPassByValueExample.java`

JAVA

```
public final class MethodsAndPassByValueExample {  
    static final class Person {  
        String name;  
  
        Person(String name) {  
            this.name = name;  
        }  
    }  
  
    static void rename(Person person) {  
        person.name = "Updated";  
    }  
  
    static void replace(Person person) {  
        person = new Person("Replacement");  
    }  
  
    public static void main(String[] args) {  
        Person person = new Person("Original");  
        rename(person);  
        System.out.println(person.name);  
        replace(person);  
        System.out.println(person.name);  
    }  
}
```

Expected output:

TEXT

```
Updated  
Updated
```

The first call mutates the shared object. The second reassigns only its local parameter.

Edge-case matrix

Case	Mistake	Better contract
primitive parameter	expects caller value to change	return the new value
reference parameter	calls it pass-by-reference	say a reference value is copied
array helper	silently mutates input	name/document mutation or return a copy
return type only differs	tries to overload	change the parameter list or method name
varargs input	assumes a fresh array always exists	caller may pass an existing array
recursive call	no base case or no progress	state both before coding
deep recursion	ignores stack limit	use an iterative form when appropriate

Interview room

Interviewer: Is Java pass-by-reference for objects?

Model answer: No. Java always passes values by value. For an object argument, the value being copied is a reference. Both copies can designate the same object, which explains visible mutation, but reassigning the parameter cannot reassign the caller's variable.

Follow-up: Can a method swap two caller variables?

Model answer: Not by reassigning two parameters. It can mutate a shared container or return the swapped values, but Java does not pass the caller's variable locations.

Practice

- 1 **Foundation:** Write a method with two parameters and an integer result.
- 2 **Predict:** Trace a primitive parameter, a mutable object parameter, and parameter reassignment.
- 3 **Debugging:** Repair a method that tries to swap two `String` variables by reassigning parameters.
- 4 **Interview Core:** Write `reverseCopy(int[])` without mutating the input.
- 5 **SDE-2 Follow-up:** Design a method contract stating nullability, mutation, result ownership, failure, and complexity.

Chapter takeaway

Every parameter is initialized from a copied argument value. Separate mutation of a shared object from reassignment of a local parameter, and use small helpers whose contracts are easy to test and explain.

SDE-2 CORE CHAPTER

Chapter 6 - Arrays from First Principles

Arrays are the bridge between Java syntax and most early DSA problems. Before learning two pointers or sliding windows, become comfortable with length, indexes, mutation, copying, and row boundaries.

Declaration, creation, and initialization

JAVA

```
int[] numbers;           // declares a reference variable
numbers = new int[4];     // creates an array with four zero values

int[] scores = {70, 85, 90};
```

An array is an object with a fixed length. `numbers.length` is 4 and valid indexes are 0 through 3.

Default element values are zero for numeric primitives, `false` for boolean, the zero code unit for `char`, and `null` for references.

Read and write by index

JAVA

```
int first = scores[0];
scores[1] = 88;
```

Java checks every reached array access. An invalid index throws `ArrayIndexOutOfBoundsException`. A null array reference throws `NullPointerException` before an element can be accessed.

Traverse safely

JAVA

```
for (int index = 0; index < scores.length; index++) {
    System.out.println(index + ": " + scores[index]);
}
```

Use enhanced `for` when only values are needed:

JAVA

```
for (int score : scores) {
    System.out.println(score);
}
```

Changing the enhanced-for variable does not replace the array element because the variable receives a copied value.

Arrays passed to methods

JAVA

```
static void fillWithZero(int[] values) {
    for (int index = 0; index < values.length; index++) {
        values[index] = 0;
    }
}
```

The copied reference still designates the caller's array, so element mutation is visible. Reassigning `values` inside the method would not replace the caller's variable.

Aliasing versus copying

JAVA

```
int[] original = {1, 2, 3};
int[] alias = original;
alias[0] = 9;
System.out.println(original[0]); // 9
```

Assignment copies the reference. To create new array storage:

JAVA

```
int[] copyA = original.clone();
int[] copyB = java.util.Arrays.copyOf(original, original.length);
int[] copyC = new int[original.length];
System.arraycopy(original, 0, copyC, 0, original.length);
```

For a primitive array, these copy primitive values. For an object array, they copy references, so element objects remain shared. That is a shallow copy.

Comparing and printing

JAVA

```
int[] left = {1, 2};
int[] right = {1, 2};

System.out.println(left == right); // false
System.out.println(java.util.Arrays.equals(left, right)); // true
System.out.println(java.util.Arrays.toString(left)); // [1, 2]
```

Array `==` checks identity. Use `Arrays.equals` for one-dimensional content and `Arrays.deepEquals` for nested content when that is the intended contract.

Two-dimensional and jagged arrays

Java's `int[][]` is an array whose elements are `int[]` references:

JAVA

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};

int[][] jagged = {
    {1},
    {2, 3},
    null,
    {4, 5, 6}
};
```

Rows can have different lengths and can be null. Traverse using the current row, not `matrix[0].length`.

Deep-copy a jagged primitive matrix

Manual first-principles implementation:

JAVA

```
static int[][] deepCopy(int[][] source) {
    if (source == null) {
        return null;
    }
    int[][] result = new int[source.length][];
    for (int row = 0; row < source.length; row++) {
        result[row] = source[row] == null ? null : source[row].clone();
    }
    return result;
}
```

This copies the outer array and every non-null primitive row. For arbitrary object graphs, "deep copy" requires a domain-specific ownership definition.

Useful library operations

- `Arrays.fill(array, value)` writes every element.
- `Arrays.sort(array)` sorts in place.
- `Arrays.binarySearch(array, key)` assumes sorted input.
- `Arrays.copyOf(array, newLength)` copies and can resize the result.
- `System.arraycopy` copies a range and handles overlapping ranges according to its contract.

These APIs do not replace understanding. In an interview, state mutation, preconditions, and complexity before using them.

Complete example

File: ArraysExample.java

JAVA

```
import java.util.Arrays;

public final class ArraysExample {
    static int sum(int[] values) {
        int total = 0;
        for (int value : values) {
            total += value;
        }
        return total;
    }

    public static void main(String[] args) {
        int[] values = {4, 1, 3};
        int[] copy = values.clone();
        Arrays.sort(copy);

        System.out.println(sum(values));
        System.out.println(Arrays.toString(values));
        System.out.println(Arrays.toString(copy));
    }
}
```

Expected output:

TEXT

```
8
[4, 1, 3]
[1, 3, 4]
```

Sorting the copy preserves the caller's original order.

Edge-case matrix

Input or operation	Risk	Required thought
null array	dereference failure	reject, accept, or map to an empty result explicitly
empty array	first/last index does not exist	loops should naturally perform zero work
index equals length	out of bounds	valid interval is <code>[0, length)</code>
assignment to another variable	aliasing	choose alias or copy intentionally
nested <code>clone()</code>	rows remain shared	copy each row when independent rows are required
null jagged row	row dereference fails	define skip/reject/preserve behavior
<code>Arrays.binarySearch</code> on unsorted input	result has no useful guarantee	sort or prove sorted precondition
direct <code>println(array)</code>	identity-style text	use <code>Arrays.toString</code> or <code>deepToString</code>

Interview room

Interviewer: What does `int[][] copy = original.clone()` copy?

Model answer: It creates a new outer array and copies each row reference. The row arrays remain shared, so modifying `copy[0][0]` changes the same row seen through `original`. To copy a jagged primitive matrix independently, clone each non-null row.

Follow-up: Is passing an array to a method pass-by-reference?

Model answer: No. The array reference value is copied into the parameter. Both references designate the same array, which allows shared mutation, but parameter reassignment is local.

Practice

- Foundation:** Find minimum and maximum without sorting.
- Foundation:** Reverse an array in place and then return a reversed copy.
- Predict:** Trace aliasing through two array variables and a method call.
- Debugging:** Repair a rectangular-only traversal for jagged input.
- Interview Core:** Deep-copy an `int[][]` while preserving null rows.
- SDE-2 Follow-up:** Design an API that sorts values without surprising its caller about mutation.

Chapter takeaway

An array is fixed-length mutable storage reached through a reference. Correct array code makes the valid index range, mutation policy, aliasing, and copy depth explicit.

SDE-2 CORE CHAPTER

Chapter 7 - Strings, Characters, and StringBuilder

Strings appear in almost every interview, but three different ideas are easy to mix up: the `String` object, the content it represents, and the `char` code units used by many Java APIs.

String values and immutability

JAVA

```
String language = "Java";
String updated = language.toUpperCase();
```

`String` is immutable. `toUpperCase` does not modify `language`; it returns a string result. Reassignment can make a variable refer to another string, but the original object does not change.

Immutability enables safe sharing and stable hashing. It does not mean every string operation is allocation-free.

== versus equals

JAVA

```
String first = new String("java");
String second = new String("java");

System.out.println(first == second);    // false
System.out.println(first.equals(second)); // true
```

`==` compares reference identity. `equals` compares string content.

String literals may share pooled instances:

JAVA

```
String a = "pool";
String b = "pool";
System.out.println(a == b); // normally true by literal interning semantics
```

Do not turn that into an identity-comparison strategy. Runtime-created strings and explicit `new String(...)` do not have to be the same object as a pooled literal. Use `equals` for content.

Essential String operations

Operation	Purpose
<code>length()</code>	number of UTF-16 code units
<code>charAt(index)</code>	code unit at an index
<code>substring(start, end)</code>	half-open slice
<code>equals, equalsIgnoreCase</code>	content comparison
<code>compareTo</code>	lexicographic comparison
<code>contains, indexOf, lastIndexOf</code>	search
<code>startsWith, endsWith</code>	boundary test
<code>replace</code>	replacement result
<code>split</code>	regular-expression split
<code>trim, strip</code>	surrounding-space handling
<code>toCharArray</code>	mutable code-unit copy

`split` accepts a regular expression. `text.split(".")` does not mean split on a literal dot; use an escaped expression such as `"\\."`. Trailing empty results are discarded by default; use a negative limit when they matter.

Empty, blank, and null are different

JAVA

```
String empty = "";           // length zero
String blank = "   ";       // nonzero length, only whitespace
String missing = null;      // no String object
```

- `empty.isEmpty()` is true.
- `blank.isBlank()` is true on Java 11+.
- calling either method on `missing` throws `NullPointerException`.

Choose a domain contract rather than silently treating all three as equivalent.

Why repeated concatenation can be expensive

JAVA

```
String result = "";  
for (String word : words) {  
    result = result + word;  
}
```

Each result is a new immutable string and repeated copying can make total work quadratic in the final text size.

Use a builder for incremental assembly:

JAVA

```
StringBuilder builder = new StringBuilder();  
for (String word : words) {  
    builder.append(word);  
}  
String result = builder.toString();
```

Useful builder operations include `append`, `insert`, `delete`, `setCharAt`, `reverse`, and `toString`. `StringBuilder` is mutable and not thread-safe; normal interview code keeps it local to one method.

char and digit conversion

A `char` is a UTF-16 code unit. For an ASCII decimal digit:

JAVA

```
char character = '7';  
int digit = character - '0';
```

This is safe only after proving `character` is between `'0'` and `'9'`.

For a radix-aware library conversion:

JAVA

```
int digit = Character.digit(character, 10);  
if (digit < 0) {  
    throw new IllegalArgumentException("not a decimal digit");  
}
```

`Character.isDigit`, `isLetter`, `isWhitespace`, `toLowerCase`, and `toUpperCase` are useful, but case conversion can be language-sensitive at whole-string boundaries.

Code units, code points, and visible characters

Many common English letters use one `char`. Some Unicode code points, including many emoji, use a surrogate pair: two `char` values. A user-perceived character can contain more than one code point.

At the fundamentals level, remember:

- `String.length()` counts UTF-16 code units;
- `codePointCount` counts Unicode code points;
- neither automatically counts every user-perceived grapheme.

The Strings book owns algorithmic string patterns, and Advanced Java owns deeper Unicode boundary handling.

Complete example

File: `StringsAndCharactersExample.java`

JAVA

```
public final class StringsAndCharactersExample {
    static String reverseAscii(String input) {
        StringBuilder result = new StringBuilder(input.length());
        for (int index = input.length() - 1; index >= 0; index--) {
            result.append(input.charAt(index));
        }
        return result.toString();
    }

    public static void main(String[] args) {
        String first = new String("java");
        String second = new String("java");
        char character = '7';

        System.out.println(first == second);
        System.out.println(first.equals(second));
        System.out.println(character - '0');
        System.out.println(reverseAscii("SDE2"));
    }
}
```

Expected output:

TEXT

```
false
true
7
2EDS
```

The method is deliberately named `reverseAscii`: reversing UTF-16 code units is not a universal Unicode grapheme reversal.

Edge-case matrix

Case	Risk	Required decision
null input	dereference	reject, accept, or return a defined result
empty input	assumes first character	return the natural empty result
blank input	confused with empty	use the domain's whitespace policy
comparison with <code>==</code>	identity, not content	use <code>equals</code> or null-safe equality
loop concatenation	repeated copying	use a local <code>StringBuilder</code>
<code>split</code> delimiter	regex metacharacter	escape and choose a limit
<code>char</code> – <code>'0'</code>	invalid non-digit input	validate the range first
supplementary code point	two <code>char</code> values	use code-point APIs when required
locale-sensitive case	wrong normalization rule	define locale/protocol requirements

Interview room

Interviewer: Why is `first == second` unreliable for string content?

Model answer: For reference operands, `==` asks whether both references designate the same object. Literals can share a pooled instance, so identity can appear to match. `new String` and runtime results can be different objects with equal content. `String.equals` is the content contract.

Follow-up: When is `StringBuilder` useful?

Model answer: When building text incrementally, especially in a loop. It maintains mutable storage and avoids creating an immutable intermediate string on each append. A single clear concatenation expression does not need a builder by ritual.

Practice

- Foundation:** Compare two strings safely when either may be null.
- Foundation:** Count ASCII digits after validating each `char`.
- Predict:** Compare literals, `new String`, and a concatenated runtime string with both `==` and `equals`.
- Debugging:** Repair a loop that builds a large string using repeated `+`.
- Interview Core:** Normalize consecutive spaces without losing non-space characters.
- SDE-2 Follow-up:** Define whether an identifier policy works in bytes, code units, code points, or user-perceived characters.

Chapter takeaway

Strings are immutable values reached through references. Compare their content, use a builder for incremental assembly, validate character arithmetic, and state whether your algorithm operates on code units, code points, or a narrower input alphabet.

SDE-2 CORE CHAPTER

Chapter 8 - Practice Lab A - Language and Execution

Attempt these without the solution guide. Record both your answer and the rule that supports it.

Start here: work in four short passes instead of treating this as one exam. First answer K01-K25 aloud. Next predict O01-O19 before running anything. Then repair D01-D19 in a scratch file and compile each fix. Finish by implementing C01-C19 and discussing F01-F13 with explicit contracts and edge cases.

Keep an error log with three columns: the mistaken prediction, the Java rule that corrects it, and one boundary input that exposes the mistake. A correct answer without the rule is recall; a correct answer with the rule is reusable interview reasoning. Use Solution Studio A only after an honest attempt, then retry missed items the next day without looking.

Knowledge check

- **K01 [Foundation]** Distinguish source code, bytecode, and machine execution.
- **K02 [Foundation]** Define JDK, runtime/JRE, and JVM without treating them as synonyms.
- **K03 [Foundation]** Explain each token in `public static void main(String[] args)`.
- **K04 [Foundation]** When must a public top-level class match its filename?
- **K05 [Foundation]** Distinguish declaration, initialization, assignment, and reassignment.
- **K06 [Foundation]** Why must a local variable be definitely assigned before reading?
- **K07 [Foundation]** List Java's eight primitive types and identify integral versus floating types.
- **K08 [Interview Core]** Why does primitive bit width not prove total object or field-layout size?
- **K09 [Foundation]** Distinguish a primitive value from a reference value.
- **K10 [Interview Core]** Explain why Java is always pass-by-value.
- **K11 [Foundation]** What is the type and range role of `char`?
- **K12 [Interview Core]** Why can `long result = intA * intB` still overflow?
- **K13 [Foundation]** Distinguish widening and narrowing conversion.
- **K14 [Interview Core]** Describe binary numeric promotion for `byte + byte`.
- **K15 [Foundation]** Explain integer division and remainder with negative operands.
- **K16 [Foundation]** Distinguish `&&` from boolean `&`.
- **K17 [Interview Core]** What hidden cast does compound assignment provide?
- **K18 [Foundation]** Distinguish prefix from postfix increment.
- **K19 [Foundation]** When is a guard clause clearer than nested `if` blocks?
- **K20 [Foundation]** Distinguish traditional switch fall-through from a switch expression.
- **K21 [Foundation]** Choose among `for`, `while`, `do-while`, and enhanced-for.
- **K22 [Interview Core]** What does `break` affect in nested loops?
- **K23 [Interview Core]** What makes an overload distinct, and why is return type insufficient?
- **K24 [Interview Core]** State the base case and recursive case in a recursive method preview.
- **K25 [SDE-2 Follow-up]** Explain the limits of Java's platform-independence claim.

Predict the output

- **O01 [Foundation]** Predict `System.out.println(5 / 2);`.
- **O02 [Foundation]** Predict `System.out.println(-5 % 2);`.
- **O03 [Interview Core]** Predict `(byte) 130` printed as an integer.
- **O04 [Interview Core]** Predict `long x = Integer.MAX_VALUE + 1; System.out.println(x);`.
- **O05 [Foundation]** Predict `int x=3; System.out.println(x++ + " " + x);`.
- **O06 [Foundation]** Predict `int x=3; System.out.println(++x + " " + x);`.
- **O07 [Interview Core]** Predict `short x=10; x+=5; System.out.println(x);`.
- **O08 [Foundation]** Predict `System.out.println('7' - '0');`.
- **O09 [Interview Core]** Predict `System.out.println('A' + 1);`.
- **O10 [Foundation]** Predict `boolean x=false; System.out.println(x && fail());` assuming `fail` would throw.
- **O11 [Interview Core]** Predict `int x=0; if (true || ++x>0){} System.out.println(x);`.
- **O12 [Foundation]** Predict a loop `for(int i=2;i>=0;i--) print(i)`.
- **O13 [Foundation]** Predict `int i=0; do { i++; } while(false); print(i)`.
- **O14 [Interview Core]** Predict a traditional switch with case 1 printing A and falling into case 2 printing B.
- **O15 [Foundation]** Predict `sum(2, 3)` when the method returns `left + right`.
- **O16 [Interview Core]** Predict a caller's primitive after a method increments only its parameter.
- **O17 [Interview Core]** Predict an object's field after a method mutates it through a copied reference.
- **O18 [Interview Core]** Predict a caller reference after a method reassigns only its parameter.
- **O19 [SDE-2 Follow-up]** Predict the evaluation result of `true ? 1 : 2.0` and state its type.

Debug the code

- **D01 [Foundation]** Repair a missing semicolon after `int count = 3`.
- **D02 [Foundation]** Repair reading an uninitialized local `int total`;
- **D03 [Foundation]** Repair a public class `Main` stored in `Program.java` without renaming the file.
- **D04 [Interview Core]** Repair `long area = width * height` for two arbitrary `int` inputs.
- **D05 [Interview Core]** Repair `short value = value + 1`.
- **D06 [Foundation]** Repair an average computed with integer division.
- **D07 [Interview Core]** Repair a null check that uses `text != null & !text.isEmpty()`.
- **D08 [Foundation]** Repair `if (score = 100)`.
- **D09 [Foundation]** Repair `for (int i=0; i<=array.length; i++)`.
- **D10 [Interview Core]** Repair a reverse loop that starts at `array.length`.
- **D11 [Foundation]** Repair a loop whose update moves away from its termination condition.
- **D12 [Interview Core]** Repair accidental switch fall-through.
- **D13 [Foundation]** Repair an enhanced-for loop that tries to use an undeclared index.
- **D14 [Interview Core]** Repair an overload pair differing only by return type.
- **D15 [Interview Core]** Repair an ambiguous `print(null)` call with `String` and `StringBuilder` overloads.
- **D16 [Foundation]** Repair a method declared `int` that reaches the end without returning.
- **D17 [Interview Core]** Repair recursion with no reachable base case.
- **D18 [Interview Core]** Repair `Math.abs(Integer.MIN_VALUE)` when a non-negative long magnitude is required.
- **D19 [SDE-2 Follow-up]** Repair a deeply nested validator using early returns while preserving behavior.

Small coding tasks

- **C01 [Foundation]** Print a greeting with a default name when no command-line argument is supplied.
- **C02 [Foundation]** Return the larger of two integers without using `Math.max`.
- **C03 [Foundation]** Classify an integer as negative, zero, or positive.
- **C04 [Foundation]** Return whether a year is a leap year.
- **C05 [Foundation]** Sum integers from 1 through `n` with a loop.
- **C06 [Foundation]** Traverse an array forward and print values.
- **C07 [Foundation]** Traverse an array backward.
- **C08 [Foundation]** Count values at even indexes.
- **C09 [Interview Core]** Compute a safe `long` product of two `int` values.
- **C10 [Interview Core]** Implement checked narrowing from `long` to `int`.
- **C11 [Foundation]** Implement a calculator with traditional switch and explicit breaks.
- **C12 [Foundation]** Implement the same calculator with a Java 14+ switch expression.
- **C13 [Foundation]** Write a method with parameters and a return value for rectangle area.
- **C14 [Interview Core]** Overload `clamp` for `int` and `long`.
- **C15 [Interview Core]** Demonstrate primitive pass-by-value with a caller assertion.
- **C16 [Interview Core]** Demonstrate shared object mutation through a parameter.
- **C17 [Interview Core]** Demonstrate why reference reassignment does not escape a method.
- **C18 [Interview Core]** Convert an ASCII digit only after validation.
- **C19 [SDE-2 Follow-up]** Refactor one large input-validation method into guard clauses and small helpers.

Interview follow-ups

- **F01 [Interview Core]** What changes when a compiler accepts source but runtime dependencies are incompatible?
- **F02 [SDE-2 Follow-up]** What does `--release 17` constrain and what does it not prove?
- **F03 [Interview Core]** When should overflow wrap, throw, saturate, or widen?
- **F04 [Interview Core]** Why is memorizing the full precedence table less useful than parentheses?
- **F05 [Interview Core]** Explain `&&` as both a correctness and cost tool.
- **F06 [SDE-2 Follow-up]** Defend a loop termination proof using a decreasing measure.
- **F07 [Interview Core]** When is a labeled break clearer than a helper method with early return?
- **F08 [Interview Core]** How can adding an overload break source compatibility?
- **F09 [Interview Core]** Why is `varargs` represented as an array inside the method?
- **F10 [SDE-2 Follow-up]** Explain `StackOverflowError` without claiming a fixed recursion depth.
- **F11 [Interview Core]** Which input boundaries expose integer-division mistakes?
- **F12 [SDE-2 Follow-up]** What evidence separates a compile-time, runtime, and logical defect?
- **F13 [SDE-2 Follow-up]** Explain a platform-dependent behavior that valid Java bytecode cannot hide.

Cumulative assessment 1

A01: In 35 minutes, write and explain a program that parses command-line integers, validates them, computes a safe `long` sum and product, reports negative/zero/positive counts, and avoids overflow-before-widening. Include six tests and classify every possible failure stage.

SDE-2 CORE CHAPTER

Chapter 9 - Classes, Objects, and Constructors

Java interview solutions often use only a few classes, but you still need a clear object model. A class defines a type; an object is one runtime instance; a reference is the value used to reach that object.

State and behavior

JAVA

```
final class Student {
    private final int id;
    private String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void rename(String name) {
        this.name = name;
    }

    String name() {
        return name;
    }
}
```

Fields hold state. Methods expose behavior. `new Student(1, "Ada")` creates an object and produces a reference to it.

JAVA

```
Student first = new Student(1, "Ada");
Student alias = first;
alias.rename("Grace");
System.out.println(first.name()); // Grace
```

The assignment copies a reference value. It does not copy the object.

Constructors establish valid starting state

A constructor has the class name and no return type, not even `void`.

JAVA

```
final class Rectangle {
    private final int width;
    private final int height;

    Rectangle(int width, int height) {
        if (width <= 0 || height <= 0) {
            throw new IllegalArgumentException("positive dimensions required");
        }
        this.width = width;
        this.height = height;
    }
}
```

Java supplies a default no-argument constructor only when the class declares no constructor. Once any constructor is declared, create every desired form explicitly.

Constructor overloading and chaining

JAVA

```
final class Window {
    private final int width;
    private final int height;

    Window() {
        this(800, 600);
    }

    Window(int width, int height) {
        if (width <= 0 || height <= 0) {
            throw new IllegalArgumentException("positive dimensions required");
        }
        this.width = width;
        this.height = height;
    }
}
```

`this(...)` calls another constructor in the same class and must be the first statement. `super(...)` calls a superclass constructor and follows the same first-statement rule. A constructor cannot explicitly call both.

this means the current object

In `this.name = name`, `this.name` is the field and `name` is the parameter. `this` can also be passed to another method or returned, but exposing an incompletely constructed `this` is dangerous and belongs in the advanced book.

Instance and static members

An instance member belongs to an object. A static member belongs to the class:

JAVA

```
final class Identifier {  
    private static int nextValue = 1;  
  
    private Identifier() {}  
  
    static int next() {  
        return nextValue++;  
    }  
}
```

Call it as `Identifier.next()`. A static method has no `this` and cannot directly use instance fields.

Constants often use `static final`:

JAVA

```
private static final int MAX_ATTEMPTS = 3;
```

Mutable static state is shared across calls, tests, and threads. Use it only with an explicit lifecycle and concurrency policy.

Access modifiers

Modifier	Same class	Same package	Subclass elsewhere	Unrelated elsewhere
private	yes	no	no	no
package-private	yes	yes	no	no
protected	yes	yes	through inheritance rules	no
public	yes	yes	yes	yes

Package-private is the absence of an access modifier. Across packages, protected access is tied to the subclass's inherited view; it is not simply public access for every subclass reference.

Packages and imports

JAVA

```
package interview.model;

import java.util.List;
```

The package declaration normally comes first. Imports shorten source names; they do not copy a class into the current package or load it at runtime. Avoid the default package for maintainable multi-file projects.

WHY IT WORKS | Encapsulation is about invariants

Encapsulation does not mean adding setters for every field. It means callers use operations that keep the object valid.

JAVA

```
final class BankAccount {
    private long balanceCents;

    BankAccount(long openingBalanceCents) {
        if (openingBalanceCents < 0) {
            throw new IllegalArgumentException("negative balance");
        }
        balanceCents = openingBalanceCents;
    }

    void withdraw(long cents) {
        if (cents <= 0 || cents > balanceCents) {
            throw new IllegalArgumentException("invalid withdrawal");
        }
        balanceCents -= cents;
    }
}
```

A raw `setBalance` would allow invalid state. The domain operation is the safer interface.

Complete example

File: ClassesAndConstructorsExample.java

JAVA (continued 1/2)

```
public final class ClassesAndConstructorsExample {
    static final class Counter {
        private static int created;
        private int value;

        Counter() {
            this(0);
        }

        Counter(int initialValue) {
            if (initialValue < 0) {
                throw new IllegalArgumentException("negative initial value");
            }
            value = initialValue;
            created++;
        }

        void increment() {
            value++;
        }
    }
}
```

JAVA (continued 2/2)

```
    int value() {
        return value;
    }

    static int created() {
        return created;
    }
}

public static void main(String[] args) {
    Counter first = new Counter();
    Counter second = new Counter(4);
    second.increment();

    System.out.println(first.value());
    System.out.println(second.value());
    System.out.println(Counter.created());
}
}
```

Expected output:

TEXT

```
0
5
2
```

Edge-case matrix

Case	Mistake	Correct rule
constructor declares <code>void</code>	creates an ordinary method	constructors have no return type
explicit constructor exists	expects generated no-arg constructor	declare it yourself if needed
two references reach object	assumes independent objects	assignment copies the reference
<code>final List</code> field	calls object immutable	<code>final</code> prevents reference reassignment only
static method reads instance field	no receiver exists	pass an object or use an instance method
public setter	breaks invariant	expose a meaningful validated operation
import statement	thinks package changes	import only shortens a compile-time name

Interview room

Interviewer: When does Java provide a default constructor?

Model answer: Only when the class declares no constructor. It is a no-argument constructor that invokes an accessible superclass constructor. Declaring any constructor means Java does not add another automatically.

Follow-up: Does `final` make an object immutable?

Model answer: No. A final variable cannot be reassigned after initialization. If it refers to a mutable object, that object's state may still change. Immutability depends on the complete class design and reachable state.

Practice

- 1 **Foundation:** Create a validated `Rectangle` and its `area()` method.
- 2 **Foundation:** Add no-arg and parameterized constructors using `this(...)`.
- 3 **Predict:** Trace two references that designate the same mutable object.
- 4 **Debugging:** Repair a constructor accidentally written with `void`.
- 5 **Interview Core:** Refactor public mutable fields into an invariant-preserving class.
- 6 **SDE-2 Follow-up:** Explain why mutable static state complicates testing and concurrency.

Chapter takeaway

Constructors establish valid objects, methods protect their invariants, references can alias, static state is shared, and access control limits who can depend on an implementation detail.

SDE-2 CORE CHAPTER

Chapter 10 - Object-Oriented Foundations in Interview-Sized Steps

Object-oriented programming is useful when it makes a model easier to change and reason about. For DSA interviews, keep the model small. For Java interviews, explain the relationship and dispatch rules precisely.

Inheritance: a subtype promise

JAVA

```
class Notification {
    String format() {
        return "notification";
    }
}

final class EmailNotification extends Notification {
    @Override
    String format() {
        return "email";
    }
}
```

`EmailNotification` is a `Notification`. Inheritance should express a truthful IS-A relationship, not merely reuse a few lines of code.

Superclass construction occurs before the subclass constructor body. Use `super(...)` to select a superclass constructor. Java allows one direct superclass and multiple implemented interfaces.

Overriding and runtime polymorphism

JAVA

```
Notification notification = new EmailNotification();
System.out.println(notification.format()); // email
```

The reference's declared type is `Notification`; the runtime object is `EmailNotification`. An overridden instance method is dynamically dispatched to the object type.

Overloading is different: the compiler chooses among parameter lists using compile-time types. Fields and static methods are not dynamically dispatched like overridden instance methods.

Upcasting and downcasting

An upcast to a superclass or interface is safe:

JAVA

```
Notification notification = new EmailNotification();
```

A downcast can fail with `ClassCastException`. Test the runtime type when the design genuinely requires the subtype:

JAVA

```
if (notification instanceof EmailNotification email) {  
    System.out.println(email.format());  
}
```

Pattern matching for `instanceof` is permanent since Java 16. For older targets, test and cast separately.

Abstract classes

An abstract class can have constructors, fields, concrete methods, and abstract methods:

JAVA

```
abstract class Parser {  
    Parser() {}  
  
    final int parsePositive(String text) {  
        int value = parse(text);  
        if (value <= 0) {  
            throw new IllegalArgumentException("positive value required");  
        }  
        return value;  
    }  
  
    protected abstract int parse(String text);  
}
```

Use an abstract class when closely related types share construction, state, or a protected implementation template.

Interfaces

An interface defines a role that unrelated classes can implement:

JAVA

```
interface Formatter {
    String format(int value);

    default String label(int value) {
        return "value=" + format(value);
    }
}
```

Interfaces can declare abstract methods, default methods, static methods, and private helper methods on supported Java versions. They do not have constructors or per-instance fields. A class can implement multiple interfaces.

Composition and delegation

Composition expresses HAS-A:

JAVA

```
interface Clock {
    long nowMillis();
}

final class ExpiringToken {
    private final Clock clock;
    private final long expiresAt;

    ExpiringToken(Clock clock, long expiresAt) {
        this.clock = clock;
        this.expiresAt = expiresAt;
    }

    boolean isExpired() {
        return clock.nowMillis() >= expiresAt;
    }
}
```

The token has a clock; it is not a clock. A test can supply a deterministic implementation. Composition avoids inventing a false subtype and makes collaborators replaceable.

Decision table

Choose	When it best describes the model
concrete class	one complete implementation is the useful concept
interface	a role, contract, or replacement seam
abstract class	related types share state or construction
inheritance	the subtype preserves the parent's behavioral promise
composition	one object uses another capability

Equality and hashing introduction

`==` compares object identity. A class can override `equals` for semantic equality. Equal objects must have equal hash codes.

JAVA

```
record Coordinate(int row, int column) {}
```

Records provide value-oriented generated members for simple data carriers on Java 16+. An ordinary class remains the right choice when construction, identity, mutation, or framework requirements differ.

Do not mutate fields used by equality and hash code while an object is stored in a hash-based set or map. The Collections and Hashing books develop this contract fully.

Complete example

File: ObjectOrientedFoundationsExample.java

JAVA

```
public final class ObjectOrientedFoundationsExample {
    interface DiscountPolicy {
        int apply(int priceCents);
    }

    static final class FixedDiscount implements DiscountPolicy {
        private final int discountCents;

        FixedDiscount(int discountCents) {
            this.discountCents = discountCents;
        }

        @Override
        public int apply(int priceCents) {
            return Math.max(0, priceCents - discountCents);
        }
    }

    static final class Checkout {
        private final DiscountPolicy policy;

        Checkout(DiscountPolicy policy) {
            this.policy = policy;
        }

        int total(int priceCents) {
            return policy.apply(priceCents);
        }
    }

    public static void main(String[] args) {
        DiscountPolicy policy = new FixedDiscount(250);
        Checkout checkout = new Checkout(policy);
        System.out.println(checkout.total(1_000));
    }
}
```

Expected output:

TEXT

750

Checkout composes a policy. The interface reference supports runtime polymorphism without making checkout a subtype of a discount.

Edge-case matrix

Case	Wrong shortcut	Accurate explanation
overloaded method	calls it runtime polymorphism	overload selection is compile-time
overridden instance method	looks only at reference type	runtime object selects the override
static method with same signature	calls it overriding	static methods are hidden
field with same name	expects polymorphism	field access follows declared type
forced downcast	assumes actual subtype	verify or redesign the base contract
abstract class	says it cannot construct state	it can have constructors
interface	says all methods are abstract	default/static/private behavior exists
multiple inheritance	expects two superclasses	Java permits multiple interfaces, one class
inheritance for reuse	ignores substitutability	prefer composition without a true IS-A relation

Interview room

Interviewer: What is the difference between overloading and overriding?

Model answer: Overloading uses the same method name with different parameter lists and is selected at compile time from declared types. Overriding replaces an inherited instance-method implementation, and runtime dispatch chooses the override using the actual receiver class.

Follow-up: When do you prefer composition?

Model answer: When one object uses another capability but is not a true subtype, or when I need easy replacement and testing. Inheritance creates a behavioral substitutability commitment, so I use it only when that promise is valid.

Practice

- 1 **Foundation:** Override one method in a truthful subtype.
- 2 **Predict:** Trace a base reference holding a subclass object.
- 3 **Debugging:** Identify an unsafe downcast and redesign the base contract.
- 4 **Interview Core:** Replace false inheritance with a composed collaborator.
- 5 **Interview Core:** Compare an interface, abstract class, and concrete class for a parser family.
- 6 **SDE-2 Follow-up:** Explain why equality across a mutable inheritance hierarchy is difficult.

Chapter takeaway

Use inheritance for a real subtype, interfaces for roles, abstract classes for related shared structure, and composition for replaceable collaborators. Keep compile-time overloading separate from runtime overriding.

SDE-2 CORE CHAPTER

Chapter 11 - Practice Lab B - Arrays, Strings, and Objects

Start here: draw references before writing code. For every array or object question, mark which variables hold primitive values, which hold reference values, and which references designate the same object. For strings, state whether the requirement is identity, UTF-16 content, code points, or user-visible text before selecting an API.

Complete K26-K50 aloud, predict O20-O38 on paper, compile repairs for D20-D38, and implement C20-C38 in small files. Use a mutation table for each task: input alias, state before, operation, state after, and whether the caller observes the change. Check Solution Studio B later and add every missed aliasing, constructor, dispatch, or equality rule to your error log.

Knowledge check

- **K26 [Foundation]** Distinguish array declaration, creation, and initialization.
- **K27 [Foundation]** What default values do array elements receive?
- **K28 [Interview Core]** Explain array aliasing.
- **K29 [Interview Core]** Distinguish shallow from deep copy.
- **K30 [Foundation]** Why does printing an array reference not print its contents?
- **K31 [Interview Core]** Distinguish `Arrays.equals` from `Arrays.deepEquals`.
- **K32 [Foundation]** What makes a Java two-dimensional array jagged-capable?
- **K33 [Foundation]** Why is `String` immutable?
- **K34 [Interview Core]** Distinguish string identity, content, and pooling.
- **K35 [Foundation]** Distinguish empty, blank, and null strings.
- **K36 [Interview Core]** When should repeated concatenation use `StringBuilder`?
- **K37 [Foundation]** What does a Java `char` represent?
- **K38 [Interview Core]** Distinguish code unit, code point, and user-perceived character.
- **K39 [Foundation]** Define class, object, field, method, state, and behavior.
- **K40 [Interview Core]** What does a reference variable store?
- **K41 [Foundation]** When is a default constructor supplied?
- **K42 [Foundation]** List the three common uses of `this` in fundamentals.
- **K43 [Interview Core]** Why is mutable static state hard to test?
- **K44 [Interview Core]** Explain protected access across package boundaries.
- **K45 [Foundation]** What does an import declaration do and not do?
- **K46 [Interview Core]** Why is encapsulation more than getters and setters?
- **K47 [Interview Core]** Distinguish overloading, overriding, field hiding, and static hiding.
- **K48 [Foundation]** Compare interface, abstract class, and concrete class.
- **K49 [Interview Core]** When is composition preferable to inheritance?
- **K50 [SDE-2 Follow-up]** State the `equals/hashCode` requirement for hash keys.

Predict the output

- **O20 [Foundation]** Predict an `int[3]` element before assignment.
- **O21 [Interview Core]** Predict output after mutating an aliased one-dimensional array.
- **O22 [Interview Core]** Predict output after mutating a nested row through `matrix.clone()`.
- **O23 [Foundation]** Predict `Arrays.toString(new int[]{1,2})`.
- **O24 [Interview Core]** Predict `new int[2][ ]` row values.
- **O25 [Foundation]** Predict `"java".substring(1,3)`.
- **O26 [Interview Core]** Predict literal equality for `"a" == "a"` and explain why not to rely on it generally.
- **O27 [Interview Core]** Predict equality for `new String("a") == "a"`.
- **O28 [Foundation]** Predict a string after ignored `toUpperCase()` return value.
- **O29 [Foundation]** Predict a builder after `append`, `deleteCharAt`, and `reverse`.
- **O30 [Interview Core]** Predict the length of one supplementary emoji encoded as UTF-16.
- **O31 [Foundation]** Predict two objects' independent field values after one is renamed.
- **O32 [Interview Core]** Predict a shared object's field after alias mutation.
- **O33 [Foundation]** Predict constructor-chaining print order with `this(...)`.
- **O34 [Interview Core]** Predict static field count after creating three instances.
- **O35 [Interview Core]** Predict field and method outputs through a base reference to a child.
- **O36 [Interview Core]** Predict overloaded method selection through a base reference.
- **O37 [Interview Core]** Predict overridden method dispatch through an interface reference.
- **O38 [SDE-2 Follow-up]** Predict map lookup after mutating an equality-participating key.

Debug the code

- **D20 [Foundation]** Repair array traversal that reads index `length`.
- **D21 [Interview Core]** Repair a method that unintentionally mutates the caller's array.
- **D22 [Interview Core]** Repair a supposed deep copy that clones only the outer array.
- **D23 [Foundation]** Repair `firstArray == secondArray` for content equality.
- **D24 [Foundation]** Repair direct array printing.
- **D25 [Interview Core]** Repair matrix traversal that assumes every row has `matrix[0].length`.
- **D26 [Foundation]** Repair `firstString == secondString` for value equality.
- **D27 [Interview Core]** Repair null-unsafe `text.equals(target)`.
- **D28 [Interview Core]** Repair repeated string concatenation in a large loop.
- **D29 [Foundation]** Repair ASCII digit conversion without validation.
- **D30 [Interview Core]** Repair string reversal that corrupts surrogate pairs under a code-point contract.
- **D31 [Foundation]** Repair a constructor mistakenly declared `void Student(...)`.
- **D32 [Foundation]** Restore a missing no-argument constructor intentionally.
- **D33 [Interview Core]** Repair a static method that directly reads an instance field.
- **D34 [Interview Core]** Repair a public setter that violates an account invariant.
- **D35 [Interview Core]** Repair an intended override with the wrong parameter type.
- **D36 [Interview Core]** Repair unsafe downcasting with a type check or stronger contract.
- **D37 [Interview Core]** Repair an interface implementation that reduces method visibility.
- **D38 [SDE-2 Follow-up]** Repair a mutable list field in an allegedly immutable class.

Small coding tasks

- **C20 [Foundation]** Create and traverse a one-dimensional integer array.
- **C21 [Foundation]** Create and traverse a rectangular matrix.
- **C22 [Foundation]** Create a jagged array with row lengths 1, 2, and 3.
- **C23 [Interview Core]** Demonstrate array aliasing and then repair it with a copy.
- **C24 [Interview Core]** Deep-copy a nullable jagged array.
- **C25 [Foundation]** Compare and print one-dimensional arrays correctly.
- **C26 [Foundation]** Demonstrate string content versus identity comparison.
- **C27 [Interview Core]** Demonstrate literal pooling versus explicit construction.
- **C28 [Foundation]** Build a comma-separated string with `StringBuilder`.
- **C29 [Interview Core]** Convert validated decimal characters with `Character.digit`.
- **C30 [Foundation]** Implement the validated `Student` class from this volume.
- **C31 [Foundation]** Add constructor overloading and chaining to a `Rectangle`.
- **C32 [Interview Core]** Implement a class-level ID generator and discuss shared state.
- **C33 [Interview Core]** Encapsulate a bounded counter with invariant-preserving methods.
- **C34 [Interview Core]** Implement an immutable `Coordinate` with equality and hash code.
- **C35 [Foundation]** Implement a base `Notification` and overriding subtype.
- **C36 [Interview Core]** Demonstrate runtime polymorphism through a list of interface references.
- **C37 [Interview Core]** Implement an abstract parser with a concrete decimal parser.
- **C38 [SDE-2 Follow-up]** Replace a false inheritance relationship with a composed collaborator and test double.

Interview follow-ups

- **F14 [Interview Core]** When should an array method preserve versus mutate its input?
- **F15 [SDE-2 Follow-up]** Define a copy contract for jagged arrays containing null rows.
- **F16 [Interview Core]** Why can `Arrays.binarySearch` be wrong on an unsorted array?
- **F17 [Interview Core]** When is `char` traversal the correct contract despite Unicode limits?
- **F18 [SDE-2 Follow-up]** How would you state string complexity in code units versus code points?
- **F19 [Interview Core]** Why does `final` not prove deep immutability?
- **F20 [Interview Core]** What construction bug can an overridable method expose?
- **F21 [SDE-2 Follow-up]** Explain protected access through a subclass in another package.
- **F22 [Interview Core]** When is an abstract class better than an interface?
- **F23 [Interview Core]** How can a default interface method create an evolution conflict?
- **F24 [SDE-2 Follow-up]** Defend composition for replaceability and testability.
- **F25 [Interview Core]** Why must equal hash keys have equal hash codes?
- **F26 [SDE-2 Follow-up]** Design an immutable class that owns a mutable list safely.

Cumulative assessment 2

A02: In 45 minutes, build a small library checkout model using an enum state, an interface policy, composition, an immutable identifier, a mutable aggregate with guarded transitions, and a collection-free array snapshot. Explain aliases, constructor invariants, dispatch, equality, and five boundary tests.

SDE-2 CORE CHAPTER

Chapter 12 - Wrapper Classes, Basic Generics, and Enums

Collections store references, while early interview algorithms often work with primitive values. Wrapper classes and basic generics connect those two worlds safely. Enums give names and type safety to a fixed set of states.

Wrapper classes

Primitive	Wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Wrappers are immutable and provide parsing, comparison, conversion, constants, and utilities:

JAVA

```
int count = Integer.parseInt("42");
long maximum = Long.MAX_VALUE;
int order = Integer.compare(10, 20);
```

Parsing invalid or out-of-range text throws `NumberFormatException`.

Autoboxing and unboxing

JAVA

```
java.util.List<Integer> values = new java.util.ArrayList<>();
values.add(7);                // boxes int to Integer
int first = values.get(0);    // unboxes Integer to int
```

Unboxing `null` throws `NullPointerException`:

JAVA

```
Integer missing = null;
// int value = missing; // runtime failure when reached
```

Wrapper `==` compares identity. Small cached values can make identity appear to work:

JAVA

```
Integer first = 127;
Integer second = 127;
System.out.println(first == second); // true for required cached constants
```

Values outside the required cached range may use different objects. Use `equals` or `Objects.equals`; never depend on wrapper identity for numeric meaning.

Why generics matter

Without a type argument, a collection can defer a type mistake until runtime. Generics let the compiler enforce the intended element type:

JAVA

```
java.util.List<String> names = new java.util.ArrayList<>();
names.add("Ada");
String firstName = names.get(0);
```

The diamond `<>` asks the compiler to infer type arguments from context.

Primitive types cannot be generic arguments, so write `List<Integer>`, not `List<int>`.

A small generic class and method

JAVA

```
final class Box<T> {
    private final T value;

    Box(T value) {
        this.value = value;
    }

    T value() {
        return value;
    }
}

static <T> T first(java.util.List<T> values) {
    if (values.isEmpty()) {
        throw new IllegalArgumentException("empty values");
    }
    return values.get(0);
}
```

T is a type parameter. `Box<String>` is one parameterization. The method declares its own T before the return type.

A light bound exposes an operation:

JAVA

```
static <T extends Comparable<? super T>> T smaller(T left, T right) {
    return left.compareTo(right) <= 0 ? left : right;
}
```

Wildcard variance, capture, erasure, and heap pollution belong in the Advanced Java Language book.

Enums model finite state

JAVA

```
enum OrderStatus {
    CREATED,
    PAID,
    SHIPPED,
    DELIVERED,
    CANCELLED;

    boolean isTerminal() {
        return this == DELIVERED || this == CANCELLED;
    }
}
```

Enum constants are typed instances. Enums can have fields, constructors, and methods and work naturally with switch. They are safer than magic strings because the compiler checks names.

Do not persist `ordinal()` as a durable external value. Reordering constants would change it. Persist an explicit stable code when the domain needs one.

Complete example

File: `WrappersGenericsEnumsExample.java`

JAVA

```
import java.util.ArrayList;
import java.util.List;

public final class WrappersGenericsEnumsExample {
    enum Difficulty { FOUNDATION, INTERVIEW_CORE, SDE2_FOLLOW_UP }

    record Exercise(String title, Difficulty difficulty) {}

    static <T> T first(List<T> values) {
        if (values.isEmpty()) {
            throw new IllegalArgumentException("empty values");
        }
        return values.get(0);
    }

    public static void main(String[] args) {
        List<Integer> scores = new ArrayList<>();
        scores.add(Integer.parseInt("90"));

        List<Exercise> exercises = List.of(
            new Exercise("Array traversal", Difficulty.FOUNDATION));

        System.out.println(first(scores));
        System.out.println(first(exercises).difficulty());
    }
}
```

Expected output:

TEXT

```
90
FOUNDATION
```

Edge-case matrix

Case	Failure	Correction
invalid numeric text	<code>NumberFormatException</code>	validate/catch at the input boundary
null wrapper arithmetic	unboxing <code>NullPointerException</code>	define null policy before arithmetic
wrapper comparison with <code>==</code>	identity-dependent result	use value equality or primitive comparison
raw collection	unchecked runtime type failure	use a specific generic type
<code>List<int></code>	does not compile	use <code>List<Integer></code>
empty generic helper	invalid first element	reject or return an explicit absence type
enum ordinal persisted	declaration reorder changes value	use an explicit stable external code
string status	typo survives compilation	model finite states with an enum

Interview room

Interviewer: Why can `Integer first = 127; Integer second = 127; first == second` be true?

Model answer: Boxing of certain constant values uses required cached instances, so both references can designate one object. `==` still means identity, not numeric equality. I use `equals` or `unbox` to compare values and never depend on cache identity.

Follow-up: Why can generics not use primitive type arguments?

Model answer: Java generics operate on reference types. Primitive values must use wrappers such as `Integer`, with boxing and unboxing at boundaries. Advanced details such as erasure belong in the generics chapter later.

Practice

- Foundation:** Parse three integers and handle invalid input explicitly.
- Predict:** Compare boxed 127 and 128 values using both `==` and `equals`.
- Debugging:** Find the null-unboxing failure in a `Map<String, Boolean>` lookup.
- Interview Core:** Write a generic `last(List<T>)` with an empty-input contract.
- Interview Core:** Replace string order states with an enum and exhaustive switch.
- SDE-2 Follow-up:** Explain the difference between unmodifiable membership and immutable element state.

Chapter takeaway

Wrappers let primitives participate in reference-based APIs but add null and identity traps. Generics move type errors to compilation. Enums model a finite state space more safely than strings.

SDE-2 CORE CHAPTER

Chapter 13 - Java Collections Basics for SDE-2 Interviews

Learning objectives

By the end of this chapter, you can create and use the collection types expected in Java coding interviews, choose one from required behavior, iterate and modify safely, explain basic mutability and ordering contracts, and avoid the API traps that commonly break otherwise correct algorithms.

This is the usage chapter. The later Collections Architecture chapter explains views, ownership, optional operations, and design trade-offs more deeply. Hash-table internals, tree balancing, heap mechanics, and concurrent collections remain in their dedicated books.

Start with the problem, not the class name

An array has fixed length. A collection manages a changing group of reference values through an interface. Generic type arguments state what may be stored.

JAVA

```
List<String> names = new ArrayList<>();  
names.add("Ada");  
names.add("Grace");
```

Read the declaration as: `names` follows the `List<String>` contract, and the current implementation is `ArrayList<String>`. Program to the interface unless a concrete implementation exposes behavior the method genuinely needs.

Use this first decision:

Requirement	Start with	Common implementation
ordered sequence, duplicates allowed	<code>List<E></code>	<code>ArrayList<E></code>
uniqueness or membership	<code>Set<E></code>	<code>HashSet<E></code>
key-to-value association	<code>Map<K, V></code>	<code>HashMap<K, V></code>
first-in, first-out work	<code>Queue<E></code> or <code>Deque<E></code>	<code>ArrayDeque<E></code>
last-in, first-out work	<code>Deque<E></code>	<code>ArrayDeque<E></code>
repeatedly remove best priority	<code>PriorityQueue<E></code>	<code>PriorityQueue<E></code>
retain insertion order	<code>List</code> , <code>LinkedHashSet</code> , <code>LinkedHashMap</code>	matching type
keep keys/elements sorted	<code>TreeMap</code> , <code>TreeSet</code>	matching type

Do not select `LinkedList` because an interview slogan says insertion is fast. Insertion is only useful after reaching the position, and node-based storage has real locality and memory costs.

List basics

A list preserves encounter order, permits duplicates, and supports positional access.

JAVA

```
List<Integer> scores = new ArrayList<>();
scores.add(70);           // [70]
scores.add(90);           // [70, 90]
scores.add(1, 80);        // [70, 80, 90]
int first = scores.get(0); // 70
scores.set(0, 75);        // [75, 80, 90]
boolean hasNinety = scores.contains(90);
int size = scores.size();
```

Indexes are zero-based and form `[0, size())`. `get`, `set`, and indexed insertion throw `IndexOutOfBoundsException` for an invalid position.

The overloaded `remove` methods are a classic trap:

JAVA

```
List<Integer> values = new ArrayList<>(List.of(10, 20, 30));
values.remove(1);           // removes index 1 -> value 20
values.remove(Integer.valueOf(30)); // removes the element 30
```

`ArrayList` is the default general-purpose list for interviews. `LinkedList` also implements `List` and `Deque`, but indexed access is linear. Use it only when its exact deque or iterator-position behavior is justified; `ArrayDeque` is normally clearer for queue/stack work.

Set basics

A set stores no duplicate elements according to equality or ordering.

JAVA

```
Set<Integer> seen = new HashSet<>();
System.out.println(seen.add(4)); // true
System.out.println(seen.add(4)); // false
System.out.println(seen.contains(4)); // true
seen.remove(4);
```

`add` returning `false` is often useful while detecting duplicates.

JAVA

```
static boolean hasDuplicate(int[] numbers) {
    Set<Integer> seen = new HashSet<>();
    for (int number : numbers) {
        if (!seen.add(number)) return true;
    }
    return false;
}
```

- `HashSet` does not promise iteration order.
- `LinkedHashSet` preserves insertion encounter order.
- `TreeSet` keeps elements sorted according to natural order or a comparator and supports operations such as `floor`, `ceiling`, `lower`, and `higher`.

Set behavior depends on stable equality/hash or comparison. Mutating a stored element's key fields can make membership behavior incorrect.

Map basics

A map associates unique keys with values. It is not a subtype of `Collection`.

JAVA

```
Map<String, Integer> attempts = new HashMap<>();
attempts.put("java", 1);
attempts.put("java", 2); // replaces the value for the same key

int value = attempts.get("java");
int missingAsZero = attempts.getDefault("dsa", 0);
boolean hasKey = attempts.containsKey("java");
attempts.remove("java");
```

Distinguish a missing key from a key mapped to null when the implementation permits null values: `get` returns null for both, while `containsKey` separates them. Prefer a domain contract that avoids ambiguous null when practical.

Frequency counting:

JAVA

```
static Map<String, Integer> frequencies(List<String> words) {
    Map<String, Integer> count = new HashMap<>();
    for (String word : words) {
        count.merge(word, 1, Integer::sum);
    }
    return count;
}
```

Useful update APIs:

- `putIfAbsent(key, value)` installs only when absent.
- `computeIfAbsent(key, function)` lazily creates a value, useful for grouping.
- `merge(key, value, function)` combines a new value with an existing one.
- `replace` changes only an existing mapping.

Grouping example:

JAVA

```
Map<Character, List<String>> groups = new HashMap<>();
for (String word : List.of("array", "api", "map")) {
    groups.computeIfAbsent(word.charAt(0), ignored -> new ArrayList<>())
        .add(word);
}
```

Iteration choices:

JAVA

```
for (Map.Entry<String, Integer> entry : attempts.entrySet()) {
    String key = entry.getKey();
    int count = entry.getValue();
}
```

Use `entrySet` when both key and value are needed; it avoids a second lookup and communicates the mapping traversal directly.

- `HashMap` does not promise iteration order.
- `LinkedHashMap` preserves insertion order, or access order when configured for specialized use.
- `TreeMap` keeps keys sorted and supports range/nearest-key navigation.

Queue and deque basics

A queue models FIFO work. A deque supports both ends and can model both a queue and a stack.

Prefer the non-throwing queue method family in interview algorithms:

Operation	Special-value form	Throwing form
insert	<code>offer</code>	<code>add</code>
remove head	<code>poll</code>	<code>remove</code>
inspect head	<code>peek</code>	<code>element</code>

FIFO with `ArrayDeque`:

JAVA

```
Deque<String> queue = new ArrayDeque<>();
queue.offerLast("first");
queue.offerLast("second");
System.out.println(queue.pollFirst()); // first
```

LIFO with `ArrayDeque`:

JAVA

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
System.out.println(stack.pop()); // 20
```

Use consistent ends. Mixing `push`, `offerLast`, `pollFirst`, and `removeLast` without a written convention makes traces difficult. `ArrayDeque` rejects null, which keeps null available as the empty-result signal for `poll` and `peek`.

Avoid legacy `Stack` for new interview code. `ArrayDeque` provides the needed stack contract without inheriting the old synchronized `Vector` API.

PriorityQueue basics

`PriorityQueue` exposes the element with the smallest natural order by default.

JAVA

```
PriorityQueue<Integer> minimums = new PriorityQueue<>();
minimums.offer(9);
minimums.offer(3);
minimums.offer(5);

System.out.println(minimums.peek()); // 3
System.out.println(minimums.poll()); // 3
```

A max-heap reverses the comparator:

JAVA

```
PriorityQueue<Integer> maximums =  
    new PriorityQueue<>(Comparator.reverseOrder());
```

For a custom type, compare without subtraction:

JAVA

```
record Candidate(String name, int score) {}  
  
Comparator<Candidate> byHighestScore =  
    Comparator.comparingInt(Candidate::score)  
        .reversed()  
        .thenComparing(Candidate::name);
```

The queue promises only the head after each operation. Iterating the priority queue is not sorted traversal. Poll a defensive copy when sorted output is required and the original must remain intact.

Iteration and safe removal

Enhanced-for is best when no index or structural mutation is needed:

JAVA

```
for (String name : names) {  
    System.out.println(name);  
}
```

Do not structurally modify an ordinary collection directly inside enhanced-for. Use one of these contracts:

JAVA

```
values.removeIf(value -> value < 0);  
  
for (Iterator<Integer> iterator = values.iterator(); iterator.hasNext();) {  
    if (iterator.next() < 0) iterator.remove();  
}
```

`ConcurrentModificationException` is a best-effort bug signal, not a thread-safety mechanism.

Factories, copies, and mutability

These declarations do not mean the same thing:

JAVA

```
List<String> mutable = new ArrayList<>(List.of("a", "b"));
List<String> unmodifiable = List.of("a", "b");
List<String> fixedSize = Arrays.asList("a", "b");
List<String> snapshot = List.copyOf(mutable);
List<String> readOnlyView = Collections.unmodifiableList(mutable);
```

- `mutable` permits structural and element replacement.
- `List.of` is unmodifiable and rejects null.
- `Arrays.asList` is fixed-size; `set` works but `add` and `remove` do not.
- `List.copyOf` creates unmodifiable membership not backed by later source changes.
- `unmodifiableList` is a view; mutation through another alias remains visible.

None of these operations makes mutable element objects deeply immutable.

Sorting and conversion basics

JAVA

```
List<Integer> numbers = new ArrayList<>(List.of(4, 1, 3));
numbers.sort(Integer::compare);
Collections.reverse(numbers);

int[] primitiveArray = numbers.stream().mapToInt(Integer::intValue).toArray();
List<Integer> boxed = Arrays.stream(primitiveArray).boxed().toList();
```

`toList()` returns an unmodifiable list in current Java. Use `new ArrayList<>(...)` when a mutable result is required. `Arrays.asList(primitiveArray)` produces a one-element `List<int[]>`, not a list of boxed integers.

Complexity intuition, not folklore

Interfaces do not promise one complexity. State the implementation and qualifier.

Operation	Common implementation	Interview-level cost
indexed get/set	<code>ArrayList</code>	$O(1)$
append	<code>ArrayList</code>	amortized $O(1)$
insert/remove at front	<code>ArrayList</code>	$O(n)$ shifting
indexed get	<code>LinkedList</code>	$O(n)$
membership	<code>HashSet</code>	expected $O(1)$
key lookup	<code>HashMap</code>	expected $O(1)$
ordered lookup	<code>TreeSet/TreeMap</code>	$O(\log n)$
deque end operation	<code>ArrayDeque</code>	amortized $O(1)$
heap offer/poll	<code>PriorityQueue</code>	$O(\log n)$
heap peek	<code>PriorityQueue</code>	$O(1)$

The Time and Space Complexity volume explains how to derive and qualify these costs. The collection-internals volume explains resizing, hashing, trees, node overhead, and implementation behavior.

Interview checklist

Before choosing a collection, state:

- 1 Are duplicates allowed?
- 2 Does encounter or sorted order matter?
- 3 Is the operation positional, membership, key lookup, FIFO, LIFO, or priority?
- 4 Can inputs or returned data be mutated?
- 5 Are nulls valid?
- 6 Which operation dominates and at what scale?
- 7 Does equality or comparator state remain stable?

WATCH OUT | Common mistakes

- Declaring `ArrayList` everywhere instead of the useful interface.
- Forgetting `Map` is not a `Collection`.
- Confusing `remove(index)` and `remove(value)` for `List<Integer>`.
- Depending on `HashMap/HashSet` iteration order.
- Assuming `LinkedList` is automatically faster for insertion.
- Using `Stack` instead of `ArrayDeque`.
- Treating `PriorityQueue` iteration as sorted.
- Writing comparator subtraction that can overflow.
- Modifying a collection directly in enhanced-for.
- Confusing fixed-size, unmodifiable, snapshot, view, and deeply immutable.
- Using mutable equality/hash fields for set elements or map keys.
- Claiming `HashMap` operations are guaranteed $O(1)$.

Practice

- 1 **Foundation:** Use `ArrayList` to insert, replace, remove by index, and remove by value.
- 2 **Foundation:** Deduplicate input while preserving first-seen order.
- 3 **Interview Core:** Build a frequency map, then return the most frequent key with a deterministic tie-breaker.
- 4 **Interview Core:** Implement FIFO and LIFO behavior with two separate `ArrayDeque` instances.
- 5 **Interview Core:** Maintain the largest `k` values with a size-`k` min-heap.
- 6 **Interview Core:** Demonstrate the difference among `List.of`, `Arrays.asList`, `List.copyOf`, and an unmodifiable view.
- 7 **SDE-2 Follow-up:** Define an API returning ordered results without exposing mutable internal membership.
- 8 **SDE-2 Follow-up:** Compare `ArrayList`, `HashSet`, and `TreeSet` for repeated membership queries, including build cost and ordering.

RECAP | Chapter summary

Choose collections from semantics: sequence, uniqueness, association, FIFO/LIFO, or priority. Use ArrayList, HashSet, HashMap, ArrayDeque, and PriorityQueue as the normal interview starting points, then select linked or sorted variants only for a stated contract. Know the everyday APIs, ownership/mutability differences, safe iteration rules, null and equality boundaries, and qualified costs before studying internals.

SDE-2 CORE CHAPTER

Chapter 14 - Exceptions, Input, Output, and Everyday Utilities

Interview code still crosses boundaries: text becomes numbers, files or consoles provide input, and library methods can reject invalid state. This chapter starts with a small rule: use exceptions to report an operation that cannot complete its contract; do not use them as a substitute for ordinary conditions.

A practical exception hierarchy

At the level needed for this book:

TEXT

```
Throwable
|- Error
|  `-- serious runtime or environment failure
`- Exception
   |- checked exception
   `-- RuntimeException
      `-- unchecked exception
```

- A **checked exception** must be caught or declared. `IOException` is the common example.
- An **unchecked exception** is a `RuntimeException` subtype. It often reports a broken precondition, invalid state, or programming error.
- An **Error** is not an exception. Code normally does not recover locally from failures such as `OutOfMemoryError` or `StackOverflowError`.

The hierarchy is a classification, not a quality ranking. Checked exceptions are not always better, and unchecked exceptions are not automatically bugs.

Catch, throw, and declare

`throw` creates or propagates a specific failure:

JAVA

```
static int requirePositive(int value) {
    if (value <= 0) {
        throw new IllegalArgumentException("value must be positive");
    }
    return value;
}
```

`throws` declares that a method may let a checked exception cross its boundary:

JAVA

```
static String readFirstLine(BufferedReader reader) throws IOException {  
    return reader.readLine();  
}
```

`try` and `catch` handle a failure when the method has enough context to respond:

JAVA

```
static Integer parseOrNull(String text) {  
    try {  
        return Integer.valueOf(text);  
    } catch (NumberFormatException exception) {  
        return null;  
    }  
}
```

Returning `null` is acceptable only when the caller's contract defines it clearly. At many boundaries, returning an explicit result type or throwing a contextual exception is clearer.

Catch narrowly and preserve the cause

A catch block should know what recovery means. This hides too much:

JAVA

```
try {  
    processRequest();  
} catch (Exception exception) {  
    // ignored  
}
```

It can suppress a programming defect and leave the caller believing work succeeded. Catch the failure you can handle. If you translate it, retain the original cause:

JAVA

```
try {  
    return Integer.parseInt(text);  
} catch (NumberFormatException exception) {  
    throw new IllegalArgumentException("invalid age: " + text, exception);  
}
```

Catch blocks run top to bottom. A more specific subtype must appear before its superclass; reversing them is a compile-time error because the specific block becomes unreachable.

finally and its limits

`finally` normally runs as control leaves the `try` or `catch`, whether the path returns or throws. It is useful for cleanup that cannot use try-with-resources.

Do not say that `finally` *always* runs. Abrupt process termination, JVM failure, or a non-terminating operation can prevent it. Also avoid returning from `finally`; that can replace an earlier return value or suppress an exception.

Try-with-resources owns cleanup

A resource implementing `AutoCloseable` can be declared in the `try` header:

JAVA

```
static String firstLine(String source) throws IOException {
    try (BufferedReader reader = new BufferedReader(new StringReader(source))) {
        return reader.readLine();
    }
}
```

Resources close in reverse declaration order. If the body and `close()` both fail, the body failure is primary and close failures are available through `getSuppressed()`.

The ownership question matters: the code that creates a resource usually closes it. A method should not silently close a caller-owned stream unless its contract says so.

Common interview failures

Failure	Typical cause	Better prevention or response
<code>NullPointerException</code>	dereferencing <code>null</code> or unboxing it	validate the boundary; avoid ambiguous null contracts
<code>ArrayIndexOutOfBoundsException</code>	index outside <code>[0, length)</code>	state and preserve the index invariant
<code>NumberFormatException</code>	malformed or out-of-range numeric text	validate/translate at the parsing boundary
<code>ArithmeticException</code>	integer divide by zero or exact-arithmetic overflow	validate divisor; choose an explicit overflow policy
<code>ClassCastException</code>	runtime object is not the forced subtype	avoid the cast or verify the runtime type
<code>IllegalArgumentException</code>	caller violates a method precondition	document and check the precondition
<code>ConcurrentModificationException</code>	unsupported structural mutation during iteration	use iterator removal or a two-phase update
<code>StackOverflowError</code>	recursion has no reachable base case or is too deep	prove termination; consider iteration

An exception stack trace is read from the first relevant application frame outward. The top line gives the type and message; the **Caused by** chain often preserves the boundary where the original failure began.

Input choices

Scanner: convenient and readable

JAVA

```
try (Scanner scanner = new Scanner("3 10 20 30")) {
    int size = scanner.nextInt();
    int[] values = new int[size];
    for (int index = 0; index < size; index++) {
        values[index] = scanner.nextInt();
    }
}
```

Scanner is convenient but does more parsing work. Be careful when mixing token reads such as `nextInt()` with `nextLine()`: the line separator after the token is still waiting to be consumed.

BufferedReader: explicit parsing and lower overhead

JAVA

```
try (BufferedReader reader = new BufferedReader(new StringReader("10 20 30"))) {
    String[] tokens = reader.readLine().trim().split("\\s+");
    int first = Integer.parseInt(tokens[0]);
}
```

For high-volume competitive input, a buffered token parser may avoid repeated string splitting. In interviews, many platforms call your method directly, so do not spend coding time on console input unless asked.

StringTokenizer is a lightweight legacy tokenizer still seen in contest code. Know how to read it, but prefer the clearest parser for the environment.

Output

`System.out.println` is enough for small examples. **PrintWriter** can buffer many writes:

JAVA

```
PrintWriter output = new PrintWriter(System.out);
output.println("ready");
output.flush();
```

Do not close `System.in` or `System.out` inside reusable interview methods.

Utility APIs worth knowing

Math

- `min`, `max`, `abs`, `sqrt`, `floor`, and `ceil` express common numeric operations.
- `addExact`, `subtractExact`, and `multiplyExact` throw `ArithmeticException` instead of silently wrapping.
- `Math.abs(Integer.MIN_VALUE)` is still negative because the positive magnitude is not representable as an `int`. The same rule applies to `Long.MIN_VALUE`.
- `Math.pow` returns `double`; it is not a general exact-integer exponentiation function.

Arrays

JAVA

```
int[] values = {3, 1, 2};  
Arrays.sort(values); // [1, 2, 3]  
int position = Arrays.binarySearch(values, 2); // 1  
int[] copy = Arrays.copyOf(values, values.length);
```

`binarySearch` requires a compatible sorted order. A negative result encodes an insertion point; it is not merely `-1`.

`Arrays.asList` deserves two warnings:

JAVA

```
List<String> words = Arrays.asList("a", "b"); // fixed-size view  
List<int> oneElement = Arrays.asList(new int[] {1, 2});
```

The first list permits `set` but not structural `add` or `remove`. The primitive array in the second example is one element; it is not boxed into `List<Integer>`.

Collections, Objects, wrappers, and Character

- `Collections.sort`, `reverse`, `min`, `max`, `frequency`, and `binarySearch` work with lists and their stated preconditions.
- `List.of`, `Set.of`, and `Map.of` create unmodifiable collections and reject null. `Unmodifiable` does not make mutable elements immutable.
- `Objects.equals(left, right)` compares safely when either reference may be null.
- `Objects.requireNonNull(value, "name")` makes a null boundary explicit.
- `Integer.parseInt`, `Long.parseLong`, and wrapper `compare` methods avoid hand-written parsing and subtraction comparators.
- `Character.isDigit`, `isLetter`, `isWhitespace`, and case-conversion methods are clearer than ASCII-only arithmetic when broader text is allowed.

Complete example

File: ExceptionsIoUtilitiesExample.java

JAVA (continued 1/2)

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.StringReader;
import java.util.Arrays;

public final class ExceptionsIoUtilitiesExample {
    static int[] parseLine(String line) {
        if (line == null || line.isBlank()) {
            return new int[0];
        }

        String[] tokens = line.trim().split("\\s+");
        int[] values = new int[tokens.length];
        for (int index = 0; index < tokens.length; index++) {
            try {
                values[index] = Integer.parseInt(tokens[index]);
            } catch (NumberFormatException exception) {
                throw new IllegalArgumentException(
                    "invalid integer at token " + index + ": " + tokens[index],
                    exception);
            }
        }
        return values;
    }
}
```

JAVA (continued 2/2)

```
    }

    static int[] readValues(String source) throws IOException {
        try (BufferedReader reader =
            new BufferedReader(new StringReader(source))) {
            return parseLine(reader.readLine());
        }
    }

    public static void main(String[] args) throws IOException {
        int[] values = readValues("30 10 20");
        Arrays.sort(values);
        System.out.println(Arrays.toString(values));

        try {
            readValues("4 nope 6");
        } catch (IllegalArgumentException exception) {
            System.out.println(exception.getMessage());
            System.out.println(exception.getCause().getClass().getSimpleName());
        }
    }
}
```

Expected output:

TEXT

```
[10, 20, 30]
invalid integer at token 1: nope
NumberFormatException
```


The parsing boundary adds token position while preserving the original cause. The resource is closed automatically, and sorting occurs only after input is valid.

Edge-case matrix

Case	Weak assumption	Deliberate choice
empty input line	token zero exists	define empty-input behavior
extra whitespace	split on one literal space	trim and split on <code>\\s+</code> , or tokenize
numeric overflow text	parse always succeeds	handle <code>NumberFormatException</code>
integer divide by zero	result is infinity	validate; integer division throws
<code>Math.abs(MIN_VALUE)</code>	always non-negative	widen or special-case the minimum
broad catch	every failure is recoverable	catch the failure you understand
<code>finally</code> return	harmless cleanup	can suppress an earlier result/failure
caller-owned reader	helper may close it silently	document ownership explicitly
binary search	array may be unsorted	sort or prove compatible ordering first

Interview room

Interviewer: When should you catch an exception instead of declaring it?

Model answer: I catch it where I can recover, translate it into a boundary-specific failure, or add useful context. If this method cannot make a correct decision, I let the failure propagate according to the API contract. I avoid catching broadly only to log and continue.

Follow-up: Does `finally` always execute?

Model answer: It normally executes as the `try` or `catch` completes, including return and exception paths, but not under every possible process or JVM termination. I also avoid control flow such as `return` inside `finally` because it can replace earlier outcomes.

Practice

- 1 **Foundation:** Parse a line of integers with a defined blank-line policy.
- 2 **Predict:** Determine the result of `Math.abs(Integer.MIN_VALUE)` and explain it.
- 3 **Debugging:** Repair catch blocks ordered from `Exception` to `IOException`.
- 4 **Debugging:** Fix a `Scanner` program that loses a line after `nextInt()`.
- 5 **Interview Core:** Translate a parsing failure while preserving its cause and token index.
- 6 **Interview Core:** Explain ownership for a method that accepts a caller-created reader.
- 7 **SDE-2 Follow-up:** Describe primary and suppressed exceptions in try-with-resources.

Chapter takeaway

Treat failures and I/O as explicit boundaries. Catch only when you can make a correct decision, preserve causes when translating, assign resource ownership, and learn utility preconditions instead of treating library calls as magic.

SDE-2 CORE CHAPTER

Chapter 15 - Practice Lab C - Wrappers, Generics, Collections, Exceptions, I/O

Start here: select an abstraction from the required behavior before naming an implementation. State ordering, duplicates, null policy, mutation, lookup direction, failure behavior, and resource ownership. Only then discuss expected cost or write code.

Answer K51-K75, predict O39-O57, compile every D39-D57 repair, and implement C39-C57 with one normal and two boundary tests. For exception and I/O tasks, mark the ownership boundary and decide whether the method catches, translates, or propagates each failure. Use Solution Studio C after attempting the work; a remembered API name without its contract does not count as mastery.

Knowledge check

- K51 [Foundation] Map each primitive to its wrapper class.
- K52 [Foundation] Distinguish parsing, boxing, and unboxing.
- K53 [Interview Core] Why can null unboxing fail far from the source of null?
- K54 [Interview Core] Why is wrapper identity not a value contract?
- K55 [Foundation] What type safety does `List<String>` provide?
- K56 [Foundation] Why can primitives not be generic arguments?
- K57 [Interview Core] What does the diamond operator infer?
- K58 [Interview Core] What operation does a simple upper bound permit?
- K59 [Foundation] Why is an enum safer than magic strings for finite state?
- K60 [Interview Core] Why should enum ordinal rarely be persisted?
- K61 [Foundation] Choose List, Set, Map, Queue, Deque, or PriorityQueue by need.
- K62 [Interview Core] Why is `LinkedList` not always faster for insertion?
- K63 [Interview Core] Qualify the expected-cost claim for `HashMap`.
- K64 [Foundation] Distinguish `offer/poll/peek` from throwing queue methods.
- K65 [Interview Core] Why does PriorityQueue iteration not establish order?
- K66 [Interview Core] State a safe comparator sign contract.
- K67 [Foundation] Distinguish checked exceptions, unchecked exceptions, and errors.
- K68 [Interview Core] Distinguish `throw` from `throws`.
- K69 [Interview Core] What does try-with-resources guarantee about close order?
- K70 [Interview Core] Why must catch blocks go from specific to general?
- K71 [Foundation] When is `Scanner` sufficient?
- K72 [Interview Core] Why can `BufferedReader` plus token parsing be preferable?
- K73 [Interview Core] Who should close a resource?
- K74 [Interview Core] Distinguish fixed-size, unmodifiable, and immutable collections.
- K75 [SDE-2 Follow-up] Why is broad `catch (Exception)` dangerous inside core logic?

Predict the output

- **O39 [Foundation]** Predict `Integer.parseInt("42") + 1`.
- **O40 [Interview Core]** Predict unboxing a null `Integer`.
- **O41 [Interview Core]** Predict `Integer.valueOf(127) == Integer.valueOf(127)`.
- **O42 [Interview Core]** Predict value equality for two boxed 128 values.
- **O43 [Foundation]** Predict `new Box<>("x").value()`.
- **O44 [Foundation]** Predict an enum switch result for `PAID`.
- **O45 [Foundation]** Predict a `HashSet` size after adding 2, 2, and 3.
- **O46 [Foundation]** Predict a frequency map count after merging three equal keys.
- **O47 [Foundation]** Predict FIFO removals from an `ArrayDeque`.
- **O48 [Foundation]** Predict LIFO removals using `push` and `pop`.
- **O49 [Interview Core]** Predict the first value polled from a min-heap containing 9, 3, 5.
- **O50 [Interview Core]** Predict comparator sign from `Integer.compare(MAX_VALUE, -1)`.
- **O51 [Foundation]** Predict `Arrays.asList("a","b").set(0,"x")` contents.
- **O52 [Interview Core]** Predict the result of adding to that fixed-size list.
- **O53 [Interview Core]** Predict the size of `Arrays.asList(new int[]{1,2})`.
- **O54 [Interview Core]** Predict which catch handles `NumberFormatException`.
- **O55 [Interview Core]** Predict resource close order for two declared resources.
- **O56 [Foundation]** Predict `Character.digit('A', 16)`.
- **O57 [SDE-2 Follow-up]** Predict `Math.addExact(Integer.MAX_VALUE, 1)`.

Debug the code

- **D39 [Foundation]** Repair `List<int>`.
- **D40 [Interview Core]** Repair a nullable wrapper used in arithmetic.
- **D41 [Interview Core]** Repair wrapper comparisons that use `==`.
- **D42 [Foundation]** Repair a raw `List` that permits mixed values.
- **D43 [Interview Core]** Repair a generic `first` method that accepts an empty list silently.
- **D44 [Foundation]** Replace string order states with an enum.
- **D45 [Interview Core]** Repair a switch that forgets a new enum state.
- **D46 [Foundation]** Repair a deduplication loop that still appends duplicates.
- **D47 [Interview Core]** Repair a frequency map that treats missing as null arithmetic.
- **D48 [Interview Core]** Replace legacy `Stack` with `ArrayDeque` operations.
- **D49 [Interview Core]** Repair a comparator implemented as subtraction.
- **D50 [Interview Core]** Repair code that assumes `PriorityQueue` iteration is sorted.
- **D51 [Interview Core]** Repair enhanced-for structural removal.
- **D52 [Interview Core]** Repair a mutable hash-map key.
- **D53 [Foundation]** Repair catch blocks ordered superclass before subclass.
- **D54 [Interview Core]** Repair a `finally` block that returns.
- **D55 [Interview Core]** Replace manual resource closing with `try-with-resources`.
- **D56 [Foundation]** Repair `Scanner` code that reads an empty token after mixed line/token calls.
- **D57 [SDE-2 Follow-up]** Repair library code that closes a caller-owned stream.

Small coding tasks

- **C39 [Foundation]** Parse an integer and report invalid input without hiding other defects.
- **C40 [Interview Core]** Convert nullable boxed counts to a defined primitive total safely.
- **C41 [Foundation]** Implement a generic immutable `Box<T>`.
- **C42 [Interview Core]** Implement a generic `first` method with an empty-input contract.
- **C43 [Foundation]** Model order status with an enum method `isTerminal`.
- **C44 [Foundation]** Use `ArrayList` to build a result sequence.
- **C45 [Foundation]** Deduplicate integers with `HashSet`.
- **C46 [Interview Core]** Preserve first-seen order with `LinkedHashSet`.
- **C47 [Interview Core]** Count word frequencies with `HashMap.merge`.
- **C48 [Foundation]** Implement FIFO operations with `ArrayDeque`.
- **C49 [Foundation]** Implement LIFO operations with `ArrayDeque`.
- **C50 [Interview Core]** Find three smallest values with `PriorityQueue`.
- **C51 [Interview Core]** Sort records using `comparingInt` and a tie-breaker.
- **C52 [Interview Core]** Remove matching list elements safely.
- **C53 [Foundation]** Catch and explain `NumberFormatException` at an input boundary.
- **C54 [Interview Core]** Implement a checked file-reading helper with a clear throws contract.
- **C55 [Interview Core]** Use try-with-resources with `BufferedReader`.
- **C56 [Foundation]** Read an integer array with `Scanner`.
- **C57 [SDE-2 Follow-up]** Read and validate a jagged matrix with `BufferedReader` and `StringTokenizer`.

Interview follow-ups

- **F27 [Interview Core]** When does boxing create hidden null or allocation risk?
- **F28 [SDE-2 Follow-up]** Why should a cache range never become business logic?
- **F29 [Interview Core]** What error does generics prevent before runtime?
- **F30 [Interview Core]** When should a generic bound stay out of a beginner API?
- **F31 [Interview Core]** How does enum evolution affect persistence and switches?
- **F32 [SDE-2 Follow-up]** Defend a collection choice using semantics before complexity.
- **F33 [Interview Core]** When is insertion order a correctness requirement?
- **F34 [Interview Core]** Why is ArrayDeque usually preferred to Stack?
- **F35 [SDE-2 Follow-up]** Explain PriorityQueue's head guarantee and iterator non-guarantee.
- **F36 [Interview Core]** When should an unchecked exception represent a violated precondition?
- **F37 [SDE-2 Follow-up]** Preserve a primary exception when resource closing also fails.
- **F38 [SDE-2 Follow-up]** Define resource ownership in a reusable parsing method.

Cumulative assessment 3

A03: In 45 minutes, read tokens, build a frequency map, keep the first-seen order of unique values, and print the top three by frequency using a safe comparator. Define null, malformed-input, tie, and empty-input behavior.

Cumulative assessment 4

A04: Review a 40-line solution that uses raw collections, wrapper identity, comparator subtraction, broad catches, and a caller-owned reader. Produce a behavior-preserving refactor and an evidence-based defect list.

SDE-2 CORE CHAPTER

Chapter 16 - Writing Interview-Quality Java

An interview solution is evaluated as code and as reasoning. The strongest answer is usually not the shortest one. It makes its contract, invariant, failure behavior, complexity, and tests easy to inspect.

Begin with a contract

Before typing the loop, clarify:

- Can the input be null or empty?
- Are values sorted, unique, non-negative, or bounded?
- May the method mutate the input?
- What should happen when no answer exists?
- Can arithmetic exceed `int`?
- Is any output order required?

Do not invent expensive defensive behavior for constraints the interviewer has already guaranteed. State the assumption, then code to it.

Choose types from the largest intermediate value

The input element type does not determine the safe accumulator type:

JAVA

```
static long calculateSum(int[] numbers) {  
    long sum = 0L;  
    for (int number : numbers) {  
        sum += number;  
    }  
    return sum;  
}
```

Widen before an operation that can overflow:

JAVA

```
long area = (long) width * height;
```

A cast after the multiplication is too late. For an exact-overflow contract, use `Math.addExact` or `Math.multiplyExact` and explain the thrown failure.

Make the control flow readable

Guard clauses keep the main path visible:

JAVA

```
static int firstPositive(int[] numbers) {  
    if (numbers == null) {  
        throw new IllegalArgumentException("numbers must not be null");  
    }  
    for (int number : numbers) {  
        if (number > 0) {  
            return number;  
        }  
    }  
    return -1;  
}
```

Returning `-1` is safe here only if the contract says it means "not found." Sometimes an empty optional, result object, or exception is clearer. Pick one contract rather than mixing signals.

Name the meaning, not the syntax

Weak:

JAVA

```
int f(int[] a) {  
    int x = 0;  
    for (int i = 0; i < a.length; i++) {  
        x += a[i];  
    }  
    return x;  
}
```

Improved:

JAVA

```
long calculateSum(int[] numbers) {  
    long sum = 0L;  
    for (int number : numbers) {  
        sum += number;  
    }  
    return sum;  
}
```

`i` and `j` are conventional for short index loops. `left`, `right`, `mid`, `row`, and `column` are also meaningful domain names. Avoid long names that merely repeat the type.

Use helper methods when they expose an idea

A helper is useful when it:

- names a reusable predicate or transition;
- isolates parsing or validation;
- removes duplicated logic;
- makes the main algorithm read like its explanation;
- is independently testable.

Do not create a class hierarchy or ten one-line methods to make a small interview solution look "enterprise." Appropriate abstraction is proportional to the problem.

Mutability and ownership

If sorting or overwriting the input simplifies the solution, ask whether mutation is allowed. When it is not, copy deliberately:

JAVA

```
int[] sorted = Arrays.copyOf(numbers, numbers.length);  
Arrays.sort(sorted);
```

Copying changes space complexity. Say so. A shallow copy isolates the container but not mutable referenced elements.

Standard libraries are tools, not substitutes for reasoning

Use a library operation when you know its contract and can explain why it fits. Interviewers may ask you to implement a structure manually to test mechanics, but production-quality code usually favors tested standard libraries.

Useful examples:

- `HashMap` for lookup state, with a stated average-case expectation rather than a universal guarantee;
- `ArrayDeque` for queue or stack behavior;
- `PriorityQueue` when only the next best element must be exposed;
- `Arrays.sort` when sorting is allowed and its cost fits;
- `Integer.compare` instead of subtraction in a comparator.

WHY IT WORKS | State the invariant while coding

For a one-pass lookup solution, a useful invariant might be:

Before processing index `i`, the map contains each value from indexes `[0, i)` paired with the index chosen by the contract.

The invariant answers three questions:

- 1 What does the state mean?
- 2 Why does the next operation preserve it?
- 3 Why does termination produce a correct result?

This is more persuasive than saying "the hash map makes it fast."

A complete refactoring: two sum

Suppose the contract is:

- input is non-null;
- return the first pair discovered while scanning left to right;
- the two indexes must be different;
- return an empty array when no pair exists;
- values and target are `int`, so calculate the complement in `long` before deciding whether it is representable.

File: `InterviewQualityExample.java`

JAVA

```

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public final class InterviewQualityExample {
    static int[] findTwoSumIndices(int[] numbers, int target) {
        if (numbers == null) {
            throw new IllegalArgumentException("numbers must not be null");
        }

        Map<Integer, Integer> earliestIndex = new HashMap<>();
        for (int index = 0; index < numbers.length; index++) {
            long complement = (long) target - numbers[index];
            if (complement >= Integer.MIN_VALUE && complement <= Integer.MAX_VALUE) {
                Integer earlier = earliestIndex.get((int) complement);
                if (earlier != null) {
                    return new int[] {earlier, index};
                }
            }
            earliestIndex.putIfAbsent(numbers[index], index);
        }
        return new int[0];
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(
            findTwoSumIndices(new int[] {3, 2, 4}, 6)));
        System.out.println(Arrays.toString(
            findTwoSumIndices(new int[] {3, 3}, 6)));
        System.out.println(Arrays.toString(
            findTwoSumIndices(new int[] {1, 2}, 9)));
    }
}

```

Expected output:

TEXT

```

[1, 2]
[0, 1]
[]

```

TRACE IT | Dry run

For [3, 2, 4] and target 6:

index	value	needed	map before check	action
0	3	3	{}	store 3 -> 0
1	2	4	{3=0}	store 2 -> 1
2	4	2	{3=0, 2=1}	return [1, 2]

The lookup happens before insertion, so one array element cannot pair with itself. `putIfAbsent` preserves the earliest index for duplicate values. The `long` subtraction avoids a wrapped complement.

Complexity statement

The method performs one scan, so expected time is $O(n)$ with a well-behaved hash map. Extra space is $O(n)$ in the worst case for stored entries. Java does not promise every `HashMap` operation is universally $O(1)$ under every key distribution and implementation path.

Test a small but discriminating set

Do not stop at the happy path. For the contract above, test:

- 1 ordinary solution in the middle;
- 2 duplicate values forming a pair;
- 3 no solution;
- 4 one or zero elements;
- 5 negative values;
- 6 integer-boundary values;
- 7 null, if the method owns that boundary.

A useful test earns its place by separating a correct implementation from a plausible bug.

Communicate while coding

A concise interview narration:

- 1 Restate the contract and assumptions.
- 2 Give the simple baseline.
- 3 Identify the bottleneck.
- 4 State the optimized representation and invariant.
- 5 Implement in small compiling steps.
- 6 Dry-run a representative and an edge case.
- 7 State time and space costs, including mutation or copying.
- 8 Name one trade-off or alternative.

Do not recite a memorized speech while the code contradicts it. Let variable names and state transitions match the explanation.

Common review comments

Review finding	Why it matters	Repair
accumulator is <code>int</code>	intermediate overflow	choose type from bounds
comparator subtracts	overflow can reverse order	use <code>Integer.compare</code> or comparator builders
helper returns null silently	ambiguous missing/error state	define an explicit contract
input sorted in place	caller data changed	ask permission or copy
map iteration assumed ordered	result can vary	use an ordered structure or sort output
nested conditions dominate	hard to prove main path	introduce guard clauses
one giant method	state has no names	extract meaningful transitions
many decorative classes	obscures the algorithm	keep abstraction proportional
complexity says only $O(n)$	hides space/assumptions	state both resources and operation model

Interview room

Interviewer: Why did you look up the complement before inserting the current value?

Model answer: The contract requires two distinct indexes. Checking the state from earlier indexes first enforces that directly. After the check, inserting the current value restores the invariant for the next iteration.

Follow-up: Why did you compute the complement as `long`?

Model answer: Subtracting two `int` values can overflow before a map lookup. Computing in `long` preserves the mathematical result. I only cast back after verifying that the complement is representable as an `int` key.

Practice

- Foundation:** Refactor a sum method with clear names and a safe accumulator.
- Predict:** Trace the two-sum state for `[3, 3]` and target `6`.
- Debugging:** Fix a solution that inserts before lookup and reuses one index.
- Debugging:** Replace a subtraction comparator and test integer extremes.
- Interview Core:** Add an explicit contract for null and missing-result behavior.
- Interview Core:** Produce a mutation-free solution and account for its copied storage.
- SDE-2 Follow-up:** Defend the hash-map complexity statement without claiming a hard constant-time guarantee.

Readiness check

You are ready for the Time and Space Complexity volume when you can:

- predict type promotion and overflow before running code;
- distinguish a primitive value, a reference value, and an object;
- explain Java pass-by-value using mutation and reassignment;
- traverse arrays and strings without off-by-one errors;
- choose a basic list, set, map, deque, or priority queue from required semantics;
- define method contracts and failure behavior;
- dry-run state against an invariant;
- state time, extra space, mutation, and important assumptions.

Chapter takeaway

Interview-quality Java makes reasoning visible. Clarify the contract, choose safe types, name the state, preserve an invariant, test discriminating cases, and state the real resource trade-offs.

SDE-2 CORE CHAPTER

Chapter 17 - Forty Java Interview Traps

These are behavior questions, not trivia contests. For each one: predict before running, identify the language or API rule, repair the code, and state the boundary an interviewer could change.

1. == versus equals

JAVA

```
String left = new String("java");
String right = new String("java");
System.out.println(left == right);
System.out.println(left.equals(right));
```

Predicted/actual output: `false`, then `true`. `==` compares reference identity; `equals` compares string content. Correct value test: `Objects.equals(left, right)` when either value may be null. Follow-up: what contract must a custom key's `equals` satisfy?

2. String-pool assumptions

JAVA

```
String first = "sde2";
String second = "sde" + "2";
System.out.println(first == second);
```

Actual output is `true` because this constant expression is folded and the literal may share the pooled instance. Do not infer that all equal strings have one object. Correct comparison: `first.equals(second)`. Follow-up: what changes if part of the concatenation is a variable?

3. new String(...)

JAVA

```
String pooled = "java";
String explicit = new String("java");
System.out.println(pooled == explicit);
```

Actual output: `false`. The constructor creates a distinct `String` object even though it is content-equal to the literal. Correct value test: `pooled.equals(explicit)`. Follow-up: what does `explicit.intern()` return?

4. Java pass-by-value

JAVA

```
static void replace(StringBuilder value) {  
    value = new StringBuilder("new");  
}  
StringBuilder caller = new StringBuilder("old");  
replace(caller);  
System.out.println(caller);
```

Actual output: `old`. The parameter receives a copy of the reference value; reassigning that copy does not reassign `caller`. Mutation such as `value.append("!")` would affect the shared object. Follow-up: draw both variables before and after reassignment.

5. Primitive overflow

JAVA

```
int maximum = Integer.MAX_VALUE;  
System.out.println(maximum + 1);
```

Actual output: `-2147483648`. `int` arithmetic wraps to the defined 32-bit two's-complement result; it does not widen automatically or throw. Correct when overflow is invalid: `Math.addExact(maximum, 1)`. Follow-up: when is modular wraparound intentional?

6. Overflow before assignment to long

JAVA

```
long wrong = 100_000 * 100_000;  
long correct = 100_000L * 100_000;  
System.out.println(wrong + " " + correct);
```

Actual output: `1410065408 100000000000`. Both operands of the first product are `int`, so overflow happens before assignment. Widen an operand before the operation. Follow-up: why is `(long) (a * b)` still too late?

7. Integer division

JAVA

```
System.out.println(5 / 2);  
System.out.println(5 / 2.0);
```

Actual output: `2`, then `2.5`. Integer division truncates toward zero. Correct an average by widening before division: `(double) sum / count`. Follow-up: what is `-5 / 2`?

8. Floating-point equality

JAVA

```
System.out.println(0.1 + 0.2 == 0.3);
```

Actual output: usually `false` by specified binary floating-point evaluation of these literals. Compare within a domain-defined tolerance, or use exact decimal/integer units when the domain requires them. Follow-up: why is one universal epsilon incorrect?

9. Narrowing conversion

JAVA

```
double value = 10.8;
int whole = (int) value;
System.out.println(whole);
```

Actual output: `10`. The cast truncates toward zero; it does not round. Correct rounding may use `Math.round` with explicit range handling. Follow-up: what happens for a value outside the `int` range?

10. Prefix versus postfix increment

JAVA

```
int value = 5;
int first = value++;
int second = ++value;
System.out.println(value + " " + first + " " + second);
```

Actual output: `7 5 7`. Postfix yields the old value; prefix increments before yielding. Correct interview style separates increments from larger expressions. Follow-up: predict `numbers[index++] = index` and explain evaluation order.

11. Short-circuit evaluation

JAVA

```
String text = null;
System.out.println(text != null && !text.isEmpty());
```

Actual output: `false`; the second operand is not evaluated. Replacing `&&` with boolean `&` evaluates both sides and throws. Correct guard: cheapest/safest prerequisite first. Follow-up: how can short-circuiting avoid an out-of-bounds read?

12. Null unboxing

JAVA

```
Integer count = null;  
// int value = count;
```

If reached, the assignment throws `NullPointerException` during unboxing. Correct by rejecting null, defining a default, or keeping absence explicit. Follow-up: why can `map.get(key) == 0` throw?

13. Integer wrapper caching

JAVA

```
Integer a = 127, b = 127;  
Integer c = 128, d = 128;  
System.out.println(a == b);  
System.out.println(c == d);
```

The first is `true` for the required cache; the second identity result must not be used as a value contract and is commonly `false`. Correct: `a.equals(b)` and `c.equals(d)`. Follow-up: why may a larger implementation cache not improve correctness?

14. Math.abs(Integer.MIN_VALUE)

JAVA

```
System.out.println(Math.abs(Integer.MIN_VALUE));
```

Actual output remains `-2147483648`: the positive magnitude is not representable as `int`. Correct: widen first with `Math.abs((long) value)` or use `Math.absExact` when overflow must throw. Follow-up: can `Math.abs` be used safely as a hash bucket index?

15. Math.abs(Long.MIN_VALUE)

JAVA

```
System.out.println(Math.abs(Long.MIN_VALUE));
```

Actual output remains negative for the same asymmetry at 64 bits. `Math.absExact(long)` throws on this input; `BigInteger` handles a larger exact magnitude. Follow-up: design a non-negative modulo-based index without `abs`.

16. Array aliasing

JAVA

```
int[] first = {1, 2};
int[] alias = first;
alias[0] = 9;
System.out.println(first[0]);
```

Actual output: 9. Assignment copies the array reference, not its elements. Correct independent storage: `int[] copy = first.clone()`. Follow-up: does cloning a two-dimensional array copy nested rows?

17. Shallow copying

JAVA

```
int[][] original = {{1}, {2}};
int[][] copy = original.clone();
copy[0][0] = 9;
System.out.println(original[0][0]);
```

Actual output: 9. Only the outer array was copied; row references are shared. Correct deep copy: clone each non-null row. Follow-up: what contract should define behavior for null rows?

18. Arrays.asList fixed-size behavior

JAVA

```
List<String> values = Arrays.asList("a", "b");
values.set(0, "x");
values.add("c");
```

`set` succeeds; `add` throws `UnsupportedOperationException`. The list is fixed-size and backed by an array. Correct resizable copy: `new ArrayList<>(Arrays.asList("a", "b"))`. Follow-up: are changes to the original array visible through the list?

19. Primitive arrays passed to Arrays.asList

JAVA

```
int[] numbers = {1, 2, 3};
List<int> list = Arrays.asList(numbers);
System.out.println(list.size());
```

Actual output: 1. `int[]` is one reference element; generic varargs do not box its contents. Correct: loop into `List<Integer>` or use `Arrays.stream(numbers).boxed().toList()`. Follow-up: is the stream result mutable?

20. String immutability

JAVA

```
String text = "java";
text.toUpperCase();
System.out.println(text);
```

Actual output: java. String methods return results; the original object does not change. Correct: `text = text.toUpperCase(Locale.ROOT)`. Follow-up: why can immutable strings still be referenced by mutable variables?

21. Repeated string concatenation

JAVA

```
String result = "";
for (int i = 0; i < n; i++) result += i;
```

This may repeatedly copy a growing immutable string and become quadratic in produced characters. Correct repeated construction: `StringBuilder builder = new StringBuilder();`
`builder.append(i);`. Follow-up: when can one expression's `+` remain readable and efficient?

22. char arithmetic

JAVA

```
char digit = '7';
System.out.println(digit - '0');
System.out.println(digit + 1);
```

Actual output: 7, then 56; binary numeric promotion produces `int`. Correct next character: `(char)(digit + 1)`. Validate ASCII digits before `digit - '0'`; `Character.digit(digit, 10)` handles broader digit definitions. Follow-up: what type is `'a' + 'b'`?

23. Unicode limitations of char

JAVA

```
String symbol = "\uD83D\uDE00";
System.out.println(symbol.length());
```

Actual output: 2. A Java `char` is a UTF-16 code unit; this supplementary code point uses a surrogate pair. Correct code-point count: `symbol.codePointCount(0, symbol.length())`. Follow-up: why is even a code point not always one user-perceived character?

24. Switch fall-through

JAVA

```
int value = 1;
switch (value) {
    case 1: System.out.print("one ");
    case 2: System.out.print("two");
}
```

Actual output: **one two**. Traditional colon cases fall through without **break**, **return**, or **throw**. Correct with breaks or a version-labeled switch expression. Follow-up: when is deliberate fall-through acceptable?

25. Missing default constructor

JAVA

```
class User { User(String name) {} }
// User user = new User();
```

The commented construction would not compile. Declaring any constructor prevents the compiler from supplying a no-argument constructor. Correct by calling the declared constructor or declaring an explicit no-argument constructor. Follow-up: what if the superclass lacks an accessible no-argument constructor?

26. Constructor chaining rules

JAVA

```
class Size {
    Size() {
        // System.out.println("start");
        // this(1); // illegal after another statement
    }
    Size(int value) {}
}
```

this(...) or **super(...)** must be the constructor's first statement. Correct: put **this(1);** first and move common work to the target constructor. Follow-up: can one constructor invoke both explicitly?

27. Static versus instance access

JAVA

```
class Counter {
    static int total;
    int local;
    static void reset() { total = 0; }
}
```

A static method has no receiver and cannot directly read `local`. Correct: pass a `Counter`, make the method an instance method, or operate only on class state. Follow-up: why is `instance.reset()` legal but misleading?

28. Overriding versus overloading

JAVA

```
class Base { String label(Object value) { return "base"; } }
class Child extends Base { String label(String value) { return "child"; } }
Base item = new Child();
System.out.println(item.label("x"));
```

Actual output: `base.label(String)` overloads rather than overrides `label(Object)`; compile-time selection uses `Base`'s visible signature. Correct override uses the identical parameter types and `@Override`. Follow-up: can return type alone overload a method?

29. Covariant return types

JAVA

```
class Parent { Number value() { return 1; } }
class Child extends Parent { @Override Integer value() { return 2; } }
```

This compiles: an overriding reference return type may be a subtype of the original return type. Primitive return types are not covariant. Follow-up: what return type is visible at a call site through a `Parent` reference?

30. Field hiding versus method overriding

JAVA

```
class Parent { int value = 1; int value() { return 1; } }
class Child extends Parent { int value = 2; @Override int value() { return 2; } }
Parent item = new Child();
System.out.println(item.value + " " + item.value());
```

Actual output: `1 2`. Fields use the declared reference type; instance methods dispatch to the runtime object. Correct design avoids same-name instance fields across a hierarchy. Follow-up: how do static methods behave?

31. Unsafe downcasting

JAVA

```
Object value = "java";  
// Integer number = (Integer) value;
```

If reached, the cast throws `ClassCastException`. Correct: keep a stronger type or guard with `instanceof`. Follow-up: why can a cast compile yet fail at runtime?

32. Exception catch ordering

JAVA

```
try {  
    Integer.parseInt("x");  
} catch (RuntimeException exception) {  
    System.out.println("runtime");  
} catch (NumberFormatException exception) { } // unreachable  
}
```

A catch for a subtype after its supertype is a compile-time error because the earlier catch already handles it. Correct: most specific catches first. Follow-up: when is one broad boundary catch appropriate?

33. finally behavior

JAVA

```
static int result() {  
    try { return 1; }  
    finally { return 2; }  
}
```

Actual result: 2; the `finally` return suppresses the pending return and is dangerous. Correct: never return from `finally`; reserve it for cleanup that does not replace the primary outcome. Also, `finally` is not guaranteed after forced process termination or fatal host failure. Follow-up: how does try-with-resources preserve suppressed close failures?

34. Mutable keys in HashMap

JAVA

```
List<Integer> key = new ArrayList<>(List.of(1));  
Map<List<Integer>, String> map = new HashMap<>();  
map.put(key, "value");  
key.add(2);  
System.out.println(map.get(key));
```

The lookup is commonly `null` because the key's hash code changed while the entry stayed in its old bucket. Correct: use an immutable key whose equality fields never change. Follow-up: why may iterating still reveal the entry?

35. Comparator subtraction overflow

JAVA

```
Comparator<Integer> broken = (left, right) -> left - right;  
System.out.println(broken.compare(Integer.MAX_VALUE, -1));
```

Subtraction overflows and can report the wrong sign. Correct: `Integer.compare(left, right)`. Follow-up: how do you add a deterministic tie-breaker?

36. PriorityQueue iteration is not sorted

JAVA

```
PriorityQueue<Integer> queue = new PriorityQueue<>(List.of(4, 1, 3, 2));  
for (int value : queue) System.out.print(value + " ");
```

Only `peek/poll` guarantee the head according to priority; iterator order is unspecified. Correct sorted consumption repeatedly polls a copy if the original must be preserved. Follow-up: what is the cost and mutation trade-off?

37. Modifying during enhanced-for

JAVA

```
List<Integer> values = new ArrayList<>(List.of(1, 2, 3));  
for (int value : values) {  
    if (value == 2) values.remove(Integer.valueOf(value));  
}
```

This can throw `ConcurrentModificationException`; fail-fast behavior is a bug detector, not a synchronization guarantee. Correct: use the iterator's `remove`, `removeIf`, or collect changes separately. Follow-up: why does the exact moment of failure not form an API guarantee?

38. Null handling

JAVA

```
String value = null;  
// System.out.println(value.equals("x"));
```

The call would throw. Correct null-safe choices depend on contract: reject with `requireNonNull`, compare as `"x".equals(value)`, or use `Objects.equals`. Follow-up: why can silently treating null as empty lose information?

39. Off-by-one array errors

JAVA

```
int[] values = {2, 4, 6};
for (int index = 0; index <= values.length; index++) {
    System.out.println(values[index]);
}
```

The fourth access throws `ArrayIndexOutOfBoundsException`; valid indexes are 0 through `length - 1`. Correct half-open loop: `index < values.length`. Follow-up: state its loop invariant.

40. Incorrect loop termination

JAVA

```
int value = 10;
while (value != 0) {
    value -= 3;
}
```

This never reaches zero and continues through negative values until eventual integer wraparound. Correct the contract and progress rule, for example `while (value > 0) value -= 3;`. Follow-up: how would you prove termination for arbitrary step sizes?

Rapid trap checklist

Before submitting, inspect equality, null, arithmetic type, overflow order, division, casts, aliasing, copy depth, loop bounds, comparator sign, collection mutation, constructor availability, dispatch kind, exception order, and resource ownership.

Cross-book boundary

Numeric traps continue in Number Systems and Bit Manipulation. Loop termination continues in Loop Mastery. Aliasing and copying continue in Arrays. Hash keys continue in Hashing. Priority ordering continues in Heaps. Deep exception and JVM failure behavior continue in Advanced Java and JVM.

RECAP | Chapter summary

Each trap has the same repair method: state the type and object contract, predict evaluation order, identify shared state, label the failure stage, and test a boundary that distinguishes the wrong mental model from the correct one.

SDE-2 CORE CHAPTER

Chapter 18 - Practice Lab D - Traps, Refactoring, and Readiness

Start here: this lab is a retrieval test across the full volume. Do not run output questions until a prediction and governing rule are written. For debugging items, preserve the intended contract, make the smallest correct change, and add a test that fails before the repair. For coding tasks, state time and auxiliary space after the code works.

Use three sessions: K76-K100 plus O58-O75, D58-D75 plus C58-C66, then C67-C75 plus F39-F50 and the assessments. Finish with the readiness rubric at the end of this chapter. If a miss comes from an earlier concept, return to that chapter instead of memorizing this lab's answer.

Knowledge check

- **K76 [Foundation]** Distinguish reference identity from semantic equality.
- **K77 [Interview Core]** State what string interning does and does not guarantee.
- **K78 [Interview Core]** Why does explicit String construction affect identity but not content?
- **K79 [Interview Core]** Separate mutation through a copied reference from parameter reassignment.
- **K80 [Interview Core]** Why can overflow happen before a wider assignment?
- **K81 [Interview Core]** When does floating comparison require a domain tolerance?
- **K82 [Foundation]** Why does narrowing truncate rather than round?
- **K83 [Interview Core]** What is special about primitive minimum magnitudes?
- **K84 [Interview Core]** What is copied by an array clone?
- **K85 [Interview Core]** Why is `Arrays.asList` different for object and primitive arrays?
- **K86 [Foundation]** Why must String method results usually be captured?
- **K87 [Interview Core]** When is `Character.digit` safer than subtracting `'0'`?
- **K88 [Interview Core]** How does fall-through differ from multi-label switch cases?
- **K89 [Foundation]** Why can a constructor not declare a return type?
- **K90 [Interview Core]** What is the first-statement rule for constructor chaining?
- **K91 [Interview Core]** Which members are dynamically dispatched?
- **K92 [Interview Core]** Why is downcasting evidence of a possible contract gap?
- **K93 [Interview Core]** Why does catch ordering affect compilability?
- **K94 [SDE-2 Follow-up]** Name conditions under which `finally` may not complete normally.
- **K95 [Interview Core]** Why must hash keys remain equality-stable?
- **K96 [Interview Core]** Why must comparator logic avoid subtraction?
- **K97 [Interview Core]** What does fail-fast collection iteration guarantee and not guarantee?
- **K98 [Foundation]** State the half-open array-index interval.
- **K99 [Interview Core]** What progress measure proves a loop terminates?
- **K100 [SDE-2 Follow-up]** List the final review order for contract, type, state, control flow, API, and complexity.

Predict the output

- **O58 [Interview Core]** Predict two equal new Strings under `==` and `equals`.
- **O59 [Interview Core]** Predict literal-plus-constant literal identity.
- **O60 [Interview Core]** Predict caller state after object mutation and parameter reassignment in one method.
- **O61 [Interview Core]** Predict `100_000 * 100_000` assigned to long.
- **O62 [Foundation]** Predict `-5 / 2` and `-5 % 2`.
- **O63 [Interview Core]** Predict `0.1 + 0.2 == 0.3`.
- **O64 [Foundation]** Predict `(int) -10.8`.
- **O65 [Interview Core]** Predict `Math.abs(Integer.MIN_VALUE)`.
- **O66 [Interview Core]** Predict a nested array mutation through a shallow copy.
- **O67 [Interview Core]** Predict add versus set on `Arrays.asList`.
- **O68 [Foundation]** Predict ignored `String.replace` result.
- **O69 [Interview Core]** Predict code-point count versus String length for a surrogate pair.
- **O70 [Foundation]** Predict traditional switch without breaks.
- **O71 [Interview Core]** Predict field and method access through parent reference/child object.
- **O72 [Interview Core]** Predict which overload runs for a base-typed argument.
- **O73 [Interview Core]** Predict a return in try replaced by return in finally.
- **O74 [Interview Core]** Predict map lookup after mutable-key mutation.
- **O75 [SDE-2 Follow-up]** Predict whether heap iteration order is guaranteed after seeing one sample run.

Debug the code

- **D58 [Foundation]** Replace object `==` with the correct null-safe value comparison.
- **D59 [Interview Core]** Repair code whose correctness depends on string interning.
- **D60 [Interview Core]** Repair a swap method that tries to swap caller references.
- **D61 [Interview Core]** Repair multiplication widened only after overflow.
- **D62 [Interview Core]** Repair floating equality for a currency-domain value.
- **D63 [Foundation]** Repair narrowing code that intended rounding.
- **D64 [Interview Core]** Repair a hash index derived with `Math.abs(hash)`.
- **D65 [Interview Core]** Repair a matrix deep-copy routine.
- **D66 [Interview Core]** Repair `Arrays.asList(primitiveArray)` under a `List<Integer>` requirement.
- **D67 [Foundation]** Capture a String transformation result.
- **D68 [Interview Core]** Repair character logic under a Unicode code-point contract.
- **D69 [Foundation]** Repair accidental switch fall-through.
- **D70 [Foundation]** Add a missing no-argument constructor deliberately.
- **D71 [Interview Core]** Repair a constructor that calls `this` after validation code.
- **D72 [Interview Core]** Repair code expecting static method overriding.
- **D73 [Interview Core]** Repair an unsafe cast by changing the base contract.
- **D74 [Interview Core]** Repair a catch hierarchy and preserve specific handling.
- **D75 [SDE-2 Follow-up]** Repair a loop whose termination depends on eventually hitting exactly zero.

Small coding tasks

- **C58 [Foundation]** Build a null-safe string equality helper.
- **C59 [Interview Core]** Demonstrate pool, constructor, and `intern` behavior without using identity for business logic.
- **C60 [Interview Core]** Build a pass-by-value diagram generator for one mutable object scenario.
- **C61 [Interview Core]** Implement safe average and safe product helpers.
- **C62 [Interview Core]** Implement domain-tolerant floating comparison with documented scale.
- **C63 [Interview Core]** Compute a non-negative bucket index without `Math.abs` overflow.
- **C64 [Interview Core]** Deep-copy a jagged object array under a stated element-copy contract.
- **C65 [Interview Core]** Convert a primitive array to a boxed resizable list.
- **C66 [Foundation]** Build text with `StringBuilder` using `append`, `insert`, `delete`, `setCharAt`, and `reverse`.
- **C67 [Interview Core]** Count Unicode code points without splitting surrogate pairs.
- **C68 [Foundation]** Rewrite a fall-through switch as an arrow switch expression.
- **C69 [Interview Core]** Demonstrate constructor chaining and initialization order.
- **C70 [Interview Core]** Demonstrate overload, override, field hiding, and static hiding separately.
- **C71 [Interview Core]** Implement safe polymorphic behavior without downcasting.
- **C72 [Interview Core]** Preserve a primary exception and inspect suppressed close failures.
- **C73 [Interview Core]** Create an immutable hash-map key and prove stable lookup.
- **C74 [Interview Core]** Poll a copy of a `PriorityQueue` into sorted order.
- **C75 [SDE-2 Follow-up]** Refactor a weak array-sum solution into interview-quality Java with contract, type choice, tests, and complexity.

Interview follow-ups

- **F39 [Interview Core]** Which equality operation belongs to primitives, strings, arrays, and custom keys?
- **F40 [SDE-2 Follow-up]** How can a seemingly harmless refactor change compile-time overload selection?
- **F41 [Interview Core]** Which boundary tests expose overflow-before-widening?
- **F42 [SDE-2 Follow-up]** How do you choose integer units versus decimal arithmetic versus tolerance?
- **F43 [Interview Core]** When is a shallow copy the intended contract?
- **F44 [SDE-2 Follow-up]** How would you document Unicode assumptions in a string algorithm?
- **F45 [Interview Core]** When is traditional switch fall-through useful enough to keep?
- **F46 [SDE-2 Follow-up]** Why are fields not polymorphic even when methods are?
- **F47 [Interview Core]** What should a boundary catch log or translate without hiding?
- **F48 [SDE-2 Follow-up]** Explain how mutable-key failure follows from the equality/hash contract.
- **F49 [Interview Core]** What ordering does PriorityQueue promise at each operation?
- **F50 [SDE-2 Follow-up]** Give a five-minute verbal review of an interview solution before coding.

Cumulative assessment 5

A05: In 60 minutes, diagnose a mixed program containing eight traps from this chapter, repair it, and explain why every corrected line follows Java's language or library contract. Add tests for null, empty, singleton, extremes, duplicates, and malformed input.

Final readiness assessment

Complete without notes:

- 1 Explain source-to-execution, primitive/reference values, promotion, pass-by-value, equality, construction, dispatch, generics, collection choice, exception ownership, and I/O boundaries.
- 2 Implement one array/string task and one map/queue task in Java 21.
- 3 Predict ten outputs chosen randomly from O01-O75 with at least eight correct.
- 4 Repair ten defects chosen randomly from D01-D75 with the governing rule stated.
- 5 Complete five coding tasks chosen randomly from C01-C75 and explain time and auxiliary space.
- 6 Answer ten follow-ups chosen randomly from F01-F50 without relying on folklore.

Readiness standard: at least 80 percent on prediction/debugging, compiling solutions for all five coding tasks, no incorrect pass-by-reference or equality explanation, and explicit boundary tests. If numeric overflow is weak, return to Number Systems ([DSA-02](#)). If loop invariants are weak, continue with Loop Mastery ([DSA-05](#)) after Time and Space Complexity ([DSA-01](#)). Otherwise proceed to Bit Manipulation, Arrays, Strings, and Hashing.

SDE-2 CORE CHAPTER

Chapter 19 - Solution Studio A - Language and Execution

Use these after attempting Practice Lab A. Each answer names the rule, because an unexplained answer is not interview-ready.

How to review: compare your rule before comparing your final value. Mark a response correct only if both match. For each miss, retype the smallest compiling example and change one operand, boundary, or control-flow condition to prove that the rule transfers. The grouped coding guidance is intentionally not a paste-ready answer: implement it, compile it, and defend the contract yourself.

Knowledge answers K01-K25

- **K01:** Source is human-authored `.java`; `javac` emits class-file bytecode; a compatible JVM executes the class-file operations on the host.
- **K02:** JDK is the development kit; a runtime/JRE is JVM plus runtime libraries/support; JVM executes class files. Modern packaging may not expose a separate JRE product.
- **K03:** `public` accessible, `static` no receiver required, `void` no result, `main` launcher entry name, `String[]` text argument array, `args` parameter name.
- **K04:** A public top-level class must normally use a same-named source file so the compiler/source lookup contract is satisfied.
- **K05:** Declaration introduces a variable; initialization gives its first value; assignment stores a value; reassignment replaces a prior value when permitted.
- **K06:** Java's definite-assignment analysis rejects a local read unless every reachable path assigns it; locals do not receive field defaults.
- **K07:** Integral: byte, short, int, long, char. Floating: float, double. boolean is logical.
- **K08:** The language fixes primitive value sets and numeric behavior, while field padding, object headers, alignment, and optimized representation are JVM/layout concerns.
- **K09:** A primitive variable stores its primitive value; a reference variable stores a reference value that may designate an object or be null.
- **K10:** Every argument expression produces a value copied into a parameter. For objects that value is a reference value, so mutation can be shared but parameter reassignment cannot escape.
- **K11:** `char` is an unsigned 16-bit UTF-16 code unit from 0 through 65535, not necessarily a full code point or glyph.
- **K12:** Expression type is decided before assignment; two int operands produce an int product, which may overflow before conversion to long.
- **K13:** Widening moves to a type whose range/representation is accepted implicitly; narrowing needs a cast and can discard range or precision.
- **K14:** Each byte promotes to int, so byte addition produces int.
- **K15:** Integer division truncates toward zero; remainder satisfies `a == (a / b) * b + a % b` and follows the dividend's sign when nonzero.
- **K16:** `&&` short-circuits after false; boolean `&` evaluates both operands.
- **K17:** `x += y` behaves like `x = (typeOfX) (x + y)` with one evaluation of the left side, so a narrowing conversion may be implicit.
- **K18:** Prefix updates then yields the new value; postfix yields the old value then updates.
- **K19:** A guard clause is clearer when invalid/special cases can exit early and keep the main path shallow.

- **K20:** Colon cases can fall through; arrow switch expression arms do not fall through and produce a value or complete an action.
- **K21:** **for** for explicit counter/update, **while** for condition-driven repetition, **do-while** for at least one pass, enhanced-for for value traversal without index mutation.
- **K22:** Unlabeled break exits only the nearest loop or switch; use a label or helper return deliberately for outer exit.
- **K23:** Name plus parameter types distinguish overloads. Return type is not part of overload selection at a call expression.
- **K24:** Base case stops recursion; recursive case reduces the problem toward it. Without progress, frames grow until StackOverflowError.
- **K25:** Class files are portable across compatible JVMs, but native libraries, OS services, paths, charsets, resources, timing, and environment remain host-dependent.

Output answers O01-O19

- **O01:** `2`; both operands are `int`, so the fractional part is discarded.
- **O02:** `-1`; remainder follows the dividend and preserves the division identity.
- **O03:** `-126`; 130 modulo 256 maps to that signed byte value.
- **O04:** `-2147483648`; overflow occurs as `int` before widening to `long`.
- **O05:** `3 4`; postfix yields 3, then `x` becomes 4.
- **O06:** `4 4`; prefix updates before yielding.
- **O07:** `15`; compound assignment includes the required narrowing cast.
- **O08:** `7`; digit code units are consecutive.
- **O09:** `66`; `char` participates in `int` arithmetic.
- **O10:** `false`; `fail()` is skipped by short-circuiting.
- **O11:** `0`; true on the left skips `++x`.
- **O12:** `2 1 0`; the condition includes zero and the decrement then reaches -1.
- **O13:** `1`; do-while runs once before testing.
- **O14:** `AB`; case 1 falls into case 2 without break.
- **O15:** `5`; arguments 2 and 3 are added and returned.
- **O16:** Caller primitive unchanged; only the copied parameter value increments.
- **O17:** Field changed; both reference values designate the same mutable object.
- **O18:** Caller reference unchanged; only the local parameter is reassigned.
- **O19:** Value `1.0`, type `double`; numeric conditional typing promotes the `int` arm.

Debug answers D01-D19

- **D01:** Add `;`; Java statements require termination.
- **D02:** Initialize `total` on every path before read, usually `int total = 0;` for an accumulator.
- **D03:** Rename the file `Main.java` or make the top-level class non-public/rename it to `Program`.
- **D04:** `long area = (long) width * height;;` widen before multiplication.
- **D05:** Use `value++`, `value += 1`, or an explicit checked/narrowing conversion; plain addition produces `int`.
- **D06:** Use `(double) sum / count` after checking `count != 0`.
- **D07:** Use `&&`; it skips the dereference when text is null.
- **D08:** Use `==`; assignment is not a boolean expression for `int`.
- **D09:** Use `i < array.length`; array indexes form a half-open interval.
- **D10:** Start at `array.length - 1` and continue while `i >= 0`.
- **D11:** Change the update or condition so a well-founded measure approaches the exit.
- **D12:** Add `break`/return/throw or use arrow switch syntax if the target Java version supports it.
- **D13:** Use an indexed loop or maintain an explicit, correctly updated counter.
- **D14:** Change parameter lists or method names; return type alone cannot overload.
- **D15:** Cast null to the intended parameter type or redesign ambiguous overloads.
- **D16:** Return on every reachable path or declare `void` if no value is part of the contract.
- **D17:** Add a reachable base case and make every recursive call reduce toward it.
- **D18:** `long magnitude = Math.abs((long) value);;` widening before `abs` makes 2147483648 representable.
- **D19:** Validate each failure with an early return/throw, then leave one shallow success path; verify original exception/result behavior with tests.

Coding guidance C01-C19

- **C01-C04:** Define null/empty/negative contracts first, then use one clear branch chain and test each boundary.
- **C05-C08:** Write the loop interval and invariant before code; test empty, singleton, first, and last positions.
- **C09:** Cast an operand before multiplication. **C10:** compare against `Integer.MIN_VALUE/MAX_VALUE` or use `Math.toIntExact`.
- **C11-C12:** The colon switch needs explicit completion; the arrow switch should return/assign one value per arm and label its Java version.
- **C13-C14:** Validate dimensions/bounds; overloading requires distinct parameter types and should preserve one conceptual contract.
- **C15-C17:** Assert caller state after primitive copy, shared mutation, and reference reassignment; the distinction is the copied value plus aliasing.
- **C18:** Check `character >= '0' && character <= '9'` or use `Character.digit` and reject `-1`.
- **C19:** Extract named predicates/helpers only when they clarify the contract; preserve failure order and output with tests.

Follow-up guidance F01-F13

- **F01-F02:** Separate source compatibility, class-file level, runtime linkage, dependency compatibility, and behavior; `--release` constrains language/API signature set, not third-party or environment behavior.
- **F03-F05:** Make numeric and evaluation policy explicit; parentheses communicate intent and short-circuiting prevents invalid/expensive work.
- **F06-F07:** Name a non-negative measure that strictly decreases. Prefer a helper return when a label would obscure ownership of the exit.
- **F08-F09:** Recompilation can select a new overload or become ambiguous; varargs is an array parameter with call-site packing syntax.
- **F10:** Recursion depth depends on frames, JVM configuration, and execution; the language promises no fixed safe depth.
- **F11:** Test odd sums, negative values, zero count, and large totals.
- **F12-F13:** Use compiler diagnostics, thrown stack traces, wrong outputs, and environment evidence rather than guesses.

A01 model: A strong answer widens before arithmetic, rejects invalid tokens at the input boundary, never catches unrelated runtime defects, uses a single pass, and tests no args, one value, negatives, zero, extreme ints, malformed text, and product overflow policy.

SDE-2 CORE CHAPTER

Chapter 20 - Solution Studio B - Arrays, Strings, and Objects

Use this studio only after Practice Lab B. Redraw every alias diagram before reading an answer, and distinguish object identity, value equality, copy depth, mutation, and runtime dispatch. A solution is complete when the code compiles, the object/reference picture matches the code, and one boundary test would fail under the original misconception.

The coding guidance gives design constraints rather than a single canonical implementation. Preserve those constraints while choosing clear names and the smallest useful API. Add missed rules to the same error log used in Practice Lab A.

Knowledge answers K26-K50

- **K26:** Declaration introduces an array reference; creation allocates fixed-length storage; initialization supplies initial element values or defaults.
- **K27:** Numeric zero, false, `\u0000` for char, and null for references.
- **K28:** Assignment copies the same array reference, so element mutation is shared.
- **K29:** Shallow copy duplicates one container while nested references remain shared; deep copy recursively duplicates the state required by the contract.
- **K30:** Arrays inherit identity-oriented `Object.toString`; use `Arrays.toString` or `deepToString`.
- **K31:** `equals` compares one-dimensional element values/references appropriately; `deepEquals` descends into nested arrays.
- **K32:** A two-dimensional array is an array of row references, and each row can have a different length or be null.
- **K33:** Its public contract exposes no mutation; operations return new results, enabling safe sharing and stable hashing.
- **K34:** Identity asks same object, content asks same characters, pooling may share selected strings but is not the value contract.
- **K35:** Empty has length zero; blank contains only whitespace under the chosen API; null means no String reference.
- **K36:** Use a builder when repeated updates would copy a growing immutable result, especially in loops.
- **K37:** One UTF-16 code unit.
- **K38:** Code unit is one char, code point is a Unicode scalar value represented by one/two units, grapheme is a user-perceived cluster that may include multiple code points.
- **K39:** Class defines type; object is instance; field stores state; method defines behavior.
- **K40:** A reference value designating an object or null, not the full object.
- **K41:** Only when the class declares no constructor, subject to superclass construction accessibility.
- **K42:** Receiver access/disambiguation, constructor chaining with `this(...)`, and briefly passing/returning the receiver.
- **K43:** It is shared, order-dependent state that leaks across tests and requires concurrency reasoning.
- **K44:** Same-package access is broad; cross-package subclass access is through inherited context, not arbitrary base/sibling instances.
- **K45:** It enables a short source name; it does not copy, load, or repack the type.
- **K46:** Encapsulation protects representation and invariants through domain operations; indiscriminate setters may destroy both.

- **K47:** Overloading is compile-time parameter-list selection; overriding is runtime instance dispatch; fields/static methods hide by declared type.
- **K48:** Interface is a role/contract, abstract class may share state/construction/implementation, concrete class is instantiable and complete.
- **K49:** For HAS-A, configurable collaboration, false subtype relationships, localized invariants, and test seams.
- **K50:** If two keys are equal, their hash codes must be equal, and equality/hash-relevant state must remain stable while keyed.

Output answers O20-O38

- **O20:** `0`; int array elements receive default zero.
- **O21:** The original shows the mutation because both variables alias one array.
- **O22:** Original nested element changes because outer clone shares row references.
- **O23:** `[1, 2]`; the utility formats contents.
- **O24:** Both rows are null; only the outer array was created.
- **O25:** `av`; start is inclusive and end exclusive.
- **O26:** Usually/defined for identical interned literal occurrence as true, but value code must use equals because nonliteral construction/runtime concatenation need not share identity.
- **O27:** `false`; explicit construction creates another object.
- **O28:** Original lowercase value; the returned String was ignored.
- **O29:** Apply operations in source order; the builder mutates in place, unlike String.
- **O30:** `2`; the supplementary code point occupies a surrogate pair.
- **O31:** Only the renamed object's field changes; objects have independent state.
- **O32:** Both aliases observe the new field value.
- **O33:** Target constructor body prints before the delegating constructor continues.
- **O34:** Three if each constructor increments once; static state is shared.
- **O35:** Parent field, child method; fields are hidden and instance methods dispatch.
- **O36:** Compile-time overload chosen from the declared type and argument type.
- **O37:** Implementing object's override runs through the interface reference.
- **O38:** Lookup can fail after key mutation because hash/equality no longer match original bucket placement.

Debug answers D20-D38

- **D20:** Use `< length`. **D21:** copy before mutation when preservation is the contract.
- **D22:** Allocate outer storage and clone/copy each non-null row. **D23:** use `Arrays.equals` or `deepEquals`.
- **D24:** use `Arrays.toString/deepToString`. **D25:** check each row and use `matrix[row].length`.
- **D26:** use `equals` or `Objects.equals`. **D27:** reject null, `Objects.equals`, or constant-first equality according to contract.
- **D28:** use a builder. **D29:** validate ASCII range or use `Character.digit`.
- **D30:** traverse code points with `codePoints()/codePointAt` and define grapheme limitations.
- **D31:** remove `void`; otherwise it is a method. **D32:** declare the no-argument constructor explicitly and delegate to validation.
- **D33:** pass a receiver or make it an instance method. **D34:** expose validated domain operations instead of raw assignment.
- **D35:** match parameters and add `@Override`. **D36:** guard with `instanceof` or move needed behavior to the base contract.
- **D37:** implement an interface's public method as public. **D38:** defensively copy on input and return an unmodifiable copy/view that cannot expose mutation.

Coding guidance C20-C38

- **C20-C25:** State shape, null-row, mutation, and copy-depth contracts; test empty, singleton, jagged, and null rows.
- **C26-C29:** Use equals for value, builder for repeated construction, and code-point/digit APIs when the contract exceeds ASCII code units.
- **C30-C34:** Validate constructors, keep fields private, make class/static ownership explicit, and implement equality from stable value state.
- **C35-C37:** Demonstrate one actual override with `@Override`; call through base/interface references to prove runtime dispatch. Abstract classes may construct shared state.
- **C38:** Inject the collaborator through the constructor, delegate through its interface, and test with a deterministic fake; no subtype claim is needed.

Follow-up guidance F14-F26

- **F14-F16:** Mutation is an API decision; copy depth and sorted-input prerequisites must be stated and tested.
- **F17-F18:** `char` is correct for an explicitly UTF-16-code-unit/ASCII contract; complexity units must match traversal.
- **F19-F21:** Final stops reassignment, not nested mutation; constructor dispatch can expose default subclass state; protected cross-package access is inheritance-scoped.
- **F22-F24:** Abstract class for shared state/template, interface for role/open implementations, composition for replaceable HAS-A behavior.
- **F25-F26:** Hash tables rely on equal hash codes; immutable owners copy mutable input and avoid exposing mutable internal aliases.

A02 model: A strong design uses immutable value identifiers, one guarded aggregate for state transitions, an enum rather than strings, an injected policy interface, no public mutable collections, and tests illegal transitions, duplicate identifiers, null collaborators, and alias attempts.

SDE-2 CORE CHAPTER

Chapter 21 - Solution Studio C - Wrappers, Generics, Collections, Exceptions, I/O

Use this studio after completing Practice Lab C. Review answers in this order: collection semantics, type/null behavior, ordering, failure contract, and cost. For I/O and exceptions, confirm who owns the resource and whether a catch can genuinely recover or translate at that boundary.

Retype every missed comparator, unboxing, fixed-size-list, heap-order, or try-with-resources example. Change the input so a folklore-based implementation fails; the distinguishing test is more valuable than memorizing the fixed line.

Knowledge answers K51-K75

- **K51:** Byte, Short, Integer, Long, Float, Double, Character, Boolean.
- **K52:** Parsing converts text to a value; boxing wraps a primitive; unboxing extracts its primitive value.
- **K53:** Collections/maps may carry a null wrapper through several calls before arithmetic or assignment triggers implicit unboxing.
- **K54:** Caching and allocation are implementation/language optimizations; equals defines wrapper value equality.
- **K55:** The compiler rejects non-String additions and returns String from get without an unchecked caller cast.
- **K56:** Generic type arguments are reference types under Java's generic type system; use wrappers for primitives.
- **K57:** It infers type arguments from target and constructor context, not runtime element inspection.
- **K58:** An upper bound exposes members of the bound, for example compareTo for a Comparable bound.
- **K59:** Enum constants are compiler-checked finite values and support typed behavior/switching.
- **K60:** Ordinals depend on declaration order and change when constants are inserted/reordered.
- **K61:** Sequence/List, uniqueness/Set, association/Map, FIFO/Queue, double-ended or stack/Deque, priority head/PriorityQueue.
- **K62:** Locating a position can be linear, nodes add allocation/cache costs, and iterator-position insertion is a narrower scenario.
- **K63:** Expected constant-time wording assumes suitable hashing/load and ordinary implementation behavior; worst cases and adversarial keys require qualification.
- **K64:** Offer/poll/peek signal failure with false/null; add/remove/element may throw. Check each implementation's null policy.
- **K65:** Heap structure guarantees only the head; its backing traversal is not globally sorted.
- **K66:** Return negative/zero/positive consistently, use compare helpers, and add deterministic tie-breakers where total order matters.
- **K67:** Checked exceptions need catch/declare; unchecked exceptions often signal programming/precondition failures; Errors indicate serious runtime conditions and are not ordinary recovery flow.
- **K68:** `throw` raises one exception object; `throws` declares possible propagation in a method signature.
- **K69:** Resources close in reverse declaration order; close failures may be suppressed behind a primary failure.
- **K70:** A general catch first would make a subtype catch unreachable.

- **K71:** Small, convenience-first interactive/token input where its overhead and token/line semantics are acceptable.
- **K72:** It offers explicit line/token control and commonly lower overhead for large input.
- **K73:** The component that creates/owns the resource usually closes it; borrowed streams remain open unless the API says otherwise.
- **K74:** Fixed-size forbids size changes, unmodifiable forbids mutation through that view/reference, immutable promises no observable content change.
- **K75:** It can hide null, index, cast, and invariant defects, blur failure contracts, and continue with corrupt state.

Output answers O39-O57

- **O39:** 43; text parses to primitive 42 then adds 1.
- **O40:** NullPointerException during unboxing.
- **O41:** `true` for the required small wrapper cache, not a general value-comparison pattern.
- **O42:** `equals` is true regardless of identity.
- **O43:** `x`; generic return type is String.
- **O44:** The matching PAID arm's result; enum matching is typed.
- **O45:** 2; duplicate 2 is not added.
- **O46:** 3; each merge increments the same key.
- **O47:** Values leave in insertion order from the opposite end.
- **O48:** Most recently pushed leaves first.
- **O49:** 3; natural-order min-heap exposes minimum.
- **O50:** Positive 1; compare avoids overflow.
- **O51:** `[x, b]`; element replacement is allowed.
- **O52:** UnsupportedOperationException; size is fixed.
- **O53:** 1; the primitive array is one reference element.
- **O54:** The first compatible catch in source order; specific-before-general is required.
- **O55:** Reverse declaration order.
- **O56:** 10; hexadecimal A represents ten.
- **O57:** ArithmeticException; exact arithmetic rejects overflow.

Debug answers D39-D57

- **D39:** `List<Integer>`. **D40:** reject/default/check before unboxing.
- **D41:** use `equals/Objects.equals`. **D42:** supply a type argument and remove unchecked mixed usage.
- **D43:** reject empty or return Optional under an explicit contract. **D44:** declare enum constants and parse external text at the boundary.
- **D45:** cover all states or use a deliberate default with evolution trade-off. **D46:** let `Set.add`'s boolean decide whether to append.
- **D47:** use `merge`, `getOrDefault`, or `compute`. **D48:** map `push/pop/peek` to `ArrayDeque`.
- **D49:** use primitive compare helpers and `thenComparing`. **D50:** poll a copy or sort a snapshot.
- **D51:** use `iterator.remove/removeIf/separate` changes. **D52:** use an immutable, equality-stable key.
- **D53:** order specific to general. **D54:** remove `return/throw` that masks the primary completion from `finally`.
- **D55:** declare owned resources in try parentheses. **D56:** consume the pending newline deliberately or use one tokenization model consistently.
- **D57:** document borrowed ownership and do not close it; wrap only resources the method owns.

Coding guidance C39-C57

- **C39-C43:** Keep conversion at boundaries, define null/empty behavior, retain generic type safety, and avoid enum ordinals in external contracts.
- **C44-C49:** Select by required semantics and assert result order/membership/frequency, including empty and duplicates.
- **C50-C52:** Use heap head operations, safe comparators, and supported iterator mutation; state whether input is preserved.
- **C53-C55:** Catch only exceptions you can translate/recover from, propagate checked boundaries honestly, and use `try-with-resources` for owned resources.
- **C56-C57:** Validate declared sizes, token counts, missing lines, extra tokens, and jagged row lengths; separate parsing from algorithm logic.

Follow-up guidance F27-F38

- **F27-F30:** Boxing can hide allocation and nullable state; cache identity is irrelevant; bounds belong only when operations require them.
- **F31:** Persist stable external codes, not ordinal; decide whether switches should fail compilation or use an explicit unknown policy.
- **F32-F35:** Start with ordering, uniqueness, key lookup, FIFO/LIFO/priority contracts, then qualify costs; heaps promise head order only.
- **F36-F38:** Illegal preconditions can be unchecked; try-with-resources records close failures as suppressed; resource ownership is part of the method contract.

A03 model: Use `HashMap.merge`, `LinkedHashSet` when encounter order matters, a small immutable entry for heap candidates, `Integer.compare/comparingInt` plus a stable tie-breaker, and bounded heap logic if only top three are needed.

A04 model: Parameterize every collection, replace identity with value equality, replace subtraction with compare helpers, narrow catches to the boundary, and leave caller-owned resources open. Re-run behavior tests after each refactor.

SDE-2 CORE CHAPTER

Chapter 22 - Solution Studio D - Traps and Readiness

This final studio is a calibration tool, not a shortcut. Check the rule, the predicted behavior, the repair, and the follow-up separately. If one layer is weak, return to the originating chapter and rerun its companion example before retrying the item.

Use the readiness interpretation at the end to decide what to study next. A high score with incorrect pass-by-value, equality, overflow, aliasing, or dispatch explanations is not a pass, because those misconceptions contaminate later array, hashing, recursion, and graph solutions.

Knowledge answers K76-K100

- **K76:** Identity asks whether two references designate the same object; semantic equality asks whether values satisfy a type's equals contract.
- **K77:** Interning provides a canonical pooled reference for selected string content; it does not mean every equal String is created only once or justify identity comparison.
- **K78:** The constructor allocates a distinct String object while preserving the same character content.
- **K79:** Both parameters and callers may designate one object, so mutation is visible; assignment changes only the local reference copy.
- **K80:** Expression operand types determine arithmetic before assignment conversion.
- **K81:** When representation error is acceptable and the domain defines scale-aware absolute/relative tolerance; exact financial/identifier quantities often need another representation.
- **K82:** Java's floating-to-integral cast discards the fractional part toward zero by rule.
- **K83:** Two's-complement signed ranges contain one more negative value, so the minimum's positive magnitude is unrepresentable in the same type.
- **K84:** One array object and its element values; reference elements still designate the same nested objects.
- **K85:** Object arrays can serve as the generic varargs array; a primitive array is itself one Object/reference element.
- **K86:** Strings are immutable, so transformations normally return a new or existing result rather than mutate the receiver.
- **K87:** It validates and converts according to a radix and can recognize more than ASCII digits; **-1** signals failure.
- **K88:** Fall-through executes subsequent statements; multi-label arrow cases select one body for several labels without fall-through.
- **K89:** Constructor syntax is distinguished by class name and absence of a return type; adding one declares a method.
- **K90:** An explicit **this(...)** or **super(...)** must be first, and only one is explicit in a constructor.
- **K91:** Overridable instance methods. Fields and static methods are selected/hide by declared type.
- **K92:** Repeated downcasts can mean needed behavior is missing from the base interface or the model exposes the wrong abstraction.
- **K93:** A supertype catch first consumes all subtype instances, making later subtype handling unreachable.
- **K94:** It may return/throw abruptly itself, or not run/completely execute under forced termination, host/process failure, or nontermination.
- **K95:** Hash lookup uses the insertion-time hash/equality placement; changing those values breaks the lookup path.

- **K96:** Subtraction can overflow and reverse comparator sign; compare helpers preserve ordering laws.
- **K97:** It may detect structural interference and throw, but timing is not guaranteed and it provides no thread-safety contract.
- **K98:** `[0, length)`.
- **K99:** A well-founded non-negative quantity that strictly decreases (or bounded quantity that increases) on every continuing iteration.
- **K100:** Contract/input, types/arithmetic, aliases/mutation, bounds/termination, API guarantees/failures, then time/auxiliary space and tests.

Output answers O58-O75

- **O58:** `false` then `true`; distinct identity, equal content.
- **O59:** Often/defined constant folding yields the pooled literal reference, but use equals for correctness.
- **O60:** Mutation is visible; parameter reassignment is not.
- **O61:** `1410065408`; int overflow precedes long assignment.
- **O62:** `-2` and `-1`; division truncates toward zero and remainder preserves the identity.
- **O63:** `false`; these binary floating values do not equal the literal 0.3 exactly.
- **O64:** `-10`; truncation is toward zero.
- **O65:** `Integer.MIN_VALUE`, still negative.
- **O66:** Original nested content changes because row references are shared.
- **O67:** set works; add throws `UnsupportedOperationException`.
- **O68:** Original String unchanged because result was ignored.
- **O69:** String length 2 and code-point count 1 for the example.
- **O70:** All reached later colon cases print until a terminating statement.
- **O71:** Parent field and child override result.
- **O72:** Compile-time overload applicable to declared argument/reference types.
- **O73:** Finally's return replaces the pending return; avoid this code.
- **O74:** Lookup may return null because the key moved logically but not structurally.
- **O75:** No sorted-iteration conclusion is valid; only head operations carry the priority guarantee.

Debug answers D58-D75

- **D58:** primitives use operators; nullable objects use `Objects.equals`; arrays use `Arrays.equals/deepEquals`; custom keys need `equals/hashCode`.
- **D59:** replace identity checks with content checks; intern only for an explicitly designed canonicalization mechanism.
- **D60:** return the swapped pair/container or mutate caller-owned container elements; Java cannot swap caller variables by parameter reassignment.
- **D61:** widen an operand before multiplication or use `Math.multiplyExact` in the chosen width.
- **D62:** use integer minor units or `BigDecimal` for exact currency policy; do not patch with arbitrary epsilon.
- **D63:** use `Math.round/floor/ceil` according to contract and validate range.
- **D64:** use `Math.floorMod(hash, capacity)` with positive capacity, or bit masking only under a justified power-of-two contract.
- **D65:** copy each row/nested mutable element to the depth promised and handle null explicitly.
- **D66:** loop/stream-box into a new `ArrayList<Integer>`.
- **D67:** assign or return the transformation result.
- **D68:** traverse code points and state whether grapheme-cluster behavior is required.
- **D69:** add termination or use arrow syntax.
- **D70:** declare one and delegate to the validated constructor.
- **D71:** delegate first, then validate in the target constructor/factory.
- **D72:** call static methods through class names and do not expect runtime dispatch.
- **D73:** put needed behavior on the polymorphic contract or use a checked pattern at a true type boundary.
- **D74:** specific catches first; translate only at an ownership boundary and preserve cause.
- **D75:** use an inequality or modular reachability/progress proof; do not rely on hitting an exact sentinel accidentally.

Coding guidance C58-C75

- **C58-C60:** Define null and identity contracts, use `Objects.equals`, and illustrate copied reference values explicitly.
- **C61-C63:** Widen before arithmetic, define division/rounding, and use `floorMod` for non-negative indexing.
- **C64-C67:** State copy depth, box explicitly, use builder for mutable construction, and traverse code points under Unicode contracts.
- **C68-C71:** Label switch version, keep constructor invariants centralized, separate all dispatch kinds, and prefer base behavior over casts.
- **C72-C74:** Inspect `getSuppressed`, use immutable keys, and poll a defensive heap copy when sorted output plus preservation are required.
- **C75:** A strong refactor names `calculateSum`, decides `int` versus `long` from constraints, handles null by contract, uses enhanced-for where index is irrelevant, tests empty/singleton/extremes, and states $O(n)$ time/ $O(1)$ auxiliary space.

Follow-up guidance F39-F50

- **F39-F41:** Match equality to representation, remember overload selection is compile-time, and test products/sums around primitive limits.
- **F42-F44:** Numeric and Unicode representations are domain contracts, not one-size-fits-all implementation details.
- **F45-F46:** Keep deliberate fall-through only when grouped behavior is unmistakable; fields are resolved from declared type while instance methods dispatch.
- **F47-F49:** Boundary catches translate with cause/context; mutable keys violate stable lookup; heaps promise the best head after each mutation.
- **F50:** Restate input/output, constraints, examples, edge cases, type/structure choice, invariant, algorithm, complexity, and tests before implementation.

A05 model: Find failures by stage, repair the narrowest violated contract, then prove with distinguishing boundary tests. Avoid behavior-changing cleanup until the original defect is covered.

Final readiness interpretation

- Below 80 percent on K/O/D: revisit the named chapter and re-run the examples.
- Compiling but poorly explained C tasks: practice verbal invariants and contracts.
- Incorrect equality/pass-by-value/type-promotion explanation: do not advance; these faults spread into every DSA book.
- Strong fundamentals but slow implementation: move to Loop Mastery and Arrays while keeping this book as a reference.

SDE-2 CORE CHAPTER

Chapter 23 - Java Fundamentals Wave 1: Complete Solution Studio

These solutions belong after the beginner-first chapters. Try each exercise before reading its solution. All eleven are executable in `code/fundamentals/FundamentalsSolutionChecks.java`; a successful run prints `PASS 11 Java Fundamentals solution checks`.

1. Average without int overflow

JAVA

```
static double safeAverage(int[] numbers) {
    Objects.requireNonNull(numbers, "numbers");
    if (numbers.length == 0) {
        throw new IllegalArgumentException("numbers must not be empty");
    }
    long sum = 0L;
    for (int number : numbers) {
        sum += number;
    }
    return (double) sum / numbers.length;
}
```

The accumulator is `long` from the first addition. The cast before division prevents integer truncation. Empty input has no defined arithmetic mean, so the contract rejects it explicitly.

2. Widen before multiplication

JAVA

```
static long safeMultiply(int left, int right) {
    return (long) left * right;
}
```

At least one operand is widened before `*`, so binary numeric promotion makes the multiplication a `long` operation. `(long) (left * right)` would preserve an already wrapped `int` result.

3. Explain mutation and reassignment under pass-by-value

JAVA

```
static void rename(Person person) {
    person.name = "Updated";
}

static void replace(Person person) {
    person = new Person("Replacement");
}
```

Both methods receive a copy of the caller's reference value. `rename` follows that copied reference to the shared object, so mutation is visible. `replace` changes only the method's local parameter variable, so the caller still refers to the original object.

4. Deep-copy a jagged primitive matrix

JAVA

```
static int[][] deepCopy(int[][] matrix) {
    Objects.requireNonNull(matrix, "matrix");
    int[][] copy = new int[matrix.length][];
    for (int row = 0; row < matrix.length; row++) {
        copy[row] = matrix[row] == null ? null : matrix[row].clone();
    }
    return copy;
}
```

Cloning only the outer array would still share every row. This solution allocates a new outer array and a new primitive array for each non-null row. It preserves jagged row lengths and the intentional null-row state.

5. Convert and validate a decimal character

JAVA

```
static int decimalDigit(char character) {
    int value = Character.digit(character, 10);
    if (value < 0) {
        throw new IllegalArgumentException("not a decimal digit: " + character);
    }
    return value;
}
```

`character` – `'0'` is compact when the contract guarantees ASCII `'0'` through `'9'`. `Character.digit` makes the validation explicit and supports decimal digits recognized by Java's Unicode data.

WHY IT WORKS | 6. Preserve a class invariant

JAVA

```
final class BankAccount {
    private long balanceCents;

    BankAccount(long openingBalanceCents) {
        if (openingBalanceCents < 0) {
            throw new IllegalArgumentException("negative opening balance");
        }
        balanceCents = openingBalanceCents;
    }

    void withdraw(long cents) {
        if (cents <= 0 || cents > balanceCents) {
            throw new IllegalArgumentException("invalid withdrawal");
        }
        balanceCents -= cents;
    }
}
```

The invariant is `balanceCents >= 0`. Construction establishes it, and the only mutation shown preserves it. A public `setBalance` would expose invalid states instead of modeling a domain operation.

7. Write a type-safe generic last

JAVA

```
static <T> T last(List<T> values) {
    Objects.requireNonNull(values, "values");
    if (values.isEmpty()) {
        throw new IllegalArgumentException("values must not be empty");
    }
    return values.get(values.size() - 1);
}
```

The same method preserves the element type for `List<String>`, `List<Integer>`, or another reference type. The empty-input behavior is part of the contract instead of an accidental index failure.

8. Choose a deterministic most-frequent value

JAVA

```
static String mostFrequent(List<String> words) {
    if (words.isEmpty()) {
        throw new IllegalArgumentException("words must not be empty");
    }
    Map<String, Integer> frequencies = new HashMap<>();
    for (String word : words) {
        frequencies.merge(word, 1, Integer::sum);
    }

    String bestWord = null;
    int bestCount = -1;
    for (Map.Entry<String, Integer> entry : frequencies.entrySet()) {
        String word = entry.getKey();
        int count = entry.getValue();
        if (count > bestCount
            || (count == bestCount && word.compareTo(bestWord) < 0)) {
            bestWord = word;
            bestCount = count;
        }
    }
    return bestWord;
}
```

Counting alone does not define a tie. This solution returns the lexicographically smaller word when counts match, so it never depends on `HashMap` iteration order. The executable version also validates null inputs and null elements.

9. Add context to a parsing failure

JAVA

```
try {
    values[index] = Integer.parseInt(tokens[index]);
} catch (NumberFormatException exception) {
    throw new IllegalArgumentException(
        "invalid integer at token " + index + ": " + tokens[index],
        exception);
}
```

The boundary reports which token failed while retaining the original `NumberFormatException` as the cause. Catching `Exception` or discarding the cause would make diagnosis weaker.

10. Compare without arithmetic overflow

JAVA

```
record Candidate(String name, int score) {}

static final Comparator<Candidate> CANDIDATE_ORDER =
    Comparator.comparingInt(Candidate::score)
        .reversed()
        .thenComparing(Candidate::name);
```

Subtraction can overflow for extreme scores and can violate the comparator contract. Comparator builders perform a safe primitive comparison and make the name tie-break explicit.

11. Protect a complement calculation from overflow

JAVA

```
long complement = (long) target - numbers[index];
if (complement >= Integer.MIN_VALUE && complement <= Integer.MAX_VALUE) {
    Integer earlier = earliestIndex.get((int) complement);
    if (earlier != null) {
        return new int[] {earlier, index};
    }
}
```

Both inputs are `int`, so subtracting them directly would also be an `int` operation. Widening the target before subtraction preserves the mathematical difference. A cast back is safe only after the range check. Looking up before inserting the current value also guarantees two distinct indexes.

Final dry-run prompt

For every solution, answer four questions aloud:

- 1 What is the input and failure contract?
- 2 What invariant is preserved?
- 3 Which edge case would break the most tempting incorrect version?
- 4 What time, extra space, and mutation behavior does the solution have?

If an answer is unclear, return to the corresponding chapter before moving to Time and Space Complexity.

SDE-2 CORE CHAPTER

Chapter 24 - Java Syntax and Language Quick Reference

This appendix is a recall sheet, not a substitute for the language chapters. Every compact rule hides edge cases. When correctness depends on an exact compile-time rule, use the Java Language Specification linked in Appendix G.

After Java Foundations, continue to [Java-SDE2-DSA-01-Time-and-Space-Complexity.pdf](#), then use [Java-SDE2-DSA-02-Number-Systems-and-Math-Foundations.pdf](#) for a printable numeric quick-reference with worked algorithms. For the full focused sequence, open the [series index](#).

Source files, packages, and launch

A conventional source file begins with an optional package declaration, followed by imports and type declarations:

JAVA

```
package com.example.orders;

import java.time.Instant;
import java.util.List;

public final class OrderBatch {
    private final List<String> ids;
    private final Instant createdAt;

    public OrderBatch(List<String> ids, Instant createdAt) {
        this.ids = List.copyOf(ids);
        this.createdAt = createdAt;
    }

    public static void main(String[] args) {
        var batch = new OrderBatch(List.of("demo"), Instant.now());
        System.out.println("orders=" + batch.ids.size());
    }
}
```

At most one top-level public type appears in a source file, and its name normally matches the file name. A package name maps to a directory layout by convention and by most build tools. The unnamed package is unsuitable for maintained applications because named packages cannot import from it.

Common commands:

TEXT

```
javac --release 21 -d out src/com/example/orders/OrderBatch.java
java -cp out com.example.orders.OrderBatch
java --enable-preview --source 21 PreviewExample.java
javap -classpath out -c -v com.example.orders.OrderBatch
```

`--release 21` selects a supported language level and matching documented Java SE APIs. It is safer for cross-compilation than setting only `-source` and `-target`. Java 21 features covered in this book do not require preview mode when they were final in that release.

Primitive types, references, and default values

Java has eight primitive types:

Type	Width or model	Representative range or values	Default field value
<code>byte</code>	8-bit signed	-128 through 127	<code>0</code>
<code>short</code>	16-bit signed	-32,768 through 32,767	<code>0</code>
<code>int</code>	32-bit signed	-2^{31} through $2^{31} - 1$	<code>0</code>
<code>long</code>	64-bit signed	-2^{63} through $2^{63} - 1$	<code>0L</code>
<code>char</code>	16-bit unsigned UTF-16 code unit	<code>\u0000</code> through <code>\uffff</code>	<code>\u0000</code>
<code>float</code>	IEEE 754 binary32	finite values, infinities, NaN	<code>0.0f</code>
<code>double</code>	IEEE 754 binary64	finite values, infinities, NaN	<code>0.0d</code>
<code>boolean</code>	logical value	<code>true</code> , <code>false</code>	<code>false</code>

Reference variables hold reference values or `null`; they do not contain an object inline as a language guarantee. Fields and array elements receive defaults. Local variables must be definitely assigned before use.

Integer arithmetic on `byte`, `short`, and `char` is normally promoted to `int`. Integer overflow wraps using two's-complement arithmetic; it does not throw. Use `Math.addExact`, `subtractExact`, and related methods when overflow must be rejected. Floating-point NaN is unequal to every value, including itself; use `Double.isNaN` rather than equality.

Useful literals:

JAVA

```
int decimal = 1_000_000;
int hexadecimal = 0xFF;
int binary = 0b1010_0110;
long longValue = 9_000_000_000L;
float ratio = 0.25F;
char newline = '\n';
String textBlock = """
    line one
    line two
    """;
```

Conversions, boxing, and numeric comparison

Widening primitive conversions such as `int` to `long` are implicit, although a large integer can lose precision when converted to floating point. Narrowing conversions require a cast and can discard high bits or truncate a fractional component.

Boxing converts a primitive to a wrapper, and unboxing extracts the primitive. Unboxing `null` throws `NullPointerException`. Wrapper identity is not a numeric comparison:

JAVA

```
Integer a = 1_000;
Integer b = 1_000;
boolean wrongQuestion = a == b;           // compares references
boolean valueEquality = a.equals(b);       // compares int values
```

Do not rely on wrapper caches. Prefer primitives in hot numeric loops unless nullability, generic APIs, or object semantics are required.

For money, binary floating point is usually the wrong domain model. Use an integer minor-unit representation when scale is fixed, or use `BigDecimal` with an explicit scale and rounding policy. `new BigDecimal("0.1")` is predictable; `new BigDecimal(0.1)` captures the binary floating-point approximation.

Operators and precedence

From high to low, a practical precedence summary is:

Family	Operators
postfix	<code>expr++, expr--</code>
unary	<code>++, --, +, -, ~, !, cast</code>
multiplicative	<code>*, /, %</code>
additive	<code>+, -</code>
shift	<code><<, >>, >>></code>
relational	<code><, <=, >, >=, instanceof</code>
equality	<code>==, !=</code>
bitwise	<code>&, then ^, then `</code>
conditional logical	<code>&&, then `</code>
conditional	<code>condition ? yes : no</code>
assignment	<code>=, compound assignments</code>

Use parentheses when a reviewer would otherwise have to recall the table. `&&` and `||` short-circuit; `&` and `|` evaluate both operands even for booleans. Compound assignment includes an implicit conversion:

`byteValue += 1` compiles where `byteValue = byteValue + 1` needs a cast.

`==` compares primitive values or reference identity. Semantic object equality is normally expressed by `equals`. Strings must not be compared with `==`.

Control flow and pattern matching

Basic forms:

JAVA

```
if (ready) {
    start();
} else {
    defer();
}

for (int i = 0; i < values.length; i++) {
    consume(values[i]);
}

for (String value : values) {
    consume(value);
}

while (condition()) {
    step();
}

do {
    step();
} while (condition());
```

A switch expression returns a value and should be exhaustive:

JAVA

```
enum State { NEW, RUNNING, DONE }

String label = switch (state) {
    case NEW -> "queued";
    case RUNNING -> "active";
    case DONE -> "complete";
};
```

The colon form retains fall-through semantics. Prefer arrow labels unless fall-through is deliberate and obvious.

Pattern variables are flow-scoped:

JAVA

```
if (candidate instanceof String text && !text.isBlank()) {
    System.out.println(text.length());
}
```

Record patterns and pattern matching for switch are final in Java 21. A sealed hierarchy lets the compiler check exhaustiveness when all permitted direct subtypes are known in the relevant compilation context.

Methods, overloads, and parameter passing

A method signature for overloading consists of the method name and parameter types. Return type alone cannot distinguish overloads. Resolution occurs at compile time using the declared argument types, applicability, conversions, and most-specific rules. Overriding is dynamic dispatch based on the runtime receiver type.

JAVA

```
static long sum(int left, long right) {
    return left + right;
}

static int sum(int... values) {
    int total = 0;
    for (int value : values) total += value;
    return total;
}
```

Varargs is an array parameter with call-site convenience. It can allocate an array and can create overload ambiguity. A caller may pass `null` as the entire array, so a public varargs method still needs a null policy.

Java is always pass-by-value. For an object argument, the copied value is a reference. The callee can use that copied reference to mutate the same mutable object, but assigning a different object to the parameter does not change the caller's variable.

Arrays, strings, and text

Arrays are reified, covariant objects with a fixed length:

JAVA

```
String[] names = new String[3];
int[][] matrix = new int[4][5];
int[] copy = java.util.Arrays.copyOf(values, values.length);
```

Covariance can fail at runtime: assigning a `String[]` to `Object[]` is legal, but storing a non-string throws `ArrayStoreException`. Generic collections are invariant and catch the analogous mismatch at compile time.

`String` is immutable. Concatenation in one expression is compiled using platform mechanisms selected by the compiler; repeated concatenation in a loop should normally use `StringBuilder`. `String.length()` counts UTF-16 code units, not user-perceived characters. A supplementary Unicode code point uses a surrogate pair. Use `codePoints()` when code points are the needed abstraction and a text library when grapheme clusters matter.

Common equality and emptiness checks:

JAVA

```
boolean same = java.util.Objects.equals(left, right);
boolean empty = text.isEmpty();
boolean blank = text.isBlank();
```

Classes, records, interfaces, and sealed hierarchies

Class skeleton:

JAVA

```
public final class Account {
    private final String id;
    private long balance;

    public Account(String id) {
        this.id = java.util.Objects.requireNonNull(id);
    }

    public synchronized void credit(long cents) {
        balance = Math.addExact(balance, cents);
    }
}
```

Access levels:

Modifier	Same class	Same package	Subclass in another package	Everywhere
private	yes	no	no	no
package-private	yes	yes	no	no
protected	yes	yes	qualified rules apply	no
public	yes	yes	yes	yes

Constructors are not inherited. A subclass constructor begins with an explicit or implicit invocation of another constructor. Static methods are hidden, not overridden. Private methods are not overridden. An overriding method may use a covariant return type and cannot broaden checked exceptions.

A record declares a nominal data carrier:

JAVA

```
public record Range(int start, int end) {
    public Range {
        if (start > end) throw new IllegalArgumentException("reversed");
    }
}
```

Records are shallowly immutable: component fields are final, but a referenced component can still be mutable. Make defensive copies when value semantics require them.

An interface defines a contract and can contain abstract, default, static, and private methods. An abstract class can hold instance state and protected implementation machinery. Prefer composition when the relationship is behavioral reuse rather than a stable subtype relationship.

Sealed types restrict direct subtypes:

JAVA

```
sealed interface Result permits Success, Failure {}
record Success(String value) implements Result {}
record Failure(String message) implements Result {}
```

Each permitted subtype must be `final`, `sealed`, or `non-sealed` unless its form, such as a record, is implicitly final.

Equality, hashing, and ordering

The `equals` contract requires reflexivity, symmetry, transitivity, consistency, and a false result for null. Equal objects must have equal hash codes. Unequal objects may collide. Fields used by `equals` and `hashCode` should not change while the object is a key in a hash-based collection.

`Comparable<T>` defines a natural ordering. `Comparator<T>` defines an external ordering and supports composition:

JAVA

```
Comparator<Person> order = Comparator
    .comparing(Person::lastName)
    .thenComparing(Person::firstName)
    .thenComparingInt(Person::age);
```

An ordering inconsistent with equality is legal for some APIs but can make a sorted set or map treat unequal values as the same key. Document the choice.

Exceptions and resource management

Checked exceptions are subclasses of `Exception` excluding `RuntimeException`; the compiler enforces catch-or-declare. Unchecked exceptions include `RuntimeException` and `Error`. Do not catch `Throwable` as routine application control flow.

JAVA

```
try (var input = java.nio.file.Files.newBufferedReader(path)) {
    return input.readLine();
} catch (java.io.IOException failure) {
    throw new UncheckedIOException("cannot read " + path, failure);
}
```

Try-with-resources closes initialized resources in reverse order. If both the body and close fail, the body failure is primary and close failures are suppressed. Inspect `getSuppressed()` when diagnosing layered resource failures.

Catch the narrowest useful type, preserve the cause when translating, and attach context without secrets. An interruption caught as `InterruptedException` should normally be propagated or restored with `Thread.currentThread().interrupt()`.

Generics and variance

Generic types are invariant: `List<Integer>` is not a subtype of `List<Number>`. Use bounded wildcards at API boundaries:

JAVA

```
static double total(java.util.List<? extends Number> source) {  
    return source.stream().mapToDouble(Number::doubleValue).sum();  
}  
static void addDefaults(java.util.List<? super Integer> sink) { /* write */ }
```

Mnemonic: producer extends, consumer super. A wildcard is not a declaration-site type parameter and may need capture by a helper method.

Type erasure means most type arguments are not reified at runtime. Consequently, Java prohibits `new T()`, `new T[]`, `instanceof List<String>`, and overloading methods whose signatures erase to the same form. Heap pollution occurs when a variable of a parameterized type refers to an incompatible value, often through raw types, unchecked casts, or generic varargs.

Prefer a generic method when the type relationship is local to one operation. Prefer a generic class when the type relationship belongs to the object's persistent state or contract.

Lambdas, method references, and streams

A lambda implements a functional interface, an interface with one abstract method after inheritance rules:

JAVA

```
java.util.function.Predicate<String> useful =  
    value -> value != null && !value.isBlank();  
java.util.function.Function<String, Integer> length = String::length;
```

Captured local variables must be final or effectively final. Capturing mutable state does not make access thread-safe.

Streams describe a lazy traversal pipeline. Intermediate operations are lazy; a terminal operation drives evaluation:

JAVA

```
Map<String, Long> counts = words.stream()  
    .filter(useful)  
    .map(String::toLowerCase)  
    .collect(java.util.stream.Collectors.groupingBy(  
        java.util.function.Function.identity(),  
        java.util.stream.Collectors.counting()));
```

Do not reuse a consumed stream. Avoid side effects in behavioral parameters. Parallel streams use shared runtime machinery and are not automatically faster; measure a representative, sufficiently large, CPU-oriented workload and account for blocking, ordering, splitting, and contention.

Concurrency recall sheet

Key rules:

- Starting a thread happens-before actions in that thread.
- All actions in a thread happen-before another thread detects its termination, including an untimed `join()` that returns.
- An unlock on a monitor happens-before a later lock on the same monitor.
- A volatile write happens-before a later read of that same field in synchronization order.
- Final fields receive special visibility guarantees when construction is correct and `this` does not escape.
- `volatile` supplies visibility and ordering for the field; it does not turn `count++` into one atomic action.
- Higher-level concurrent collection and executor methods have their own documented memory-consistency effects.

Prefer ownership, immutability, message passing, and well-defined concurrent components over scattered low-level synchronization. Every blocking operation needs a cancellation and shutdown story.

Java 21 virtual threads make thread-per-task practical for many blocking workloads. They do not remove resource limits, make CPU work parallel beyond available processors, or repair data races. Bound downstream resources such as connections and request quotas rather than pooling virtual threads merely to limit their count.

Common interview traps

- Java has no pass-by-reference parameter mode.
- `String` immutability does not make every object containing a string immutable.
- `final` freezes a variable binding or prevents overriding/inheritance in context; it is not a universal deep-immutability or thread-safety keyword.
- `volatile` is not mutual exclusion.
- `HashMap` has no general iteration-order contract.
- `PriorityQueue` exposes only its head ordering; iteration is not sorted.
- `Arrays.asList` returns a fixed-size list backed by the array.
- `List.subList` is a backed view whose structural validity depends on disciplined modification.
- `Stream.toList()` returns an unmodifiable list by contract; do not assume a concrete implementation.
- `Optional` is primarily a return-type tool, not a universal field, parameter, or serialization model.
- `Thread.sleep` does not release a held monitor.
- `wait` must be called while owning the monitor, releases that monitor while waiting, and belongs in a condition loop.
- A successful test run cannot prove a racy program correct.
- Big-O does not describe allocation, locality, constant costs, warm-up, contention, or tail latency.

Use this appendix for rapid recall, then return to the relevant chapter to recover the mechanism, proof obligation, trade-off, and production implication.

SDE-2 CORE CHAPTER

Chapter 25 - Seventy Compiling Java Fundamentals Examples

This dependency-free Java 21 companion contains seventy executable checks spanning the mandatory fundamentals and core collection usage. Compile with `javac -Xlint:all -Werror JavaFundamentalsExamples.java` and run with `java JavaFundamentalsExamples`. A successful run prints exactly PASS 70 Java Fundamentals examples.

JAVA (continued 1/21)

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.StringReader;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Scanner;
import java.util.Set;
import java.util.TreeSet;

/**
 * Seventy dependency-free Java 21 fundamentals examples.
 *
 * Each example is a small executable check. A successful run prints exactly:
 * PASS 70 Java Fundamentals examples
 */
public final class JavaFundamentalsExamples {
    private JavaFundamentalsExamples() {}

    public static void main(String[] args) {
        List<Runnable> examples = List.of(
            JavaFundamentalsExamples::example01FirstProgram,
            JavaFundamentalsExamples::example02VariableDeclaration,
            JavaFundamentalsExamples::example03PrimitiveRanges,
```


JAVA (continued 2/21)

```
JavaFundamentalsExamples::example04NumericPromotion,  
JavaFundamentalsExamples::example05SafeLongMultiplication,  
JavaFundamentalsExamples::example06WideningConversion,  
JavaFundamentalsExamples::example07NarrowingConversion,  
JavaFundamentalsExamples::example08IntegerDivision,  
JavaFundamentalsExamples::example09ShortCircuitEvaluation,  
JavaFundamentalsExamples::example10PrefixAndPostfix,  
JavaFundamentalsExamples::example11ConditionalStatements,  
JavaFundamentalsExamples::example12TraditionalSwitch,  
JavaFundamentalsExamples::example13SwitchExpression,  
JavaFundamentalsExamples::example14ForwardLoop,  
JavaFundamentalsExamples::example15ReverseLoop,  
JavaFundamentalsExamples::example16EnhancedForLoop,  
JavaFundamentalsExamples::example17NestedLoop,  
JavaFundamentalsExamples::example18MethodParametersAndReturn,  
JavaFundamentalsExamples::example19MethodOverloading,  
JavaFundamentalsExamples::example20PrimitivePassByValue,  
JavaFundamentalsExamples::example21MutableObjectPassByValue,  
JavaFundamentalsExamples::example22ReferenceReassignment,  
JavaFundamentalsExamples::example23OneDimensionalArray,  
JavaFundamentalsExamples::example24TwoDimensionalArray,  
JavaFundamentalsExamples::example25JaggedArray,  
JavaFundamentalsExamples::example26ArrayAliasing,  
JavaFundamentalsExamples::example27ArrayCopy,  
JavaFundamentalsExamples::example28StringEquality,  
JavaFundamentalsExamples::example29StringPool,  
JavaFundamentalsExamples::example30StringBuilder,  
JavaFundamentalsExamples::example31CharacterToDigit,  
JavaFundamentalsExamples::example32ClassAndObject,  
JavaFundamentalsExamples::example33ConstructorOverloading,  
JavaFundamentalsExamples::example34ConstructorChaining,  
JavaFundamentalsExamples::example35StaticFieldAndMethod,  
JavaFundamentalsExamples::example36Encapsulation,  
JavaFundamentalsExamples::example37ImmutableClass,  
JavaFundamentalsExamples::example38Inheritance,
```

JAVA (continued 3/21)

```
JavaFundamentalsExamples::example39MethodOverriding,  
JavaFundamentalsExamples::example40RuntimePolymorphism,  
JavaFundamentalsExamples::example41AbstractClass,  
JavaFundamentalsExamples::example42Interface,  
JavaFundamentalsExamples::example43Composition,  
JavaFundamentalsExamples::example44WrapperParsing,  
JavaFundamentalsExamples::example45AutoboxingAndUnboxing,  
JavaFundamentalsExamples::example46IntegerCaching,  
JavaFundamentalsExamples::example47GenericClassAndMethod,  
JavaFundamentalsExamples::example48ArrayList,  
JavaFundamentalsExamples::example49HashSet,  
JavaFundamentalsExamples::example50HashMapFrequency,  
JavaFundamentalsExamples::example51QueueWithArrayDeque,  
JavaFundamentalsExamples::example52StackWithArrayDeque,  
JavaFundamentalsExamples::example53PriorityQueue,  
JavaFundamentalsExamples::example54Enum,  
JavaFundamentalsExamples::example55CheckedException,  
JavaFundamentalsExamples::example56UncheckedException,  
JavaFundamentalsExamples::example57TryWithResources,  
JavaFundamentalsExamples::example58ScannerInput,  
JavaFundamentalsExamples::example59BufferedReaderInput,  
JavaFundamentalsExamples::example60InterviewQualityRefactoring,  
JavaFundamentalsExamples::example61ListOperations,  
JavaFundamentalsExamples::example62ListRemovalOverloads,  
JavaFundamentalsExamples::example63SetVariants,  
JavaFundamentalsExamples::example64MapUpdateApis,  
JavaFundamentalsExamples::example65MapIteration,  
JavaFundamentalsExamples::example66DequeConventions,  
JavaFundamentalsExamples::example67PriorityQueueComparator,  
JavaFundamentalsExamples::example68CollectionFactories,  
JavaFundamentalsExamples::example69SafeIteratorRemoval,  
JavaFundamentalsExamples::example70CopiesAndViews);  
  
examples.forEach(Runnable::run);  
System.out.println("PASS " + examples.size() + " Java Fundamentals examples");
```

JAVA (continued 4/21)

```
}

private static void check(boolean condition, String message) {
    if (!condition) {
        throw new AssertionError(message);
    }
}

static void example01FirstProgram() {
    String output = "Ready for Java";
    check(output.equals("Ready for Java"), "first program");
}

static void example02VariableDeclaration() {
    int solvedProblems = 10;
    solvedProblems = 11;
    check(solvedProblems == 11, "assignment");
}

static void example03PrimitiveRanges() {
    check(Byte.MIN_VALUE == -128 && Byte.MAX_VALUE == 127, "byte range");
    check(Integer.MIN_VALUE == -2_147_483_648, "int range");
}

static void example04NumericPromotion() {
    byte left = 100;
    byte right = 20;
    int sum = left + right;
    check(sum == 120, "byte operands promote to int");
}

static void example05SafeLongMultiplication() {
    long product = 100_000L * 100_000;
    check(product == 10_000_000_000L, "widen before multiply");
}
```

JAVA (continued 5/21)

```
static void example06WideningConversion() {
    int count = 10;
    long total = count;
    check(total == 10L, "widening");
}

static void example07NarrowingConversion() {
    double value = 10.8;
    int truncated = (int) value;
    check(truncated == 10, "narrowing truncates");
}

static void example08IntegerDivision() {
    check(5 / 2 == 2, "integer division");
    check(5 / 2.0 == 2.5, "floating division");
}

static void example09ShortCircuitEvaluation() {
    String text = null;
    boolean present = text != null && !text.isEmpty();
    check(!present, "short circuit null guard");
}

static void example10PrefixAndPostfix() {
    int value = 5;
    int before = value++;
    int after = ++value;
    check(value == 7 && before == 5 && after == 7, "increments");
}

static void example11ConditionalStatements() {
    int score = 72;
    String grade;
    if (score >= 70) grade = "pass";
}
```

JAVA (continued 6/21)

```
        else grade = "retry";
        check(grade.equals("pass"), "if else");
    }

    static String traditionalSwitch(int day) {
        String kind;
        switch (day) {
            case 6:
            case 7:
                kind = "weekend";
                break;
            default:
                kind = "weekday";
        }
        return kind;
    }

    static void example12TraditionalSwitch() {
        check(traditionalSwitch(7).equals("weekend"), "traditional switch");
    }

    static String switchExpression(int day) {
        return switch (day) {
            case 6, 7 -> "weekend";
            default -> "weekday";
        };
    }

    static void example13SwitchExpression() {
        check(switchExpression(1).equals("weekday"), "switch expression");
    }

    static void example14ForwardLoop() {
        int sum = 0;
        for (int value = 1; value <= 3; value++) sum += value;
    }
```

JAVA (continued 7/21)

```
        check(sum == 6, "forward loop");
    }

    static void example15ReverseLoop() {
        StringBuilder order = new StringBuilder();
        for (int index = 2; index >= 0; index--) order.append(index);
        check(order.toString().equals("210"), "reverse loop");
    }

    static void example16EnhancedForLoop() {
        int sum = 0;
        for (int number : new int[] {2, 4, 6}) sum += number;
        check(sum == 12, "enhanced for");
    }

    static void example17NestedLoop() {
        int cells = 0;
        for (int row = 0; row < 2; row++) {
            for (int column = 0; column < 3; column++) cells++;
        }
        check(cells == 6, "nested loop");
    }

    static int add(int left, int right) {
        return left + right;
    }

    static void example18MethodParametersAndReturn() {
        check(add(4, 5) == 9, "method result");
    }

    static int clamp(int value, int minimum, int maximum) {
        return Math.max(minimum, Math.min(value, maximum));
    }
}
```

JAVA (continued 8/21)

```
static long clamp(long value, long minimum, long maximum) {
    return Math.max(minimum, Math.min(value, maximum));
}

static void example19MethodOverloading() {
    check(clamp(12, 0, 10) == 10, "int overload");
    check(clamp(12L, 0L, 10L) == 10L, "long overload");
}

static void increment(int number) {
    number++;
    check(number == 8, "local primitive changed");
}

static void example20PrimitivePassByValue() {
    int caller = 7;
    increment(caller);
    check(caller == 7, "caller primitive unchanged");
}

static final class Person {
    String name;
    Person(String name) { this.name = name; }
}

static void rename(Person person) {
    person.name = "Updated";
}

static void example21MutableObjectPassByValue() {
    Person person = new Person("Original");
    rename(person);
    check(person.name.equals("Updated"), "shared mutation");
}
```

JAVA (continued 9/21)

```
static void replace(Person person) {
    person = new Person("New");
    check(person.name.equals("New"), "local reference reassigned");
}

static void example22ReferenceReassignment() {
    Person person = new Person("Caller");
    replace(person);
    check(person.name.equals("Caller"), "caller reference unchanged");
}

static void example23OneDimensionalArray() {
    int[] values = {2, 4, 6};
    check(values.length == 3 && values[1] == 4, "one dimensional array");
}

static void example24TwoDimensionalArray() {
    int[][] matrix = {{1, 2}, {3, 4}};
    check(matrix[1][0] == 3, "matrix");
}

static void example25JaggedArray() {
    int[][] rows = {new int[1], new int[3]};
    check(rows[0].length == 1 && rows[1].length == 3, "jagged array");
}

static void example26ArrayAliasing() {
    int[] first = {1, 2};
    int[] alias = first;
    alias[0] = 9;
    check(first[0] == 9, "array aliases share mutation");
}

static void example27ArrayCopy() {
    int[] original = {1, 2};
```


JAVA (continued 10/21)

```
int[] copy = Arrays.copyOf(original, original.length);
copy[0] = 9;
check(original[0] == 1, "independent primitive array copy");
}

static void example28StringEquality() {
    String left = new String("java");
    String right = new String("java");
    check(left != right && left.equals(right), "identity versus content");
}

static void example29StringPool() {
    String first = "pool";
    String second = "pool";
    String explicit = new String("pool");
    check(first == second && first != explicit, "pool and explicit construction");
}

static void example30StringBuilder() {
    String result = new StringBuilder().append("java").append('-').append(21).toString();
    check(result.equals("java-21"), "builder");
}

static void example31CharacterToDigit() {
    char character = '7';
    int asciiDigit = character - '0';
    int generalDigit = Character.digit(character, 10);
    check(asciiDigit == 7 && generalDigit == 7, "digit conversion");
}

static final class Student {
    final int id;
    String name;
    Student(int id, String name) { this.id = id; this.name = name; }
}
```

JAVA (continued 11/21)

```
static void example32ClassAndObject() {
    Student student = new Student(1, "Ada");
    check(student.id == 1 && student.name.equals("Ada"), "class and object");
}

static final class Rectangle {
    final int width;
    final int height;
    Rectangle() { this(1, 1); }
    Rectangle(int width, int height) { this.width = width; this.height = height; }
    int area() { return width * height; }
}

static void example33ConstructorOverloading() {
    check(new Rectangle().area() == 1, "no argument constructor");
    check(new Rectangle(3, 4).area() == 12, "parameter constructor");
}

static void example34ConstructorChaining() {
    Rectangle unit = new Rectangle();
    check(unit.width == 1 && unit.height == 1, "this constructor chaining");
}

static final class Counter {
    private static int created;
    Counter() { created++; }
    static int created() { return created; }
}

static void example35StaticFieldAndMethod() {
    int before = Counter.created();
    new Counter();
    new Counter();
}
```

JAVA (continued 12/21)

```
        check(Counter.created() == before + 2, "static shared count");
    }

    static final class BoundedCounter {
        private final int maximum;
        private int value;
        BoundedCounter(int maximum) { this.maximum = maximum; }
        void increment() { if (value == maximum) throw new IllegalStateException(); value++; }
        int value() { return value; }
    }

    static void example36Encapsulation() {
        BoundedCounter counter = new BoundedCounter(2);
        counter.increment();
        check(counter.value() == 1, "encapsulated invariant");
    }

    static final class Coordinate {
        private final int row;
        private final int column;
        Coordinate(int row, int column) { this.row = row; this.column = column; }
        int row() { return row; }
        int column() { return column; }
    }

    static void example37ImmutableClass() {
        Coordinate coordinate = new Coordinate(2, 3);
        check(coordinate.row() == 2 && coordinate.column() == 3, "immutable state");
    }

    static class Animal {
        String category() { return "animal"; }
    }
```

JAVA (continued 13/21)

```
static final class Dog extends Animal {
    @Override String category() { return "dog"; }
}

static void example38Inheritance() {
    Dog dog = new Dog();
    check(dog instanceof Animal, "is a relationship");
}

static void example39MethodOverriding() {
    Dog dog = new Dog();
    check(dog.category().equals("dog"), "override");
}

static void example40RuntimePolymorphism() {
    Animal animal = new Dog();
    check(animal.category().equals("dog"), "dynamic dispatch");
}

abstract static class Parser {
    abstract int parse(String text);
    int parsePositive(String text) {
        int value = parse(text);
        if (value <= 0) throw new IllegalArgumentException();
        return value;
    }
}

static final class DecimalParser extends Parser {
    @Override int parse(String text) { return Integer.parseInt(text); }
}

static void example41AbstractClass() {
    check(new DecimalParser().parsePositive("7") == 7, "abstract class");
}
```

JAVA (continued 14/21)

```
}

interface Formatter {
    String format(int value);
    default String label(int value) { return "value=" + format(value); }
}

static void example42Interface() {
    Formatter decimal = Integer::toString;
    check(decimal.label(9).equals("value=9"), "interface default method");
}

interface Clock {
    long nowMillis();
}

static final class ExpiringToken {
    private final Clock clock;
    private final long expiresAt;
    ExpiringToken(Clock clock, long expiresAt) { this.clock = clock; this.expiresAt = expiresAt; }
    boolean expired() { return clock.nowMillis() >= expiresAt; }
}

static void example43Composition() {
    Clock fixed = () -> 100L;
    check(new ExpiringToken(fixed, 90L).expired(), "composed clock");
}

static void example44WrapperParsing() {
    int value = Integer.parseInt("42");
    check(value == 42, "wrapper parsing");
}

static void example45AutoboxingAndUnboxing() {
```

JAVA (continued 15/21)

```
    Integer boxed = 7;
    int primitive = boxed;
    check(primitive == 7, "boxing and unboxing");
}

static void example46IntegerCaching() {
    Integer first = 127;
    Integer second = 127;
    Integer highFirst = 128;
    Integer highSecond = 128;
    check(first == second, "required small cache");
    check(highFirst.equals(highSecond), "outside cache compare by value");
}

static final class Box<T> {
    private final T value;
    Box(T value) { this.value = value; }
    T value() { return value; }
}

static <T> T first(List<T> values) {
    if (values.isEmpty()) throw new IllegalArgumentException("empty");
    return values.get(0);
}

static void example47GenericClassAndMethod() {
    Box<String> box = new Box<>("java");
    check(box.value().equals("java") && first(List.of(4, 5)) == 4, "generics");
}

static void example48ArrayList() {
    List<String> values = new ArrayList<>();
    values.add("a");
    values.add("b");
}
```

JAVA (continued 16/21)

```
        check(values.get(1).equals("b"), "array list");
    }

    static void example49HashSet() {
        Set<Integer> values = new HashSet<>(List.of(2, 2, 3));
        check(values.size() == 2, "set deduplication");
    }

    static void example50HashMapFrequency() {
        Map<String, Integer> frequency = new HashMap<>();
        for (String word : List.of("java", "dsa", "java")) {
            frequency.merge(word, 1, Integer::sum);
        }
        check(frequency.get("java") == 2, "frequency map");
    }

    static void example51QueueWithArrayDeque() {
        Deque<Integer> queue = new ArrayDeque<>();
        queue.offerLast(1);
        queue.offerLast(2);
        check(queue.pollFirst() == 1, "fifo queue");
    }

    static void example52StackWithArrayDeque() {
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(1);
        stack.push(2);
        check(stack.pop() == 2, "lifo stack");
    }

    static void example53PriorityQueue() {
        PriorityQueue<Integer> heap = new PriorityQueue<>(List.of(9, 3, 5));
        check(heap.poll() == 3, "min heap");
    }
```

JAVA (continued 17/21)

```
enum OrderStatus {
    CREATED, PAID, SHIPPED, DELIVERED, CANCELLED;
    boolean terminal() { return this == DELIVERED || this == CANCELLED; }
}

static void example54Enum() {
    check(OrderStatus.DELIVERED.terminal(), "enum behavior");
}

static int parsePositive(String text) throws IOException {
    int value;
    try {
        value = Integer.parseInt(text);
    } catch (NumberFormatException exception) {
        throw new IOException("invalid integer", exception);
    }
    if (value <= 0) throw new IOException("not positive");
    return value;
}

static void example55CheckedException() {
    try {
        check(parsePositive("4") == 4, "checked exception result");
    } catch (IOException exception) {
        throw new AssertionError(exception);
    }
}

static void requirePositive(int value) {
    if (value <= 0) throw new IllegalArgumentException("positive required");
}

static void example56UncheckedException() {
```


JAVA (continued 18/21)

```
        boolean thrown = false;
        try {
            requirePositive(0);
        } catch (IllegalArgumentException exception) {
            thrown = true;
        }
        check(thrown, "unchecked precondition");
    }

    static final class FlagResource implements AutoCloseable {
        private final boolean[] closed;
        FlagResource(boolean[] closed) { this.closed = closed; }
        @Override public void close() { closed[0] = true; }
    }

    static void example57TryWithResources() {
        boolean[] closed = {false};
        try (FlagResource ignored = new FlagResource(closed)) {
            check(ignored != null && !closed[0], "open in body");
        }
        check(closed[0], "closed after body");
    }

    static void example58ScannerInput() {
        try (Scanner scanner = new Scanner("3 4 5 6")) {
            int length = scanner.nextInt();
            int sum = 0;
            for (int index = 0; index < length; index++) sum += scanner.nextInt();
            check(sum == 15, "scanner input");
        }
    }

    static void example59BufferedReaderInput() {
        try (BufferedReader reader = new BufferedReader(new StringReader("7 8\n"))) {
```

JAVA (continued 19/21)

```
        String[] tokens = reader.readLine().split("\\s+");
        int sum = Integer.parseInt(tokens[0]) + Integer.parseInt(tokens[1]);
        check(sum == 15, "buffered reader input");
    } catch (IOException exception) {
        throw new AssertionError(exception);
    }
}

static long calculateSum(int[] numbers) {
    long sum = 0L;
    for (int number : numbers) sum += number;
    return sum;
}

static void example60InterviewQualityRefactoring() {
    int[] numbers = {Integer.MAX_VALUE, 1};
    check(calculateSum(numbers) == 2_147_483_648L, "clear name and safe accumulator");
}

static void example61ListOperations() {
    List<Integer> values = new ArrayList<>(List.of(10, 30));
    values.add(1, 20);
    values.set(0, 5);
    check(values.equals(List.of(5, 20, 30)), "list operations");
}

static void example62ListRemovalOverloads() {
    List<Integer> values = new ArrayList<>(List.of(10, 20, 30));
    values.remove(1);
    values.remove(Integer.valueOf(30));
    check(values.equals(List.of(10)), "index and value removal");
}

static void example63SetVariants() {
```

JAVA (continued 20/21)

```
Set<Integer> insertionOrdered = new LinkedHashSet<>(List.of(3, 1, 3, 2));
Set<Integer> sorted = new TreeSet<>(insertionOrdered);
check(insertionOrdered.toString().equals("[3, 1, 2]"), "linked set order");
check(sorted.toString().equals("[1, 2, 3]"), "tree set order");
}

static void example64MapUpdateApis() {
    Map<String, List<Integer>> groups = new HashMap<>();
    groups.computeIfAbsent("odd", ignored -> new ArrayList<>()).add(3);
    Map<String, Integer> counts = new HashMap<>();
    counts.merge("java", 1, Integer::sum);
    counts.merge("java", 1, Integer::sum);
    check(groups.get("odd").equals(List.of(3)) && counts.get("java") == 2,
           "map update APIs");
}

static void example65MapIteration() {
    Map<String, Integer> ordered = new LinkedHashMap<>();
    ordered.put("a", 1);
    ordered.put("b", 2);
    int sum = 0;
    for (Map.Entry<String, Integer> entry : ordered.entrySet()) {
        check(!entry.getKey().isEmpty(), "entry key");
        sum += entry.getValue();
    }
    check(sum == 3, "entry iteration");
}

static void example66DequeConventions() {
    Deque<Integer> deque = new ArrayDeque<>();
    deque.offerLast(1);
    deque.offerLast(2);
    check(deque.pollFirst() == 1, "queue convention");
    deque.push(3);
}
```

JAVA (continued 21/21)

```
        check(deque.pop() == 3, "stack convention");
    }

    static void example67PriorityQueueComparator() {
        PriorityQueue<Integer> maximums = new PriorityQueue<>(Comparator.reverseOrder());
        maximums.addAll(List.of(Integer.MIN_VALUE, 0, Integer.MAX_VALUE));
        check(maximums.poll() == Integer.MAX_VALUE, "safe max heap comparator");
    }

    static void example68CollectionFactories() {
        List<String> fixedSize = Arrays.asList("a", "b");
        fixedSize.set(0, "x");
        List<String> unmodifiable = List.of("a", "b");
        check(fixedSize.equals(List.of("x", "b")) && unmodifiable.size() == 2,
            "factory capabilities");
    }

    static void example69SafeIteratorRemoval() {
        List<Integer> values = new ArrayList<>(List.of(-1, 0, 2));
        for (Iterator<Integer> iterator = values.iterator(); iterator.hasNext();) {
            if (iterator.next() < 0) iterator.remove();
        }
        check(values.equals(List.of(0, 2)), "iterator removal");
    }

    static void example70CopiesAndViews() {
        List<String> source = new ArrayList<>(List.of("a"));
        List<String> snapshot = List.copyOf(source);
        List<String> view = Collections.unmodifiableList(source);
        source.add("b");
        check(snapshot.equals(List.of("a")), "copy is independent");
        check(view.equals(List.of("a", "b")), "view reflects source");
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: trace types, expressions, methods, arrays, strings, objects, and collection operations
- Interview Core: compile the 70-example companion and complete output/debugging drills
- SDE-2 Follow-up: expose contracts, aliases, failure behavior, and collection trade-offs
- Challenge: pass the cumulative and final readiness assessments without notes

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

This is the first volume in the Java Engineering segment.

[Series index](#)

Next book: [JAVA 02 - Git and GitHub for Java Engineers](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Java Engineering - Book 01 of 09 - Study Step JAVA-01. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.